



coreConsultant

User Guide

A coreTools Application

Copyright Notice and Proprietary Information

© 2024 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History.....7

Preface.....17

 About This User Guide.....17

 Related Documents.....17

 Manual Overview.....18

 Typographical and Symbol Conventions.....18

 Synopsys Common Licensing (SCL).....19

 Synopsys Statement on Inclusivity and Diversity.....19

 Customer Support.....19

Chapter 1 Introduction.....21

 1.1. Design Reuse with Synopsys coreTools.....22

 1.1.1. coreConsultant Process Activities.....22

 1.2. Key Terms and Concepts.....26

Chapter 2 The coreConsultant Environment.....29

 2.1. coreKits and Workspaces.....30

 2.2. Activity-Based Design Flow.....31

 2.3. User Interfaces.....33

 2.3.1. Command Line Mode.....33

 2.3.2. Batch Mode.....33

 2.3.3. GUI Mode.....33

 2.4. Online Help Tools.....35

 2.5. coreConsultant Spreadsheets.....36

 2.6. Basic Operations.....37

 2.6.1. Invoking coreConsultant.....37

 2.6.2. Installing coreKits.....39

 2.6.3. Component Update Checking.....40

 2.6.4. Workspace Operations.....43

 2.6.5. Exiting coreConsultant.....45

 2.6.6. Working with the Activity List.....45

2.6.7. Generating Reports and Optional Views.....	46
Chapter 3 Creating the RTL View of a Core.....	47
3.1. Configuring a Core.....	48
3.1.1. Setting the Design, File, and Macro Prefixes.....	48
3.1.2. Select Prime Profile.....	48
3.1.3. Specifying Configuration.....	56
3.1.4. Area Estimate Report.....	57
Chapter 4 Creating the Gate-Level Netlist for a Core.....	59
4.1. Specifying Target Technology (TechSetup).....	60
4.1.1. Logical Library Setup.....	60
4.1.2. Physical Library Setup.....	61
4.1.3. Set Tool Installation Root.....	61
4.1.4. Library Setup in Command Line Mode.....	62
4.2. Technology Cell Binding.....	63
4.2.1. Tech Cell Mapping Use Case.....	64
4.3. Specifying Clocks.....	67
4.4. Specifying Operating Conditions.....	68
4.5. Specifying Port Constraints.....	70
4.6. Specifying Power Intent.....	73
4.7. Specifying Synthesis Methodology.....	75
4.7.1. General Synthesis Attributes.....	75
4.7.2. Advanced Synthesis Attributes.....	76
4.8. Specifying Test Methodology.....	78
4.8.1. General Test Methodology Attributes.....	78
4.8.2. Scan Insertion.....	78
4.8.3. Auto Fix.....	81
4.8.4. Test Wrappers.....	82
4.8.5. Wrapper Integration.....	83
4.8.6. Built-in Self Test.....	84
4.8.7. Testpoint Insertion.....	86
Chapter 5 Synthesizing a Core.....	89
5.1. Performing ASIC Synthesis.....	90
5.2. Synthesis and Integration Methodologies.....	91
5.2.1. Working with Design Compiler Strategies.....	91
5.2.2. Working with Fusion Compiler Strategies.....	91

5.2.3. Working with Physical Synthesis Strategies.....	92
5.2.4. Generating Timing Models.....	93
5.2.5. Integrating Generated Scripts.....	93
5.3. Choosing Synthesis Activities in the GUI.....	94
5.3.1. Choosing a Synthesis Strategy.....	94
5.3.2. Specifying Synthesis Execution Options.....	122
5.3.3. Viewing Licenses for Implementation Tools.....	123
5.3.4. Selecting Synthesis Reports.....	124
5.4. Launching a Synthesis Run.....	127
5.4.1. Performing ASIC Synthesis using Design Compiler.....	127
5.4.2. Performing ASIC Synthesis using Fusion Compiler.....	128
5.5. Checking Synthesis Results.....	133
5.6. Refining Your Synthesis Intent.....	134
5.6.1. Correcting Timing Problems.....	134
5.7. Generating ATPG Test Vectors.....	135
Chapter 6 Verifying Your Implementation.....	141
6.1. Overview.....	142
6.2. Formal Verification.....	144
6.2.1. Executing the Formal Verification Activity.....	144
6.2.2. Checking Formal Verification Results.....	148
6.3. Static Timing Analysis Interface.....	149
6.3.1. Checking Static Timing Analysis Results.....	156
6.4. Explore RTL Architecture Activity.....	157
6.5. Timing Constraints Manager.....	162
6.5.1. Constraints Verification.....	163
6.5.2. Constraints Verification (CV) Activity.....	164
6.5.3. Constraints Verification with Assertions (CVA) Activity.....	166
6.5.4. Running the TCM Flow.....	167
6.5.5. Constraints Verification Directory Tree Structure.....	168
6.6. Setup and Run Simulations.....	169
6.7. Integrating the Core Into Your Design.....	170
6.7.1. Connecting to Surrounding Logic.....	170
6.7.2. Performing System-Level Synthesis.....	170
6.7.3. Verifying Functionality.....	170
Chapter 7 Advanced User Modes and Features.....	171

7.1. Command Line Interface.....	172
7.1.1. Accessing the Command Line Interface.....	172
7.1.2. Setting the Value of a Design Configuration Parameter.....	172
7.1.3. Reporting Activity Parameters.....	172
7.1.4. Setting the Value of an Activity Parameter.....	173
7.1.5. Setting the Value of a Synthesis Strategy Parameter.....	173
7.1.6. Re-Executing the coreConsultant Design Flow.....	173
7.1.7. Abbreviating Commands.....	174
7.2. Batch Mode.....	175
7.2.1. Batch Command File Examples.....	175
7.2.2. Automatically Generating a Batch File.....	175
7.2.3. Batch File Options.....	176
7.2.4. Executing a Batch Command File.....	176
7.2.5. Batch Mode coreKit Installation.....	176
7.3. Editing Tool Setup and User Preferences.....	177
7.3.1. Setup Variables.....	177
7.3.2. User Preferences.....	178
7.4. Working with Verification IP.....	190
7.5. Working with Hard Macros.....	191
7.6. Dumping Debug Information.....	192
7.7. Documentation Generation.....	193
7.8. Table-Driven Data Import/Export.....	194
7.8.1. General Table Formatting (All coreTools Applications).....	194
7.8.2. Table Commands.....	195
Chapter 8 Customizing a Technology Library.....	199
8.1. Library Customization Attributes.....	201
8.2. Cell Customization Attributes.....	202

Revision History

The following table provides a summary of changes made to this User Guide.

**Note**

Links and references to section, table, figure, and page numbers in this table are only assured to be valid for the version in which the change is made.

Version	Date	Description
W-2024.09-1	October 2024	Updated: ■ Table 6-10 on page 157 with "Utilization Ratio" activity parameter details.
W-2024.09	September 2024	■ Updated: □ Table 4-3 on page 63 with "Invert connection", "Pins Connected to High or Low" and "CTL Model File" fields. □ Table 4-4 on page 65 with a new use case. □ Table 7-3 on page 179 with Include Doc attributes GUI details. ■ Added: □ Figure 4-1 on page 64
V-2024.06-2	June 2024	Updated version
V-2024.06	June 2024	■ Updated: □ Table 4-5 on page 62 □ ScanCompressionUsage and EnableTestMaxManagerFlow strategy parameter descriptions in Table 5-11 on page 108 ■ Added SeqCompressionInputs and SeqCompressionOutputs strategy parameters in 5-11 on page 108
V-2024.03-1	May 2024	Updated version and date
		■ Removed the remote option for Runstyle

Version	Date	Description
V-2024.03	March 2024	■ Changed instances of DesignWare Cores to Synopsys IP to align with corporate naming conventions. ■ Updated: □ “Select Prime Profile” on page 46 □ “Timing Constraints Manager” on page 154 □ Table 6-14 on page 158 and Table 6-15 on page 159 □ Figure 5-2 on page 121 ■ Added: □ “Prime Profile Reports” on page 53 □ ParallelJobCpuLimit activity parameter in Table 6-14 on page 158 □ EnableDistributedCodec strategy parameter in Table 5-11 on page 103
V-2023.12-1	January 2024	Updated version and date
V-2023.12	December 2023	■ Added: □ include_defines_in_fileset and include_undefined_macros_in_ipxact preference parameters in Table 7-3 on page 172 □ GenerateRMReports and DisableCongestionReport strategy parameters in Table 5-9 on page 98 and Table 5-20 on page 110 □ ReportQOR and ReportQORHistogram strategy parameters in Table 5-20 on page 110 □ EnableTestMaxManagerFlow and EnableTPI strategy parameters in Table 5-11 on page 103 □ “Performing ASIC Synthesis using QIK Reference Methodology with TestMAX Manager” on page 125 □ ReportAnalysisCoverageCheckType activity parameter in Table 6-8 on page 146 ■ Removed: □ “Performing FPGA Synthesis” section in “Synthesizing a Core” on page 81 □ Synplify Library Setup (FPGA)” section in “Creating the Gate-Level Netlist for a Core” on page 57 □ FPGA keywords applicable in the document ■ Updated: □ “Synthesizing a Core” on page 81 □ Table 2-2 on page 29 to remove the “Perform FPGA Synthesis” activity □ “Select Prime Profile” on page 46 with prime profile groups

Version	Date	Description
V-2023.09-1	September 2023	Updated version
V-2023.09	September 2023	■ Removed the Constraint Analysis Activity section in Chapter 6, “Verifying Your Implementation” ■ Added: □ EnableQIK, FinalFCRMFlowStep, TechnologySideFileSettings, and TechnologyRMScripts parameters in Table 5-10 on page 91 □ QIK parameters in Table 5-14 on page 98, Table 5-15 on page 99, Table 5-16 on page 99, Table 5-17 on page 100, and Table 5-18 on page 101 □ “Performing ASIC synthesis using QIK Reference Methodology for DesignWare Interface IP” on page 114 □ “Timing Constraints Manager” on page 146
U-2023.06-1	June 2023	Updated version
U-2023.06	June 2023	■ Added: □ “Select Prime Profile” on page 44 □ “Performing ASIC Synthesis using Design Compiler” on page 103 and “Performing ASIC Synthesis using Fusion Compiler” on page 104 □ “Working with Fusion Compiler Strategies” on page 75 ■ Updated: □ Table 2-2 on page 27 with “Select Prime Profile” Activity □ “Launching a Synthesis Run” on page 103 with synthesis using Design Compiler and Fusion Compiler. □ “Working with Physical Synthesis Strategies” on page 76 with a note
U-2022.12-2	February 2023	Added SetTechnologyAfterFloorplan strategy parameter in Table 5-9 on page 88
U-2022.12	December 2022	■ Added: □ ExploreFlowRunStyle parameter in Table 6-12 on page 134 □ AlternateStrategiesRunStyle parameter in Table 6-2 on page 121 □ DPXRunStyle parameter in Table 6-3 on page 122 □ DCRM DFT Parameters in Table 5-5 on page 82 □ FCRM DFT Parameters in Table 5-10 on page 92 □ UseSynplifyElite and UseSynplifyBase parameters in Table 5-22 on page 114 □ FCRM power parameters in Table 5-12 on page 94 ■ Updated parameter type in table headings and column in all tables

Version	Date	Description
T-2022.06-SP2	September 2022	■ Updated CTLModels parameter description in the Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 ■ Added: □ CTLModels DCRM strategy parameter in Table 5-2 on page 78 □ NDM Library physical synthesis DCRM strategy parameters in Table 5-6 on page 84
T-2022.06-SP1	July 2022	■ Added: □ CTLModels parameter in the Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 □ DPX flow in Table 6-3 on page 122 □ EnableDesignPlanFlatFlow FCRM Strategy parameter in Table 5-9 on page 90 □ DFTSynthesisOptimization DCRM strategy parameter in Table 5-2 on page 78
T-2022.06	June 2022	■ Added ScanCompressionTestClock parameter to Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90
S-2021.09-SP4	April 2022	■ Added: □ OverrideSearchPath, OverrideLinkLibraryMapping, and OverrideDefaultOperatingCondition parameters for the Static Timing Analysis interface in Table 6-7 on page 127
S-2021.09-SP3	March 2022	■ Added SelfGating parameter in Table 5-7 on page 87 and DcSelfGatingReport parameter in Table 5-16 on page 100
S-2021.09-SP2	January 2022	■ Added: □ CongestionOptimizationFactor parameter to the Design_Compiler_Reference_Methodology strategy in Table 5-6 on page 84 □ EnableECOFlow parameter to Table 6-6 on page 125 □ FixECOTimingArgs parameter to Table 6-6 on page 125 □ ReducedEffortOptimizationFlow parameter to Table 5-2 on page 78 ■ Updated all instances of TetraMAX to TestMAX and LinkToTetraMax to LinkToATPG in the document. ■ Updated the OptimizationFlow parameter description in Table 5-2 on page 78

Version	Date	Description
S-2021.09-SP1	October 2021	■ Added EnablePortSideConstraint parameter, MinRoutingLayer parameter, and MaxRoutingLayer parameter to the Design_Compiler_Reference_Methodology strategy in Table 5-6 on page 84 ■ Added EnablePortSideConstraint parameter to Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 ■ Added MinRoutingLayer parameter and MaxRoutingLayer parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-11 on page 94 ■ Added GenerateIgnoredLayersReport parameter to Table 5-16 on page 100
S-2021.09	September 2021	■ Added the UtilizationRatio parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 and Design_Compiler_Reference_Methodology strategy in Table 5-6 on page 84 ■ Added the TechnologyNode parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 and Design_Compiler_Reference_Methodology strategy in Table 5-2 on page 78 ■ Added write_absolute_dependency_path preference attribute to the GUI Preferences in Table 7-3 on page 144
R-2020.12-SP5	July 2021	■ Added the EnableLibraryConfigurationFlow parameter to the Basic settings for the Fusion Compiler Reference Methodology in Table 5-9 on page 90 ■ Added Control Files parameters to the Constraint Analysis activity in Table 6-11 on page 132
R-2020.12-SP4-1	June 30, 2021	■ No technical changes
R-2020.12-SP4	June 2021	■ Added the UseTechFile parameter to the Fusion_Compiler_Reference_Methodology strategy in Table 5-9 on page 90 ■ Added the GenerateAreaReport to the synthesis reports in Table 5-16 on page 100 ■ Added InsertDft, DFTFlowType, and UseTechFile parameters to the Explore RTL Architecture activity in Table 6-10 on page 132 ■ Added accessHandle information to the IP-XACT preferences in Table 7-3 on page 144

Version	Date	Description
R-2020.12-SP3-1	June 2021	■ Added QoR strategy and effort level to the Fusion_Compiler_Reference_Methodology parameters in Table 5-9 on page 90 ■ Added parameters for the Multi Vth synthesis constraint for the Design Compiler tool in Table 5-6 on page 84 and for the Fusion Compiler tool in Table 5-11 on page 94 ■ Added the ReadConstraints and WriteICC2Files parameters to the DCTCL_quick_map_strategy in Table 5-14 on page 96 ■ Added alternate strategies parameters to the Formal Verification activity in Table 6-2 on page 121
R-2020.12-SP3	April 2021	■ Added the TransitionDelayCoverageGoal parameter to Table 5-20 on page 110
R-2020.12-SP2	March 2021	■ Clarified the contents of workspaces and added Figure 2-1 on page 26 ■ In “Technology Cell Binding” on page 52, updated the bind_cell_mapping command to bind_tech_cell_mapping ■ Added “Tech Cell Mapping Use Case” on page 53 ■ Added automotive synthesis control in “Design_Compiler_Reference_Methodology” on page 78 and “Fusion_Compiler_Reference_Methodology” on page 90 ■ Added Automotive Reports option in Table 5-16 on page 100
R-2020.12-SP1	January 2021	■ Defined the term TechSetup in Table 1-2 on page 23 and on page 50 ■ Added “Explore RTL Architecture Activity” on page 131
R-2020.12	December 2020	■ Updated “Logical Library Setup” on page 50 to clarify Fusion Compiler library details ■ Added new activity in “Technology Cell Binding” on page 52 ■ Clarified the conditions under which the Fusion Compiler Reference Methodology synthesis strategy is available in “Fusion_Compiler_Reference_Methodology” on page 90
Q-2020.03-SP4	September 2020	■ Added the DFT Flow Type and Multi Voltage Setup File parameters for the Fusion Compiler Reference Methodology in Table 5-9 on page 90
Q-2020.03-SP3	July 2020	■ In Table 5-9 on page 90, made the following changes to the Fusion_Compiler_Reference_Methodology parameters: □ Added FloorplanFileDataFormat □ Updated TCLPhysicalConstraintFile □ Added EnableUPFFlow

Version	Date	Description
		❑ Added UPFFile ❑ Added PowerConstraintsFile
Q-2020.03-SP2	June 2020	■ Added Fusion Compiler in areas where synthesis tool support is noted ■ Added details for search_path in Table 4-1 on page 50 ■ Added Fusion Compiler synthesis control attributes in Table 4-12 on page 61 and removed or added others ■ Removed Table 5-31, “Working with DC Incremental Strategies” ■ In “Choosing Synthesis Activities in the GUI” on page 77, removed the following sections: ❑ 5.3.1.2 DCTCL_high_effort_incr ❑ 5.3.1.3 DCTCL_area_recovery ❑ 5.3.1.4 DCTCL_design_rule_fix and added the following section: ❑ “Fusion_Compiler_Reference_Methodology” on page 90 ■ Removed the following sections: ❑ 5.6.1.1 Few Small Violations ❑ 5.6.1.2 Few Large Violations ❑ 5.6.2 Correcting Area Problems
P-2019.06-SP4	December 2019	■ Added the ReportClockGateSavings report option in Table 6-8 on page 128
P-2019.06-SP3	October 2019	■ Added Design Compiler NXT to synthesis options in Table 5-15 on page 98 ■ Added fan-in/fan-out report and DFT coverage estimate report options in Table 5-16 on page 100 ■ Added TestMAX™ ATPG to the ATPG test vector options in Table 5-19 on page 108
O-2018.09-SP4	April 2019	■ Added the SDC Generation option in Table 5-2 on page 78 ■ Updated descriptions of GatingAndFunction and GatingOrFunction in Table 5-7 on page 87 ■ In the Assembly section of Table 7-3 on page 144: ❑ Updated a GUI label ❑ Added a preference for controlling auto connection of interfaces
O-2018.09-SP2	December 2018	■ Updated Table 5-15 on page 98 ■ Enhanced “Editing Tool Setup and User Preferences” on page 143 and added Table 7-3 on page 144 to explain preferences

Version	Date	Description
O-2018.06-SP1	October 2018	Removed the HAPS entry from Table 4-3 on page 51
O-2018.09	September 2018	Documented the “Constraint Analysis Activity” on page 132
N-2017.12-SP4	June 2018	■ Added macro prefixes to the following material: □ Figure 1-1 on page 21 □ Table 2-2 on page 27 □ “Setting the Design, File, and Macro Prefixes” on page 44 ■ Updated “Setting Workspace Options” on page 39 ■ Added the Write ICC2 Files parameter in Table 5-2 on page 78
N-2017.12-SP3	April 2018	Added At Speed Test parameters to Table 5-20 on page 110
N-2017.12-SP2	March 2018	Version change
N-2017.12-SP1	January 2018	Added the Extend MW Layers parameter to Table 5-6 on page 84 Added the Use the scan compression Protocol parameter to Table 5-20 on page 110
N-2017.12	December 2017	Version change
M-2017.06-SP3	October 2017	Added Optimization flow parameter to Table 5-2 on page 78
M-2017.06-SP2	September 2017	Version change
M-2017.06-SP1	July 2017	Version change
M-2017.06	June 2017	Added support for Golden UPF Flow; see Table 5-4 on page 81, Table 6-1 on page 120, and Table 6-5 on page 124.
L-2016.09-SP4	April 2017	Version change
L-2016.09-SP3	March 2017	Version change
L-2016.09-SP2	December 2016	Version change
L-2016.09-SP1	October 2016	Added the AddPiConstraints parameter to the TMAX Reference Methodology strategy in Table 5-20 on page 110
L-2016.09	September 2016	Added the TetraMAX Reference Methodology strategy; Table 5-20 on page 110
K-2015.06-SP7	July 2016	Added scan-related attributes; p. 64
K-2015.06-SP6	June 2016	Version change
K-2015.06-SP5-2	April 29, 2016	Version change
K-2015.06-SP5-1	April 2016	Version change
K-2015.06-SP5	March 2016	Reduced the synthesis strategies available and adjusted associated text; p. 75 - p. 107. Updated Figure 5-1 on page 73.

Version	Date	Description
K-2015.06	June 2015	Added the EnableSPG parameter; p.84
J-2015.03	March 2015	Added Specify Power Intent activity, p. 59
J-2014.12	December 2014	IP-XACT updates, Table 1-2, and Table 7-3.
I-2014.03-SP2	July 2014	Updates to Table 6-6 multiple files for SwitchingActivityFile parameter.
I-2014.03	March 2014	Updates to Table 4-12 and new dcrmPathGroupScript attribute.
I-2013.09	September 2013	Major release, updated version, date, and copyright.
H-2013.03-SP1	April 2013	Added StaticTimingAuxScript to Specify Synthesis Methodology, p. 61; and to Static Timing Analysis activity "Analysis Setup" tab, p. 125
H-2013.03	March 2013	Added UVM variant option to RAL generation; ReportRegisterInfo and ReportSyncFlops added, p. 128; Update copyright page and date/version.
G-2012.06-SP3	January 2013	Updated Reference Methodology Synthesis information, p. 78; Added DC Explorer option to DCTCL_quick_map_strategy, p. 95. Updated Date and Version information.
G-2012.06-SP2	October 2012	Fixed broken link to Licensing Information on www.synopsys.com
G-2012.06	June 2012	Added Table-driven Data Import/Export. p. 156 SDC supports latest version.
F-2011.12-SP2	April 2012	Added Physical Synthesis "FloorplanFileDataFormat", "PhysicalConstraintFile" strategy parameters, and also hid "TclPhysicalConstraintFile" "DefPhysicalConstraintFile" parameters. p. 84
F-2011.12	December 2011	New default synthesis strategy Design_Compiler_Reference_Methodology (renamed from DCTCL_one_pass_compile_ultra) p. 78
F-2011.06-SP2	October 2011	Added Physical Synthesis "WriteMilkywayCell" strategy parameter. p. 84
F-2011.06	June 2011	Various edits.
E-2010.09-SP2	January 2011	Removed Physical Synthesis controls and psyn_shell.
E-2010.09	September 2010	Separate FPGA synthesis activity, p. 114; HAPS board selection (FPGA synthesis), p. 51; Place & Route controls (FPGA synthesis), p. 115; Area Estimation reporting, p. 47;
D-2010.03-SP2	July 2010	Static Timing Analysis -- SuppressDuplicateBusPaths added, p. 128;
D-2010.03-SP1	April 2010	Setting a 'file' prefix, p. 44
D-2010.03	March 2010	IEEE P1685 (IP-XACT v1.5) support, p. 23; Formality analyze_points added, p. 123; Build Debug Tarfile enhanced, p. 154; Synplify (FPGA synthesis) DC DWBB Lib, p. 114
C-2009.06-SP2	January 2010	Copyright update.

Version	Date	Description
C-2009.06-SP1	October 2009	Activity report generation, p. 41; Design views generation, p. 41
C-2009.06-2	July 2009	DcTimingExceptionsReport default=0, p. 100; Static Timing Analysis ReportExceptions default=1, p. 128
C-2009.06	June 2009	Static Timing interface, p. 124; UPF file support, p. 80,81; UPF Support in Formality, p. 123; Batch script enhancements, p. 141; Documentation generation, p. 155
B-2008.12-SP2	April 2009	Added Tool Installation Root information about 64-bit mode, Synplify FPGA synthesis strategies.
B-2008.12	December 2008	Added support for Synplify, new synthesis strategies for DC Ultra and DC reference methodology. Remove DC-FPGA support. Tool root reset. GUI mode for update_workspace_links. Dump debug information.
B-2008.06-SP3	October 2008	Updates to synthesis strategies (see “Choosing a Synthesis Strategy” on page 77). Minor text edits.
B-2008.06-SP2-1	July 10, 2008	p.104 restore_checkpoint information added.
B-2008.06	June 2008	IP-XACT v1.4 - See the coreTools Release Notes for update details. NOTE: Minor “SPx” releases may only update Release Notes document.

Preface

This chapter contains the following sections:

- [“About This User Guide”](#) on page 17
- [“Related Documents”](#) on page 17
- [“Manual Overview”](#) on page 18
- [“Typographical and Symbol Conventions”](#) on page 18
- [“Synopsys Statement on Inclusivity and Diversity”](#) on page 19
- [“Customer Support”](#) on page 19

About This User Guide

This user guide is for designers who use coreConsultant to perform configuration, simulation, and verification tasks with a reusable core. You should be familiar with the following topics:

- IC design methodology including the concept of integrating blocks into a larger design.
- A hardware description language (VHDL or Verilog).
- Synthesis with Synopsys Design Compiler or Design Compiler Topographical.
- Design verification, configurable testbenches, and VHDL/Verilog simulation tools.

Related Documents

This user guide is part of the coreTools document set. Information about the coreTools is available from the following sources and from the online help system on the Help menu of each tool:

- **General Documents**
 - *Guide to coreTools Documentation*
 - *coreTools Release Notes*
- **Working with Cores or coreKits**
 - *coreAssembler User Guide*

For thorough coverage of the theory and practice of design-reuse, the following is recommended:

- Keating, Michael, and Bricaud, Pierre. Reuse Methodology Manual, 3rd Edition. Kluwer Academic Publishers, 2002.

You may also find useful the documentation for the following related Synopsys products: Synopsys IP, Design Compiler, DC Topographical, PrimeTime, and Power Compiler. For more information about these products, see:

- Documentation on the web available through SolvNetPlus:
<http://solvnetplus.synopsys.com> (a Synopsys user name and password are required).
- Documentation for Synopsys IP components, located using “Search for IP”:
<https://www.synopsys.com/designware-ip.html>

Manual Overview

This user guide contains the following information:

- [Chapter 1, “Introduction”](#) introduces coreConsultant and the other tools of the coreTools set.
- [Chapter 2, “The coreConsultant Environment”](#) introduces the coreConsultant activity-based design flow, user interfaces, and work environment; also describes how to invoke coreConsultant, install a coreKit and create a workspace.
- [Chapter 3, “Creating the RTL View of a Core”](#) describes the activities involved in assembling and configuring an RTL view of a core.
- [Chapter 4, “Creating the Gate-Level Netlist for a Core”](#) describes the activities involved in configuring and building a gate-level view of a core.
- [Chapter 5, “Synthesizing a Core”](#) describes how to select a synthesis strategy, synthesize the core, and analyze the synthesis results.
- [Chapter 6, “Verifying Your Implementation”](#) describes functional and formal verification of core views generated in the tool, and describes how to integrate the core into your application design.
- [Chapter 7, “Advanced User Modes and Features”](#) covers batch (command line) mode and tool customization.
- [Chapter 8, “Customizing a Technology Library”](#) covers customization techniques for various technology libraries.

Typographical and Symbol Conventions

[Table 1](#) describes the conventions used throughout this document.

Table 1 Documentation Conventions

Convention	Description and Example
%	Represents the UNIX prompt.
Bold	User input (text entered by the user). % \$SYNOPSYS/bin/coreConsultant
Monospace	System-generated text (prompts, messages, files, reports). "No Mismatches: 66 Vectors processed: 66 Possible"
<i>Italic</i> or <i>Italic</i>	Variables for which you supply a specific value. As a command line example:

Table 1 Documentation Conventions (continued)

Convention	Description and Example
	<code>% setenv SYNOPSIS prod_dir</code> In body text: In the previous example, <i>prod_dir</i> is the directory where your product must be installed.
(Vertical rule)	Choice among alternatives, as in the following syntax example: <code>-effort_level low medium high</code>
[] (Square brackets)	Enclose optional parameters: <code>pin1 [pin2 ... pinN]</code> In this example, you must enter at least one pin name (pin1), but others are optional ([pin2 ... pinN]).
TopMenu > SubMenu	Pulldown menu paths, such as: File > Save As ...

Synopsys Common Licensing (SCL)

You can find general SCL information on the following page:

<http://www.synopsys.com/keys>

License Information -- Synopsys Common Licensing Quickstart:

<https://www.synopsys.com/support/licensing-installation-computeplatforms/licensing.html>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

Synopsys provides various methods for contacting Customer Support, as follows:

- For fastest response, enter a case through SolvNetPlus:
 - a. Open the SolvNetPlus web page using the following URL:
<https://solvnetplus.synopsys.com/>
 - b. Click the Cases menu and then click Create a New Case (below the list of cases).

c. Complete the mandatory fields that are marked with an asterisk and click Save.

Make sure to include the following:

- Product L1: coreTools
- Product L2: coreConsultant

d. After creating the case, attach any debug files you created.

For more information about general usage, see the following article in SolvNetPlus:

<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>

■ Or, send an e-mail message to support_center@synopsys.com (your e-mail will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):

- ☐ Include the Product L1, Product L2, process, and version number in your e-mail so it can be routed correctly.
- ☐ Attach any debug files you created.

■ Or, telephone your local support center:

☐ North America:

Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.

☐ All other countries:

<https://www.synopsys.com/support/global-support-centers.html>

1

Introduction

The coreConsultant tool facilitates design reuse by providing reliable, error-free design configuration and high-quality synthesis of reusable cores. This chapter gets you started using coreConsultant.

This chapter contains the following sections:

- [“Design Reuse with Synopsys coreTools”](#) on page 22
- [“Key Terms and Concepts”](#) on page 26

1.1 Design Reuse with Synopsys coreTools

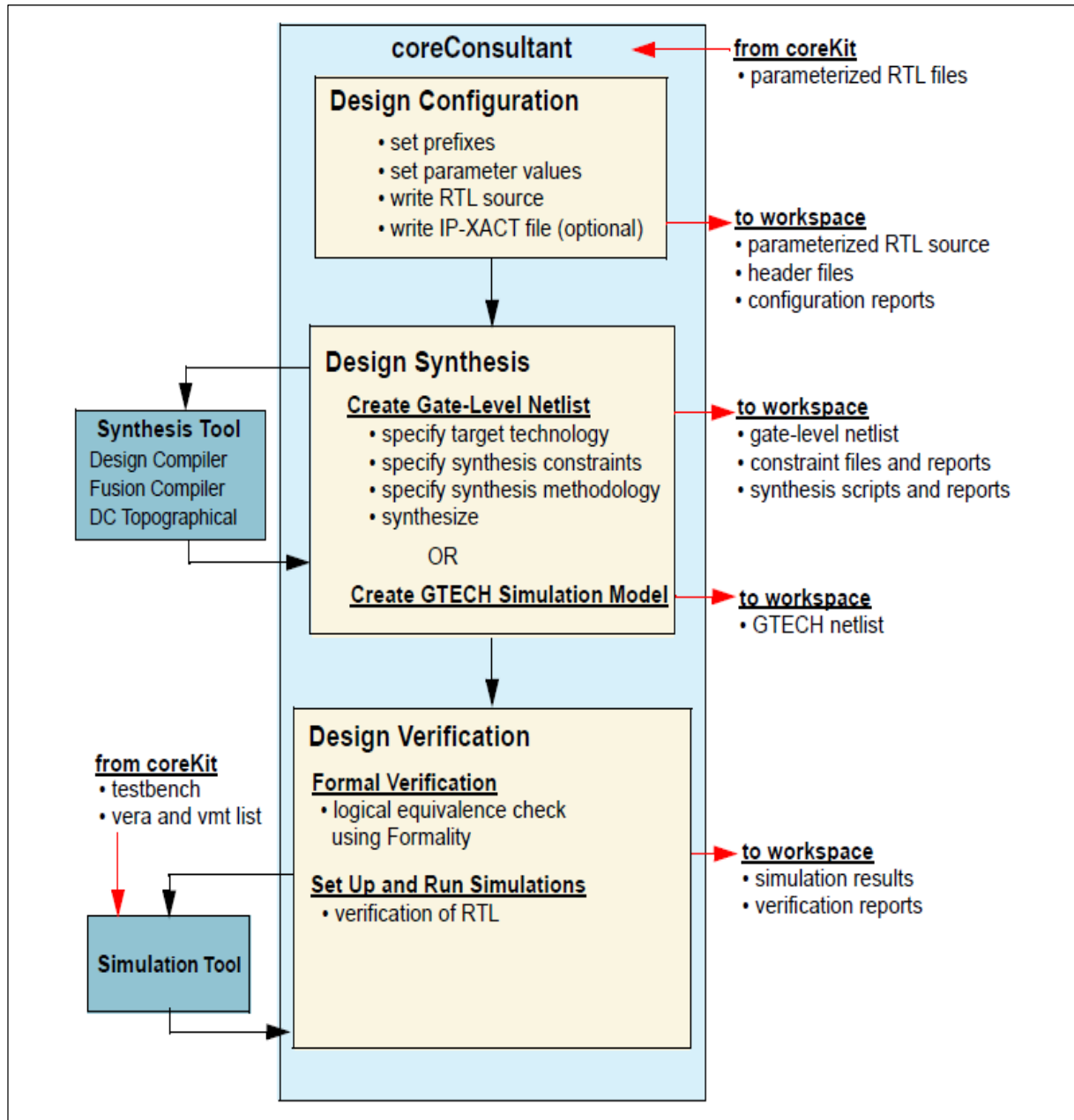
The coreTools set includes coreBuilder, coreConsultant, and coreAssembler.

- Core developers use the coreBuilder tool to package the design itself, along with the core developer's detailed knowledge of the design structure and functionality, in coreKit format.
- Core integrators use coreConsultant to work with coreKits created by the coreBuilder tool. coreConsultant automates the process of configuring, synthesizing, and verifying the core as a stand alone component.
- Core integrators use coreAssembler to assemble packaged cores into subsystems that are in turn integrated into chip-level applications. The entire subsystem can be synthesized and verified as a whole in coreAssembler.

1.1.1 coreConsultant Process Activities

[Figure 1-1](#) on page 23 illustrates the coreConsultant process flow of configuration, synthesis, and verification of your implementation of a core. Some of the coreConsultant outputs that are generated at various process steps are indicated on the right.

Figure 1-1 coreConsultant Process Flow



You can use coreConsultant to install the coreKit. After you have opened a workspace for a given coreKit, coreConsultant then steps you through the stand-alone configuration, simulation, and synthesis of the core:

1. Prior to configuration, you can set design, file, and macro prefixes to avoid name conflicts for unique elements. For more information, see [“Setting the Design, File, and Macro Prefixes”](#) on page 48).
2. During the configuration phase, coreConsultant dialogs prompt you to select configuration parameters. These may be numeric, boolean or string values. coreConsultant accepts legal input only and enforces any parameter dependencies specified by the core developer.
3. Once you have finished specifying your configuration, coreConsultant writes out the appropriate HDL source files and updates all other configuration-dependent files specified by the core developer. These files may be in encrypted form, as specified by the core developer. If enabled, coreConsultant can also write out IP-XACT XML formatted file for the component. The **Edit > Workspace Options** dialog is where you enable or disable this option.
4. To facilitate optimal synthesis, coreConsultant analyzes your technology library, prompts you to enter information about the context in which the design is being synthesized, and gives you the option of overriding the synthesis intent specified by the core developer. coreConsultant then checks to ensure that your synthesis specifications are complete and consistent.
5. Prior to synthesizing the configured core, coreConsultant prompts you to select a synthesis strategy. You may select one of the several synthesis strategies built into coreConsultant. coreConsultant then drives the required implementation tool to execute your selected strategy. [Table 1-1](#) lists the tools that coreConsultant currently supports.

Table 1-1 Supported Implementation and Verification Tools

Tool	Function
Design Compiler®	A selection of strategies for RTL-to-gates synthesis and incremental synthesis. coreAssembler interacts with Design Compiler in Tcl mode (dc_shell-t).
Fusion Compiler™	A selection of strategies for RTL-to-gates synthesis and incremental synthesis. coreAssembler interacts with Fusion Compiler in Tcl mode (fc_shell-t).
DC Explorer	Synthesis using DC-Explorer, provided that version G-2012.06-SP2 or newer DC-Explorer is available and you use the DCTCL_quick_map_strategy.
DC Topographical	A selection of physical synthesis strategies that implement both RTL-to-placed-gates and gates-to-placed-gates methodologies.
PrimeTime®	Static Timing Analysis, Timing model generation.
Formality®	Formal verification.
RTL Architect™	Enables early, physically aware RTL exploration to achieve a better starting point for RTL synthesis and better convergence during physical implementation.
VCS®/Vera®	Simulation model verification.

coreConsultant generates an optimized set of commands to drive the implementation tool based on your core configuration, the cell-set characteristics of your technology library, and the features available in your current version of the selected implementation tool. You can select to execute the selected strategy immediately or just generate the scripts for later use.

6. coreConsultant supports core-specific verification activities, as specified by the core developer. In addition, coreConsultant interfaces directly with Synopsys Formality to perform formal verification between your gate-level design and the original RTL, or between two specified gate-level implementations.
7. Once you have verified the functionality and timing of your particular configuration of the core as a stand-alone entity, you can integrate it into your application and verify at the chip level.
8. In addition, there may be core-specific implementation tasks as defined by the core developer.

1.2 Key Terms and Concepts

Discussion of design reuse methodology with coreConsultant uses the key terms and concepts defined in [Table 1-2](#).

Table 1-2 Key Terms for coreConsultant

Term	Definition
Application Design	Overall chip-level design into which a subsystem or subsystems are integrated. An application design includes one or more subsystems, plus custom, application-specific logic, hard macros, and memory.
Component	An instance of a core in a design or subsystem.
Configuration Intent	The range of values allowed for each parameter associated with a reusable core. Using the coreBuilder tool, the core developer specifies how the core may be configured for reuse in chip-level applications. The core developer also specifies parameter dependencies so that when the core is added to the subsystem, coreAssembler accepts only legal combinations of parameter settings from the core integrator.
Core	A reusable design created and configured by a core developer and incorporated into chip-level applications by one or more core integrators.
Core Developer	The creator of a reusable core. Using the coreBuilder application, the core developer applies attributes to a core design and packages the core and its attributes into a coreKit so that core integrators can easily implement the core in system-on-a-chip applications.
Core Integrator	The person who uses coreConsultant to configure a reusable core or coreAssembler to incorporate reusable cores into a larger design called a subsystem.
coreKit	The design package created by the coreBuilder application to facilitate IP reuse. Each coreKit includes the core itself, the synthesis methodology specified by the core developer, interfaces for subsystem connection, and optional items, such as verification environment files and core-specific documentation.
Design Context	Aspects of the target environment for the core or subsystem that affect synthesis. The core integrator specifies the design context before coreConsultant or coreAssembler generates a synthesis strategy for the core or the subsystem. Design context includes information such as technology libraries, wireload model, parent wireload model, and operating conditions.
IEEE P1685	Refers to the IEEE standardized version of the IP-XACT XML standard. The most recent version is P1685-2014.
Intellectual Property (IP)	In the broad context of design reuse, IP refers to any reusable design. The scope of this manual is mainly soft IP because coreConsultant focuses primarily on the reuse of soft cores (user-configurable, technology-independent blocks of synthesizable RTL code). Hard IP support is available if the hard core includes an RTL wrapper.

Table 1-2 Key Terms for coreConsultant (continued)

Term	Definition
Interface	A set of ports and parameters that defines a connection point to a component. Interfaces are “provided” by one component and “consumed” by one or more other components. For example, a bus IP would provide a master interface while each master component in the subsystem would consume this interface.
IP-XACT	An XML-based standard for describing IP and IP packaging, as well as plug-in to IP tools and flows, to enable more efficient and cost-effective integration of IP from multiple sources.
Subsystem	The highest level of RTL automatically generated by coreAssembler. The subsystem contains reusable cores connected using coreAssembler.
Synthesis Intent	Attributes the core developer applies to the top-level design, ports, and subdesigns of a core. coreConsultant and coreAssembler use these attributes to drive the synthesis process. Synthesis intent determines how coreConsultant and coreAssembler generate synthesis constraints, strategies, and directives.
TechSetup	The collective settings that are made in the Specify Target Technology activity to specify the technology libraries for ASIC synthesis.

2

The coreConsultant Environment

Before using coreConsultant to install, configure, and synthesize a reusable core, you should understand the basic concepts about the coreConsultant work environment.

This chapter contains the following sections:

- [“coreKits and Workspaces”](#) on page 30
- [“Activity-Based Design Flow”](#) on page 31
- [“User Interfaces”](#) on page 33
- [“Online Help Tools”](#) on page 35
- [“coreConsultant Spreadsheets”](#) on page 36
- [“Basic Operations”](#) on page 37

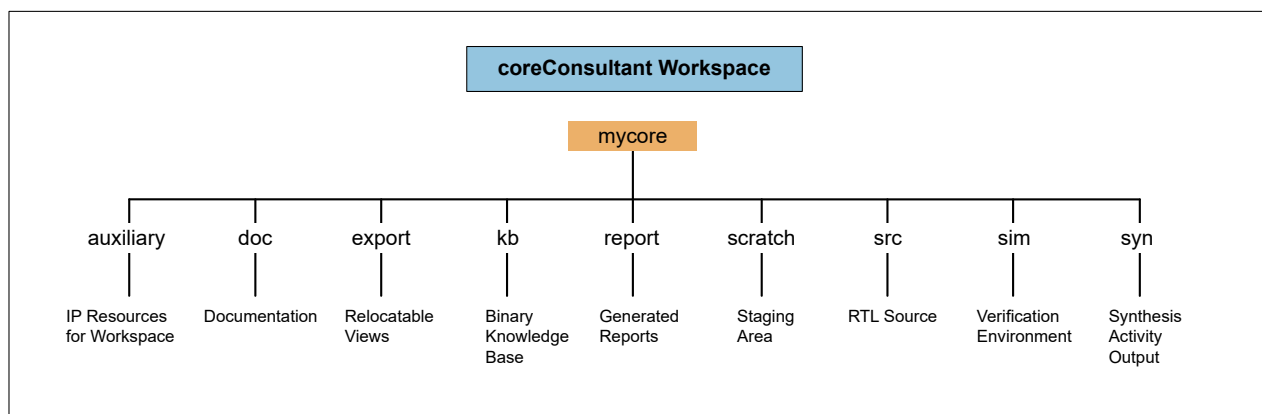
2.1 coreKits and Workspaces

The primary input to coreConsultant is a coreKit. You use coreConsultant to install the coreKits and then create workspaces.

A workspace is your working version of a coreKit, where you configure, simulate, and synthesize your own implementation of the core. It is also a UNIX directory where all synthesis scripts, logfiles, and test benches are created. You can create several workspaces so that you can experiment with different design alternatives. The exact contents of a coreKit or workspace depends on how the core developer packages the core.

Figure 2-1 shows an example of a coreConsultant workspace. Some of the directories shown are created after you perform the related activities (for example, the export and syn (synthesis) directories).

Figure 2-1 coreConsultant Workspace Tree



Note

IP-specific directories and files are not shown here, but they may also reside in your workspace.

For information about IP-specific objects, see the documentation for your IP product.

2.2 Activity-Based Design Flow

coreConsultant uses an activity-based flow to guide you through the required implementation tasks for a core. Activities that depend on other activities are disabled until you complete all prerequisite activities. For example, verification and synthesis activities are disabled until you specify your configuration for the core.

The coreConsultant activities are organized into a hierarchy of view generation and view verification processes. The different views of a core are listed in [Table 2-1](#).

Table 2-1 coreConsultant Views of a Core

View	Purpose	Outputs
RTL View	Defines the core at the RTL level in either Verilog or VHDL. Cores are defined by the core integrator and then automatically configured by coreConsultant.	■ src/* files ■ export/comp_inst.v[hd] ■ export/comp.lst ■ export/README ■ export/comp.xml (optional)
Gate-Level View	A translation of the RTL view into a technology-specific netlist, which may include physical placement information.	■ export/comp.db ■ export/comp.v[hd] ■ export/comp.sdc
GTECH View	A translation of the RTL view into a generic technology. This view is used for RTL simulation of encrypted source code by non-Synopsys simulators, which are unable to read the encrypted RTL. Generation of this view is not built into coreConsultant, but is provided by all DesignWare IP.	■ gtech/syn/finalcomp.v[hd]

The specific set of activities for a core are a combination of the default (built-in) coreConsultant activities listed in [Table 2-2](#), plus core-specific activities defined by the core developer. For example, the core developer creates core-specific verification activities and packages them into the coreKit.

Table 2-2 coreConsultant Default Activities

Activity	Function
Create RTL	
Set Design, File, and Macro Prefixes	Use prefixes to avoid name conflicts for unique elements. ■ Specify a design prefix when you plan to instantiate different versions of the same component in the same design database. ■ Specify a file prefix to make unique names for generated files. ■ Specify a macro prefix to make 'define <variable> definitions unique.
Select Prime Profile	The activity allows users to select and apply packaged, or external Prime Profiles on a component.

Table 2-2 coreConsultant Default Activities (continued)

Activity	Function
	For more information, see “Select Prime Profile” on page 48.
Specify Configuration	Specify values for HDL configuration parameters.
Create Gate-Level Netlist	
Specify Target Technology	Specify technology libraries.
Specify Clock(s)	Define clocks and all properties associated with clocks.
Specify Operating Conditions & Wireloads	Specify information defining context in which the core is used (operating conditions and wire loads).
Specify Port Constraints	View and (if necessary) modify developer-specified port attributes, most of which control constraint generation.
Specify Synthesis Methodology	View and (if necessary) modify developer-specified values for attributes that describe design characteristics and control the synthesis process.
Specify Test Methodology	Set and/or review Design for Test attributes on components.
Perform ASIC Synthesis	Select synthesis strategy and options, execute synthesis, and analyze results.
Generate Test Vectors	Generate test vectors using TestMAX.
Create GTECH Simulation Model	
Generate GTECH Model	Generate component-level GTECH netlist for RTL simulation.
Verify Component	
Formal Verification	Formally verify one view against another.
Static Timing Analysis	Generate and optionally run static timing analysis scripts.
Setup and Run Simulations	Invoke component-level testsuite regression environment (TRE).

2.3 User Interfaces

The coreConsultant tool provides the following user interfaces:

- “[Command Line Mode](#)”
- “[Batch Mode](#)”
- “[GUI Mode](#)”

2.3.1 Command Line Mode

Command line execution is available through the coreConsultant console in GUI mode or by invoking coreConsultant in shell mode. “[Advanced User Modes and Features](#)” on page 171 describes how to use coreConsultant in command-line mode.

2.3.2 Batch Mode

Batch mode allows you to execute a series of coreConsultant commands from a batch file. To facilitate the creation of batch scripts, coreConsultant provides the `write_batch_script` command to write a batch file that contains the complete set of commands to re-create the current state of your workspace. “[Advanced User Modes and Features](#)” on page 171 describes how to use coreConsultant in command line mode.

2.3.3 GUI Mode

GUI mode is recommended for new users. In GUI mode, you interact with coreConsultant through the Console pane, the Activity List pane, and the Activity View pane for the individual activities, as shown in “[coreConsultant GUI Features](#)” on page 34.

■ Console Pane

Contains three tabs. The Log tab contains the sequential log of all coreConsultant commands and their respective output. The History tab contains a sequential list of all coreConsultant commands executed directly in the console. The Errors/Warnings tab contains a sequential list of all errors, messages, and warnings, and supports linking from a specific message into the Log tab.

■ Activity List Pane

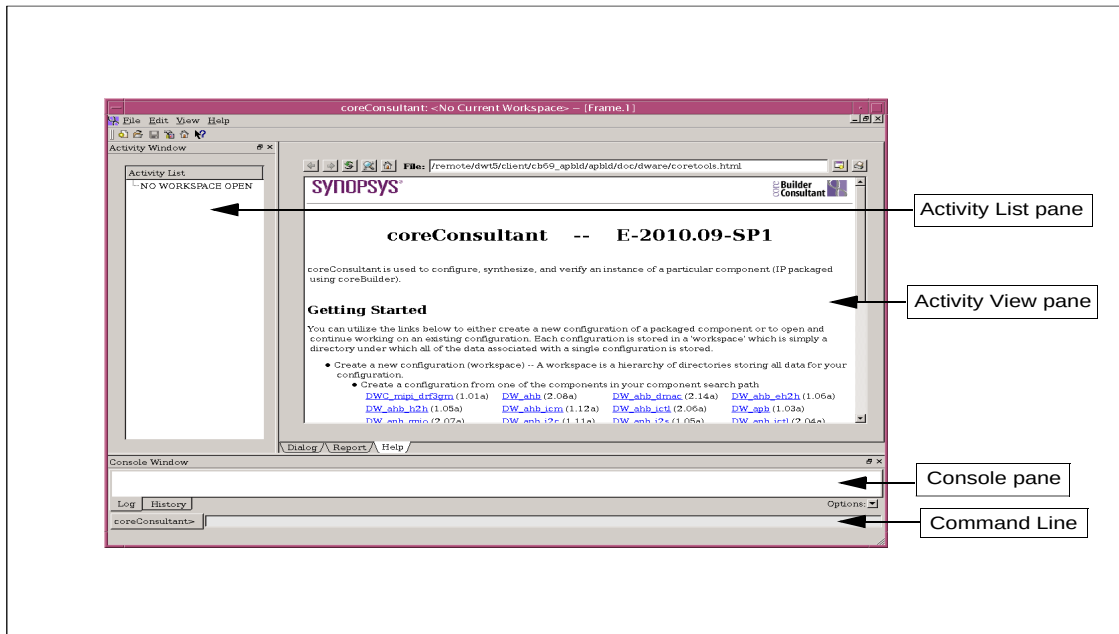
Shows all activities available in coreConsultant in a hierarchical tree. Completion status of each activity is indicated by a check mark in the box associated with each activity. Clicking on a label opens the Activity View pane for that activity.

■ Activity View Pane

This pane shows the current activity and contains the following three tabs:

- Dialog tab – allows data entry relevant to the current activity
- Report tab – Contains controls to generate reports, generate optional views, and to display reports on the status of the current activity. See “[Generating Reports and Optional Views](#)” on page 46.
- Help tab – Shows Help information for the current activity

Figure 2-2 coreConsultant GUI Features



For information about how to use coreConsultant in GUI mode to generate RTL and gate-level views, see [“Creating the RTL View of a Core”](#) on page 47 and [“Creating the Gate-Level Netlist for a Core”](#) on page 59, respectively. For information on synthesis specifications for the core, see [“Synthesizing a Core”](#) on page 89. For information on formal verification of the core, see [“Verifying Your Implementation”](#) on page 141.

2.4 Online Help Tools

Several types of online help are available through coreConsultant:

■ “What’s This?” Quick Help

When you position your mouse pointer over a GUI item and right-click, coreConsultant displays a menu with the “What’s This?” option. Clicking “What’s This?” displays a brief help message for the selected item. “What’s This?” can also be accessed by left-clicking the Question Pointer on the toolbar and then left-clicking on a GUI item.

■ The Toolbar Help Menu

Click the toolbar Help button to show a list of help topics. When you click a topic, the corresponding help information appears.

■ Activity View Help Tab

The Activity View pane features a Help tab that displays a detailed, context-specific help page, with additional links to the online command reference and other appropriate references. The Activity View Help tab contains several toolbar buttons at the top. You can use the buttons on the upper left to go back to the previous page, to go forward to the next page, to refresh the current page, to search for text, or to return to the coreTools home page (from left to right). You can use the buttons on the upper-right to open a separate window with the same page in it. Opening the page in a separate window can be useful, as it allows you to see the information in the Help tab and in the Dialog tab at the same time.

■ Help on coreConsultant Commands

The Synopsys coreTools Online Command Reference Index is available through the Help menu. It contains a collection of man pages for all coreConsultant commands, attributes, variables, and item types.

You can also access coreConsultant command help by entering one of the following commands at the coreConsultant prompt in the Console pane:

```
coreConsultant> help [cmd] [-verbose]
coreConsultant> cmd -help
coreConsultant> man [cmd]
```

2.5 coreConsultant Spreadsheets

In GUI mode, coreConsultant sometimes presents spreadsheets so that you can review and modify data associated with the current activity. In terms of operation, coreConsultant spreadsheets resemble the spreadsheets used in popular database and financial applications. Left-click to select a row, column, or cell. Right-click to access a menu of options including copy, cut, paste, help, and column fill operations. [Table 2-3](#) describes the basic spreadsheet operations.

Table 2-3 Common Spreadsheet Operations

Operation	How To Perform the Operation
Resize a column	Use your left mouse button to grab the column separator in the column title bar. Move it left or right.
Enter a value in a cell	Click in a cell and type in text or select a value from the combo box menu. coreConsultant does not save the value until you click the OK button. If you click the Cancel button, coreConsultant does not save any edits.
Evaluate formula	Click your left mouse button on a cell containing a formula, then observe the value in the Current Cell field.
Select multiple cells in a column	To select all the cells in a column, click your left mouse button on the title bar of the column. To select several (but not all) cells, use your left mouse button to select the first cell and drag down to the last cell you wish to select. Alternatively, press Shift and use the left mouse button to select the last cell. CTRL-L is the toggle control for selecting/deselecting cells.
Cut	To cut the contents of one or more cells, select the cells, click the right mouse button, and select Cut. The keyboard shortcut for Cut is CTRL-X.
Copy	To copy the contents of one or more cells, select the cells, right click with the mouse, and select Copy. The keyboard shortcut for Copy is CTRL-C.
Paste	To paste the content of a previous cut or copy operation into one or more cells, select the cells into which you want to paste the data, click the right mouse button and select Paste. The size of the paste must match the size of the corresponding cut/copy. The keyboard shortcut for Paste is CTRL-V.
Fill a column	To fill one or more consecutive cells in a column with the current contents of a cell, select the cell containing the content, plus one or more cells below it. Click the right mouse button, and select Fill Down.
Find text in a column	To find text in a column, select a cell in the column, click the right mouse button, select Find, then enter the text you for which you are searching.
Replace text	To replace text in a column, select a cell in the column, click the right mouse button, select Replace, then enter the text you want to find and the text to insert in its place.

2.6 Basic Operations

Before you begin working with a core in a coreConsultant workspace, it is important to be familiar with the following basic operations:

- “[Invoking coreConsultant](#)” on page 37
- “[Installing coreKits](#)” on page 39
- “[Component Update Checking](#)” on page 40
- “[Workspace Operations](#)” on page 43
- “[Exiting coreConsultant](#)” on page 45
- “[Working with the Activity List](#)” on page 45
- “[Generating Reports and Optional Views](#)” on page 46

2.6.1 Invoking coreConsultant

To invoke coreConsultant in GUI mode, enter the following command at the UNIX prompt:

```
% install_dir/bin/coreConsultant
```

2.6.1.1 Command Line Options

The complete syntax for invoking coreConsultant is:

```
coreConsultant [Workspace_name ] [-shell]
[-x "command_string"] [-f cmd_file]
```

Table 2-4 coreConsultant Command Line Invocation Options

Argument	Description
<i>Workspace_name</i>	Loads the workspace <i>Workspace_name</i> upon startup.
-shell	Selects shell mode instead of GUI mode. For more information about shell mode, see “ Command Line Interface ” on page 172.
-x “ <i>command_string</i> ”	Executes <i>command_string</i> upon startup. <i>command_string</i> is one or more coreConsultant commands, separated by semicolons (;).
-f <i>cmd_file</i>	Executes one or more coreConsultant commands in a specified command file (<i>cmd_file</i>) in batch mode on startup.

2.6.1.2 Examples

To invoke coreConsultant and automatically load the Config1 workspace, enter the following command:

```
% coreConsultant path/Config1
```

To invoke coreConsultant in shell mode and execute the batch command file named MyConfig.tcl, enter the following command:

```
% coreConsultant -shell -f MyConfig.tcl
```

2.6.1.3 coreConsultant Main Window

The coreConsultant Main window ([Figure 2-2](#) on page 34) appears when you invoke coreConsultant in GUI mode. The Console pane, Activity List pane, Activity View pane, and command line entry field serve as your interfaces to the coreConsultant tool.

You can perform operations by selecting menu options from the toolbar or by entering coreConsultant commands at the coreConsultant> prompt in the command line entry field. [Table 2-5](#) defines the menu options available on the toolbar.

Table 2-5 Toolbar Menu Options

Menu Command	Description	Command Line Mode
File > Install coreKit	Install a packaged coreKit.	Specify coreKit file when you invoke coreConsultant.
File > New Workspace	Create a new workspace.	create_workspace
File > Open Workspace	Open an existing workspace.	open_workspace
File > Save Workspace	Save the currently loaded workspace.	save_workspace
File > Save Workspace As	Save the currently loaded workspace into a new directory.	duplicate_workspace
File > Close Workspace	Close a workspace.	close_workspace
File > Generate Reports	Allows selection and generation of activity reports (reports are not automatically generated)	generate_reports
File > Generate Optional Views	Allows selection and generation of alternate views (alternate views are not automatically generated)	generate_views
File > Write Batch Script	Create a batch script to re-create the current state of your workspace.	write_batch_script
File > Exit	Exit coreConsultant.	exit or quit
Edit > Tool Installation Roots	Specify tool installation directories. Also specify use of 64-bit tool binaries.	set_tool_root
Edit > Preferences	Specify user preferences for tool interface, verification, execution, synthesis, and IP-XACT options for new workspaces. For more information, see “Editing Tool Setup and User Preferences” on page 177.	Not applicable
Edit > Workspace Options	Specify synthesis and IP-XACT options for the current workspace.	Not applicable
View > Home Page	Go to the coreConsultant GUI Home Page for a list of available cores and activities.	Not applicable

Table 2-5 Toolbar Menu Options (continued)

Menu Command	Description	Command Line Mode
View > Reports	View a list of the available reports for the current core.	Not applicable
View > Application Toolbar	Show or hide the toolbar that is above the Activity pane in the Main window.	Not applicable
View > Console Window	Show or hide the Console pane and the command line in the Main window.	Not applicable
View > Activity Window	Show or hide the Activity pane from the Main window.	Not applicable
Help > Check for IP Updates	Performs a manual check comparing the open workspace component against the latest component in your DESIGNWARE_HOME and on the Synopsys website.	Not applicable
Help > Show IP Update Report	Shows the latest Check Update report (the <workspace>/report/IPUpdateCheck.html file).	Not applicable
Help > Check Environment	If DesignWare component is open, performs a check of the environment and tools.	Not applicable
Help > coreConsultant Tool Help > Item	Open a Help window to view a selected help item. Includes: Error Messages, the coreConsultant User Guide, the Command Reference, Release Notes, Application Notes and core-specific databooks.	man or help

2.6.2 Installing coreKits

Synopsys ships its cores already installed, so this installation procedure is not required for Synopsys IIP cores. The following procedure is necessary only if you are using non-Synopsys cores. Custom cores must be packaged as *.corekit files in order to be installed and loaded into coreConsultant.

Use the following procedure to install a coreKit, unless a coreKit provider supplies special installation instructions for that core.

1. Extract the coreKit file from the delivery medium and install it on your computer by following the instructions provided by the core developer.
2. Invoke coreConsultant and load the coreKit using either of these methods:
 - Either specify the coreKit when you invoke coreConsultant:


```
% coreConsultant path_to_coreKit/name.coreKit
```
 - Or invoke coreConsultant and use the **File > Install coreKit** menu command on the coreConsultant toolbar to load the coreKit. coreConsultant displays a Packaged coreKit File selection dialog in which you select a coreKit file and the directory where you want it installed.
3. In the Packaged coreKit field, navigate to the directory where the packaged coreKits have been placed and select the desired coreKit file.

4. In the Installation Directory field, navigate to the directory where you want the coreKit installed.
5. Click OK to continue the installation.

coreConsultant writes all files associated with the coreKit into your selected coreKit installation directory, using a directory structure specified by the core developer.

2.6.3 Component Update Checking

coreConsultant provides a way for you to check for Synopsys Library synthesizable component updates as well as existing STARS for the Synopsys IP used in a coreConsultant workspace.

The first time you complete the “Specify Configuration” activity, or open a workspace where automatic IP update checking has not been initialized, you are prompted to enable automatic IP update checking and specify a frequency in days for how often to perform the check. Subsequently, the update check will happen automatically when the specified time period has elapsed and either of the following occurs:

- You open a workspace where the “Specify Configuration” activity is completed.
- You complete the “Specify Configuration” activity during a session.

At the conclusion of the update check, the update check report is shown only if:

- There is a newer version of the component.
- There are STARS that have not been previously reported.

In other words if nothing relevant has changed, the report is not shown.



Note

If you specify automatic update checking in coreAssembler, those settings apply to coreConsultant also. They are saved in the `.synopsys_rt_prefs.tcl` file in your home directory. Changes to this feature in one tool affect both tools.

You can also manually check for Synopsys IP updates at any time using the **Help > Check for IP Updates** menu item.

The results of both the manual and automated checks will be displayed in a top-level HTML report window.



Note

Both automatic and manual IP update checks do not update or change your design in any way. The update check only generates the report letting you know of newer versions of components or new STARS. It is then entirely up to you whether to update your `DESIGNWARE_HOME` library and/or coreConsultant workspace.

2.6.3.1 Features

- Opt in/out of the automatic checking for used IP updates at any time
- Checks the Synopsys website for IP updates and STARS for used IP
- Checks your `DESIGNWARE_HOME` library for newer versions of IP
- Links you to the download area for IP components.
- Differentiates between previously reported STARS and new STARS for your IP
- Allows you to review report from the previous update check

2.6.3.2 Synopsys IP Update Preferences

If you previously disabled automatic update checking and now wish to enable it, you use **Edit > Preferences > Synopsys IP Updates**. (For more information about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 177.)

Table 2-6 Synopsys IP Update Tool Preferences

Text Item	Description
Enable automatic Synopsys IP update checks	If enabled, automatic update checking is preformed whenever: ■ You complete the Specify Configuration activity ■ The workspace is opened and the frequency has elapsed If disabled: ■ No automatic checks are performed, and you are asked if you want to inform Synopsys about opting out. ■ If disabled, you can still perform a manual IP update check anytime using the Help > Check For IP Updates menu item.
Synopsys IP update check frequency in days	Specifies the interval between automatic update checks. If set to 0, a check is made every time the workspace is opened. If set to >0, that interval in days passes between checks. If set to -1, you have not executed the Configure Check dialog.
Inform Synopsys about opting out of automatic IP update checks	If you disable automatic IP update checking, this option allows you to notify Synopsys. If checked, Synopsys is notified “anonymously” that a single user opted out of automatic checking. If unchecked, Synopsys is not notified.

2.6.3.3 Manual IP Update Checks

At any time, you can perform a manual IP update check (exactly the same check as performed automatically) by executing the **Help > Check For IP Updates**. This option is useful when you move between phases of a project and want to verify you are using the current IP, or want to view any STARS for the components used in your subsystem design.

2.6.3.4 IP Update Check Report Format

The generated report is an HTML file named IPUpdateCheck.html, and is saved in the “report” directory within your workspace. [Table 2-7](#) describes the report format.

Table 2-7 IP Update Check Report Format

Report Heading	Description
Component Name	Synopsys component name, such as “DW_ahb_uart”.
Instance Name	Name of the instance used in your workspace such as “i_uart”.
Instantiated Version	Version number of the instance in your workspace.

Table 2-7 IP Update Check Report Format (continued)

Report Heading	Description
DesignWare Home Version	Version number of the component in your DESIGNWARE_HOME library.
Latest Available Version	The current version available for download from Synopsys.
Previously Reported STARS	Lists STARS that have appeared in the previous update checks.
New STARS	Lists any new STARS on the Synopsys website since the last report.
IP Status	The following are status text for this field: ■ No Update Available -- your component is current. ■ Download Latest IP -- a newer version of your component is available on the Synopsys website. This status text is also a link that opens your preferred browser to the download area on the Synopsys website. (Set your preferred browser from the Edit > Preferences menu choice. For more information about setting preferences, see “Editing Tool Setup and User Preferences” on page 177). ■ Replace Instantiated IP -- A newer version of your component is in your local DESIGNWARE_HOME library. Use the Replace Component menu item from the Schematic or Tree View window.

To view an existing report, use the **Help > Show IP Update Report** menu item. This menu item is only available if a previous report has been generated.

If you want to save a report (for specification purpose), be sure to copy the report from the workspace “reports” directory. If you do not, the next time a report is generated, it will overwrite the existing report.

2.6.3.5 Updating the IP Version in a Workspace

You have seen how to update your DESIGNWARE_HOME library with the latest IP version. This section shows you how to update to this IP version in your workspace. The proper solution for coreConsultant is:

1. Write a batch script from the existing workspace: **File > Write Batch Script**.
2. Edit the batch script to reference the path to the new IP version in the create_workspace command.
3. Edit the batch script to reference a different workspace name in the create_workspace command (or make sure the old workspace is removed). If you use an existing workspace name, you are prompted to “Overwrite or Rename Existing (new) Workspace” which allows you to create the new workspace with a unique name or to overwrite the current workspace.

Example: create_workspace -name i_uart_new -installation
<path>/DW_apb_uart/3.08a

4. Create the new workspace using: coreConsultant -f <batch_script_name>

Here is an alternate method to update the IP version:

1. Write a batch script from the existing workspace: **File > Write Batch Script**.
2. Remove all commands from batch script except for calls to set_configuration_parameter.
3. Create a new workspace referencing the new version of the IP.

4. Source the modified batch script to apply configuration values.
5. Interact with coreConsultant to redo the other activities.

2.6.4 Workspace Operations

You can perform these workspace operations using the coreConsultant menu items or commands.

2.6.4.1 Setting Workspace Options

You can customize the activity flow for a coreConsultant session or for all (future) coreConsultant and coreAssembler sessions. By streamlining the flow, you can improve coreConsultant performance and save disk space.

- To customize the workspace that is currently open, use **Edit > Workspace Options** to control whether synthesis related activities appear in the Activity List.
- To customize all newly created workspaces, use **Edit > Preferences | Workspace Options** to control the following:
 - Whether batch scripts are written after assembly activities are completed
 - Whether the following appear in the Activity List:
 - Synthesis related activities
 - Component-specific unit-level test activities
 - Memory Map Specification activity



Note

These are global settings that are:

- Inherited by each newly created workspace
- Used by both coreConsultant and coreAssembler

For more information about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 177.

2.6.4.2 Creating a Workspace

Think of your workspace as a private area for developing your own configuration of the core. You can create multiple workspaces for different projects or for experimentation with various design alternatives.

When you create a workspace, coreConsultant creates both a directory structure for your work and a knowledge database (.kb file) for the workspace. The knowledge database stores the incremental state of your workspace. As you continue through the project design flow, coreConsultant saves your cumulative changes in this workspace knowledge database.

To create a workspace in GUI mode:

1. Select **File > New Workspace** in the coreConsultant toolbar or choose one of the available coreKits from the “Create a new configuration (workspace)” link on the coreTools home page.
2. Enter your specifications in the “Create workspace” dialog. Then click OK to create your workspace.

When coreConsultant finishes creating your workspace, it automatically displays the Activity List pane for the workspace. At this point, you can either continue your work by initiating the first activity in the Activity List or save and close your workspace and continue at a later time. See [“Creating the RTL View of a Core”](#) on page 47 and [“Creating the Gate-Level Netlist for a Core”](#) on page 59 for instructions on completing the specification activities.

2.6.4.3 Saving a Workspace

coreConsultant maintains the current state of your workspace in a knowledge database within the workspace. You can save this information at any time in the design flow, quit and restart coreConsultant, reopen your workspace, and continue where you left off.

To save your current workspace, select **File > Save Workspace** from the coreConsultant toolbar. In command line mode, use the coreConsultant `save_workspace` command to save your workspace, for example:

```
coreConsultant> save_workspace
```

Each time you click the Apply button in an Activity View dialog to complete an activity, coreConsultant automatically saves the workspace to disk.

2.6.4.4 Saving a Workspace As

You can also save your current workspace to a different workspace name by selecting **File > Save Workspace As**. When you are using the **Save Workspace As** command, the required linked installation files is copied to the duplicate workspace.

2.6.4.5 Closing a Workspace

To close a workspace, select **File > Close Workspace**. In command line mode, use the coreConsultant `close_workspace` command to close your workspace; for example:

```
coreConsultant> close_workspace (Close without saving.)
```

```
coreConsultant> close_workspace -save (Save, then close.)
```

2.6.4.6 Opening a Workspace

You can open an existing workspace by specifying the workspace name on the command line when you invoke coreConsultant. For example:

```
% coreConsultant path/Config1
```

You can also open an existing workspace by selecting **File > Open Workspace** or by clicking the “Open an existing workspace” link in the coreTools home page. In the dialog box, select the directory containing the workspace, then click OK. Valid workspaces appear in the directory browser as file folders labeled “cC.”

In command line mode, use the coreConsultant `open_workspace` command to open a workspace. For example, to open the workspace *Config1*, enter the following command:

```
coreConsultant> open_workspace path/Config1
```

2.6.4.7 Moving a Workspace

If you want to move or copy your workspace, or move referenced components or files (such as models), you may need to update your workspace to reflect the changes.

- You can use the UNIX “mv” command to move the workspace directory as long as you are on the same network and the \$DESIGNWARE_HOME installation is still at the same location.
- If you move the \$DESIGNWARE_HOME location, you must update links with the **File > Update Workspace Links** menu item and Dialog. Choose the **Moved Install** item and provide a path to the moved component of the form <path>/iip/component/version.

You can also run the “update_workspace_links -moved_install <dir>” command on the workspace to update links to the new install area.

- To replace the links in a workspace with the actual objects (for portability), use the **File > Update Workspace Links** menu and Dialog, then check the Copy option and OK the dialog. You can also issue the “update_workspace_links -copy” command. This is useful for backup and archive purposes, but creates a much larger workspace tree.

The update_workspace_links command is used to modify the symbolic links within a workspace.

```
update_workspace_links [-copy] [-moved_install dir] [workspace]
```

For the complete description, enter “man *command_name*” from the coreConsultant command line.

2.6.5 Exiting coreConsultant

To exit coreConsultant, select **File > Exit** from the coreConsultant toolbar. If your workspace has changed, coreConsultant prompts you to save it, as described in “[Closing a Workspace](#)” on page 44.

To quit coreConsultant in command line mode, use the quit command:

```
coreConsultant> quit
```

If you execute quit from the coreConsultant GUI after executing commands that modify the workspace, coreConsultant prompts you to save your workspace before exiting. However, in shell mode (coreConsultant -shell), coreConsultant does not prompt you to save your workspace. To save your work before quitting, enter the save_workspace command at the coreConsultant prompt before executing quit.

2.6.6 Working with the Activity List

When coreConsultant finishes creating and setting up your workspace, it automatically displays the Activity List for the workspace. Click the item label on the Activity List to initiate that activity. coreConsultant displays the specific Activity View where you select or enter data associated with the activity. After entering data for an activity, click the Apply button. This completes the activity and shifts to the Report tab of the corresponding activity, where you can generate and view reports and create optional design views. If you have customized workspace options, not all activities may be visible. For workspace activity customization, see “[Setting Workspace Options](#)” on page 43.

When you close your workspace, coreConsultant closes the associated Activity List. Similarly, when you re-open your workspace, coreConsultant displays the associated Activity List showing the current state of activities completed.

For instructions on completing the system specification activities, see “[Creating the RTL View of a Core](#)” on page 47 and “[Creating the Gate-Level Netlist for a Core](#)” on page 59.

2.6.7 Generating Reports and Optional Views

When you complete an activity, the Report tab at the bottom of the View Pane shows you the reports and views that are available for that activity. To enhance performance, these reports and view are not automatically generated, but complete control is provided through the activity summary page (Report tab).

Table 2-8 The Activity Summary Page Controls

Control Name	Description
generate report	Click on the link text to generate the specific report and view the report.
generate view	Click on the link text to generate specific optional views (such a IP-XACT).
regenerate report	Discards the previous report and regenerates the report, and views the report.
<report_name>	Click on the report name (previously generated) to view the specific report.
File-->Generate Reports	Performs the same behavior as the File > Generate Reports menu item.
File-->Generate Optional Views	Performs the same behavior as the File > Generate Optional Views menu item.
View-->Reports	Performs the same behavior as the View > Reports menu item. Presents a dialog box containing an “index” of all of the generated reports.

2.6.7.1 Generating Multiple Reports

The **File > Generate Reports** menu item presents a dialog box that allows you to generate multiple activity reports from a single action. The “Delete Old Reports” option removes existing reports before regenerating the reports that you choose. There is a check box for each report you can generate, but if the item is grayed out, that report cannot yet be generated (the corresponding activity may not be complete). Click the OK button in the Generate Reports dialog box to generate the selected reports.



Note

1. Old reports are not automatically removed when you make design changes, so you may want to “Delete Old Reports” first to avoid viewing stale data.
2. Reports for synthesis and verification activities are generated by the synthesis and verification tools, and not handled by Generate Reports dialog.

2.6.7.2 Generating Multiple Views

The **File > Generate Optional Views** menu items allows you to generate multiple activity design views (such a IP-XACT files and RAL files) from a single action. There is a check box for each view you can generate, but if the item is grayed out, that view cannot yet be generated (the corresponding activity may not be complete). Click the OK button in the Generate Optional Views dialog box to generate the selected views. You can optionally generate the UVM variant of the RAL files using the write_ral_file command with the -uvm option.



Note

- Old views are not automatically removed when you make design changes, so regenerate your optional views after making design changes.

3

Creating the RTL View of a Core

This chapter describes the creation of the RTL view of a core. The Create RTL activity configures the core and creates synthesizable RTL in either Verilog or VHDL.

This chapter contains the following sections:

- [“Configuring a Core”](#) on page 48

3.1 Configuring a Core

The following topics show you how to configure a core.

3.1.1 Setting the Design, File, and Macro Prefixes

If the IP coreKit was packaged with prefixing enabled, you can set prefixes to avoid name conflicts for unique elements. In the Set Design, File, and Macro Prefixes activity, you can:

- Specify a design prefix when you plan to instantiate different versions of the same component in the same design database.

If you specify a design prefix, coreConsultant automatically adds the prefix string to all design name references in the RTL code when it writes the configured code to disk. If a component does not support prefixing, the Set Design, File, and Macro Prefixes activity does not appear in the Activity List.

- Specify a file prefix to make unique names for generated files.
- Specify a macro prefix to make 'define <variable> definitions unique.

To manually set macro prefixes, use the `prefix_macros` command. If you discover that the macro prefixes are unwanted, you can remove them using the following file:
`<workspace>/scratch/<workspace_name>_unprefix.vh.`

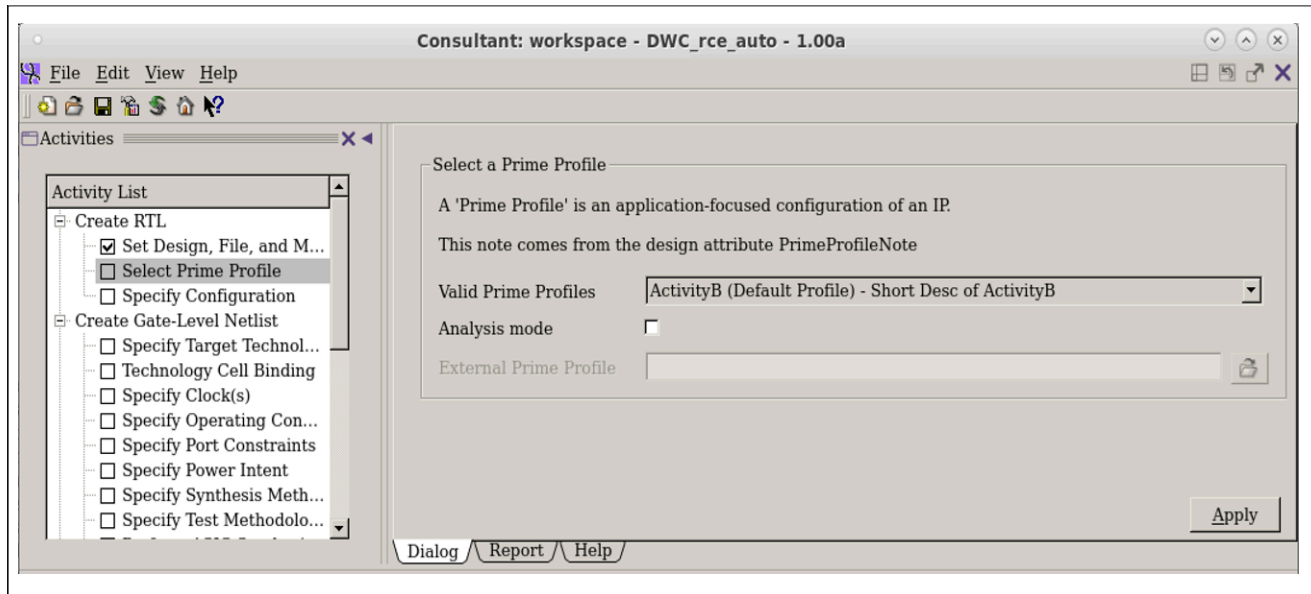
If the IP component does not support prefixing, the Set Design, File, and Macro Prefixes activity does not appear in the Activity List.

3.1.2 Select Prime Profile

A Prime Profile permits selection of an application-focused configuration of the controller. These pre-defined configurations simplify parameter selection and reduce the inadvertent selection of a non-verified crossing of the configuration parameters. The Select Prime Profile activity in the coreConsultant permits the selection of the following options in the GUI:

- Valid Prime Profiles
- Analysis mode
- External Prime Profile

[Figure 3-1](#) on page 49 shows the Select Prime Profile activity in coreConsultant.

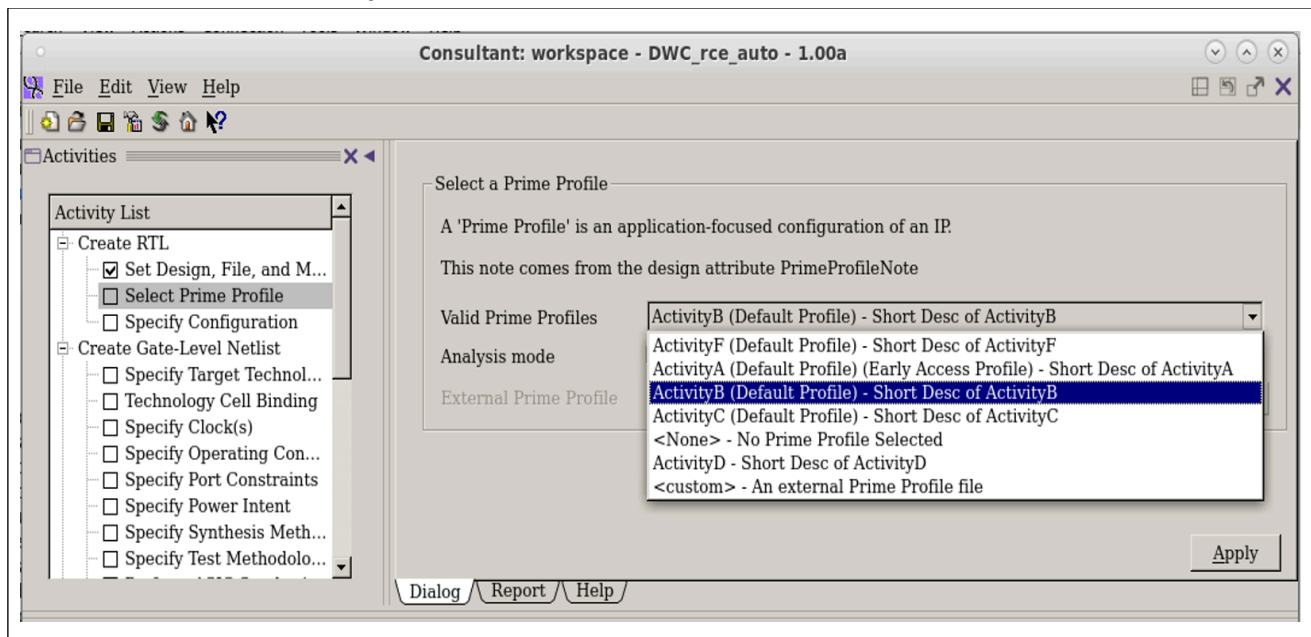
Figure 3-1 Select Prime Profile Activity

3.1.2.1 Select Prime Profile GUI Options

This section details the available GUI options in the Select Prime Profile activity.

3.1.2.1.1 Valid Prime Profiles

The Valid Prime Profiles are the list of available Prime Profiles for a component which has valid licenses. [Figure 3-2](#) shows the list of valid Prime Profiles.

Figure 3-2 Valid Prime Profile Options

After a valid Prime Profile is selected and applied, the Specify Configuration activity is populated with a pre-defined parameter configuration. Based on the selected prime profile, locked parameters appear as read-only, and the remaining parameters appear configurable. You can modify the values of the configurable parameters for further fine tuning.

Figure 3-3 shows the locked and configurable parameters in the Specify Configuration activity post the Valid Prime Profile selection.

Figure 3-3 Specify Configuration Activity - Locked and Configurable Parameters

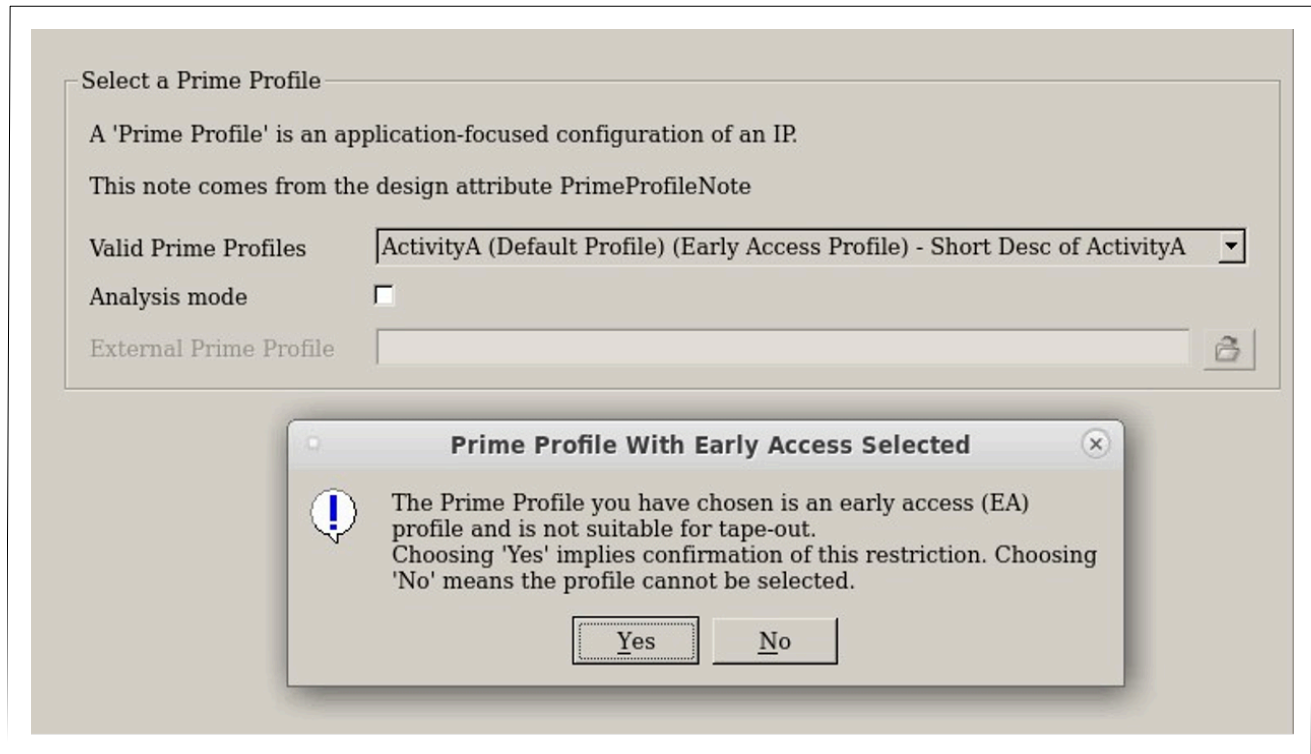
The screenshot displays a configuration window with three sections of parameters. The first section, 'BCM (DWC_rce_other)', contains 'DWC_RCE_OTHER_BCM21_F_SYNC_TYPE' with a value of 2, which is highlighted as a 'Read-only Parameter'. The second section, 'BasicConfig', contains 'APB Data Bus Width' (32) and 'ENABLE_CONDITIONAL_PORT' (unchecked), which are highlighted as 'Configurable Parameters'. The third section, 'BCM', contains 'Read data bus width of peripheral' (32), 'AUTO_BCM95_TMR' (1), 'AUTO_BCM21_F_SYNC_TYPE' (2), 'DWC_RCE_AUTO_RM_BCM00_MAJ' (unchecked), 'DUMMY0_F_SYNC_TYPE' (2), 'DUMMY1_F_SYNC_TYPE' (2 stage), 'DUMMY2_F_SYNC_TYPE' (2), and 'DUMMY3_F_SYNC_TYPE' (1), all of which are highlighted as 'Read-Only Parameters'.

Parameter Name	Value	Category
DWC_RCE_OTHER_BCM21_F_SYNC_TYPE	2	Read-only Parameters
APB Data Bus Width	32	Configurable Parameters
ENABLE_CONDITIONAL_PORT	☐	Configurable Parameters
Enable Interrupt?	☒	Configurable Parameters
Active High Interrupts ?	☐	Configurable Parameters
Read data bus width of peripheral	32	Read-Only Parameters
AUTO_BCM95_TMR	1	Read-Only Parameters
AUTO_BCM21_F_SYNC_TYPE	2	Read-Only Parameters
DWC_RCE_AUTO_RM_BCM00_MAJ	☐	Read-Only Parameters
DUMMY0_F_SYNC_TYPE	2	Read-Only Parameters
DUMMY1_F_SYNC_TYPE	2 stage	Read-Only Parameters
DUMMY2_F_SYNC_TYPE	2	Read-Only Parameters
DUMMY3_F_SYNC_TYPE	1	Read-Only Parameters



Note

The Prime Profiles categorized as Early Access are not suitable for tape-out. The coreConsultant issues a warning for such Prime Profiles as shown in Figure 3-4 on page 51.

Figure 3-4 Prime Profile Early Access Warning

3.1.2.1.2 Analysis Mode

The analysis mode for a Prime Profile allows for more flexibility in the IP configuration. To enable the analysis mode, select the Analysis mode option in the coreConsultant and then select and apply a valid Prime Profile. In the analysis mode, most of the parameters locked in the selected Prime Profile become configurable in the Specify Configuration activity. However, some parameters can remain locked to avoid known issues and limitations as shown in [Figure 3-5](#) on page 52.

Figure 3-5 Specify Configuration Activity - Locked and Configurable Parameters in Analysis Mode

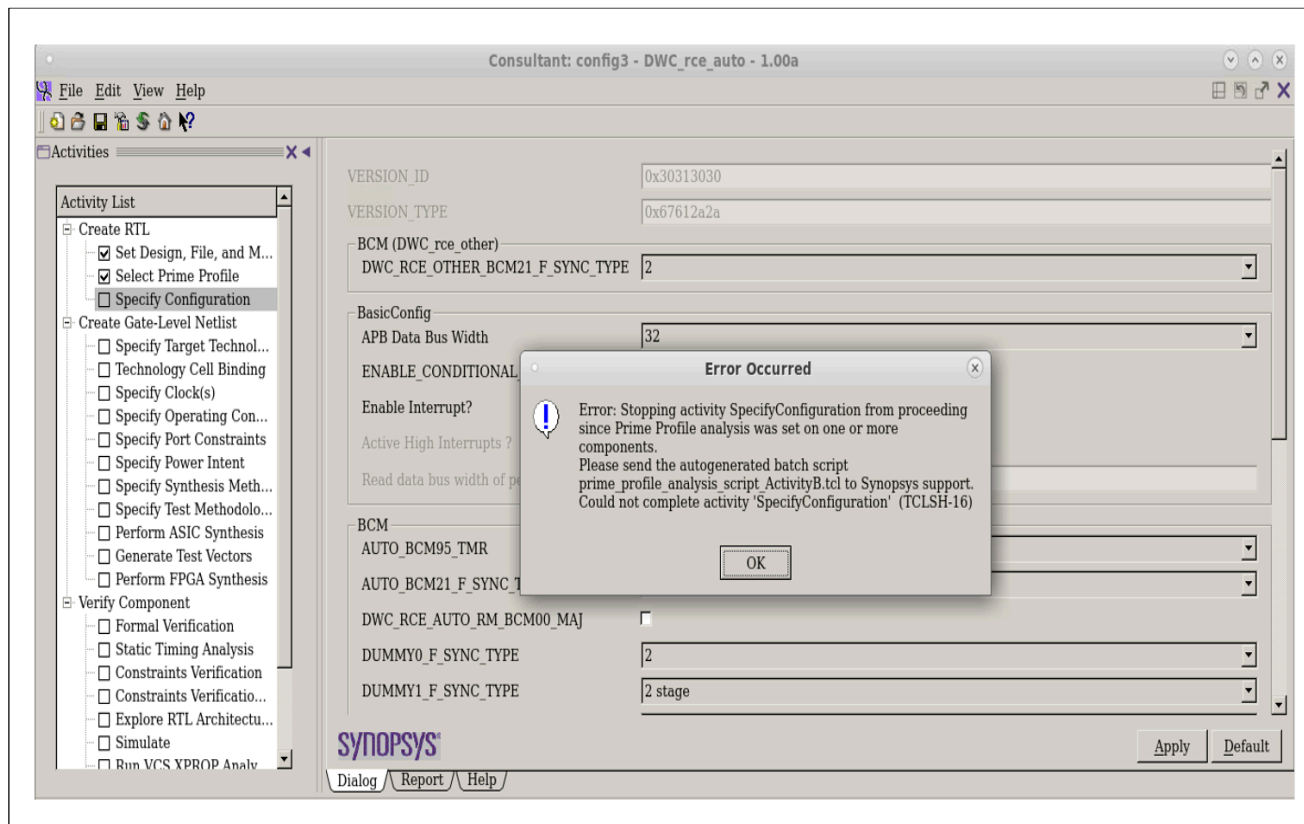
BCM (DWC_rce_other)		
DWC_RCE_OTHER_BCM21_F_SYNC_TYPE	2	Configurable Parameters
BasicConfig		
APB Data Bus Width	32	
ENABLE_CONDITIONAL_PORT	☐	
Enable Interrupt?	☐	Read-only Parameters
Active High Interrupts ?	☐	
Read data bus width of peripheral	32	
BCM		
AUTO_BCM95_TMR	1	Configurable Parameters
AUTO_BCM21_F_SYNC_TYPE	2	
DWC_RCE_AUTO_RM_BCM00_MAJ	☐	
DUMMY0_F_SYNC_TYPE	2	
DUMMY1_F_SYNC_TYPE	2 stage	Read-only Parameters
DUMMY2_F_SYNC_TYPE	2	
DUMMY3_F_SYNC_TYPE	1	

**Note**

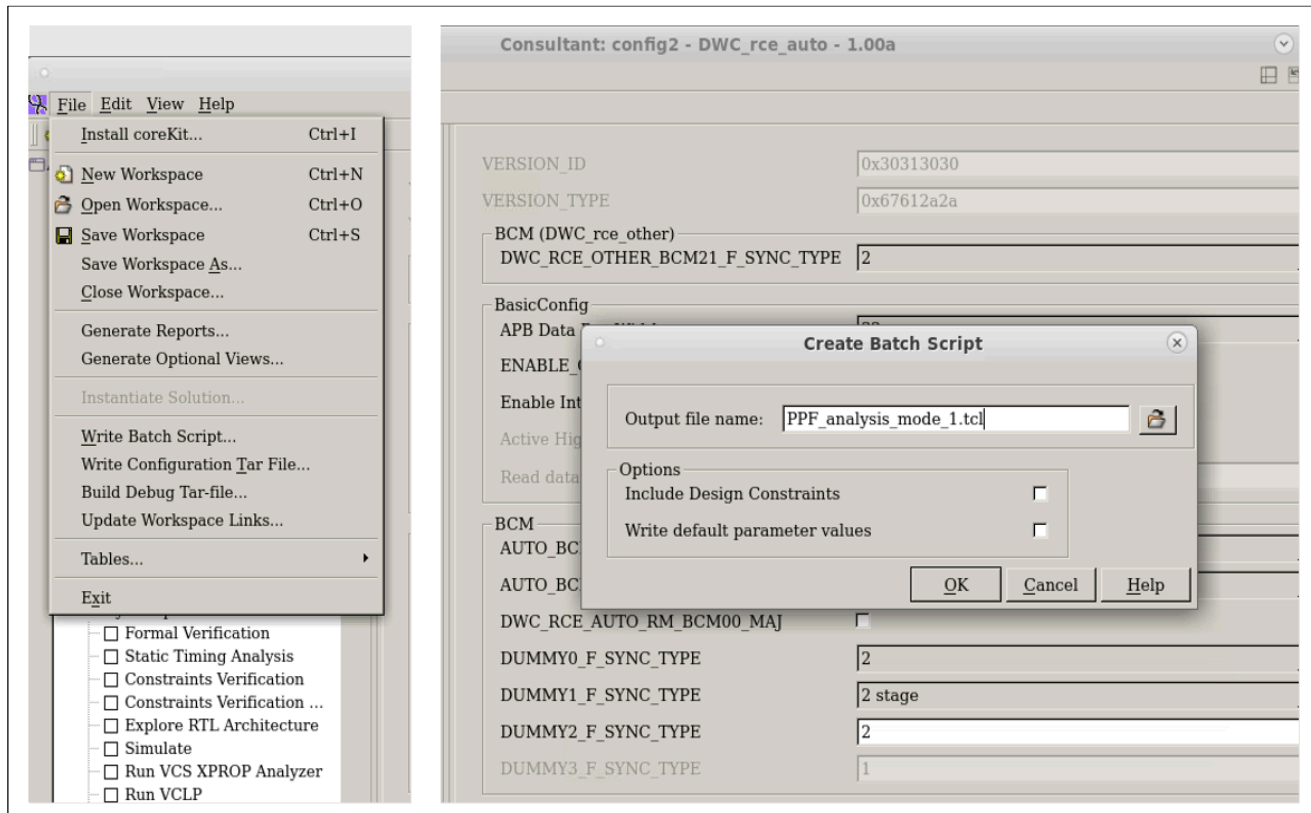
RTL generation is not allowed in the Analysis mode.

Post application of the selected Prime Profile, the coreConsultant automatically generates a batch script and stops, as shown in [Figure 3-6](#) on page 53. The batch script is available in the `<workspace>/export` folder.

Figure 3-6 Analysis Mode Error



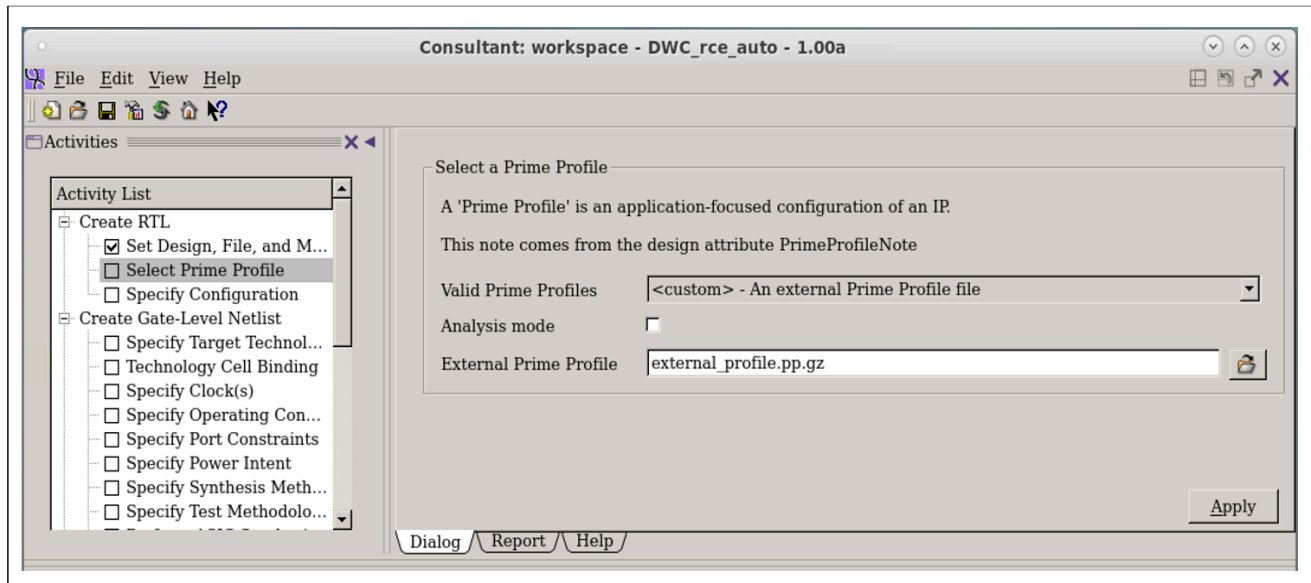
Alternatively, you can generate a batch script using **File > Write Batch Script** option as shown in [Figure 3-7](#) on page 54. The generated batch script can be shared with Synopsys for validation.

Figure 3-7 Batch Script Creation Using File Menu

3.1.2.1.3 External Prime Profiles

An alternative source of Prime Profiles are external deliverables referred to as External Prime Profiles. This approach can be utilized for special support engagements, as a path to provide configuration feedback. The External Prime Profile files have a pp.gz extension.

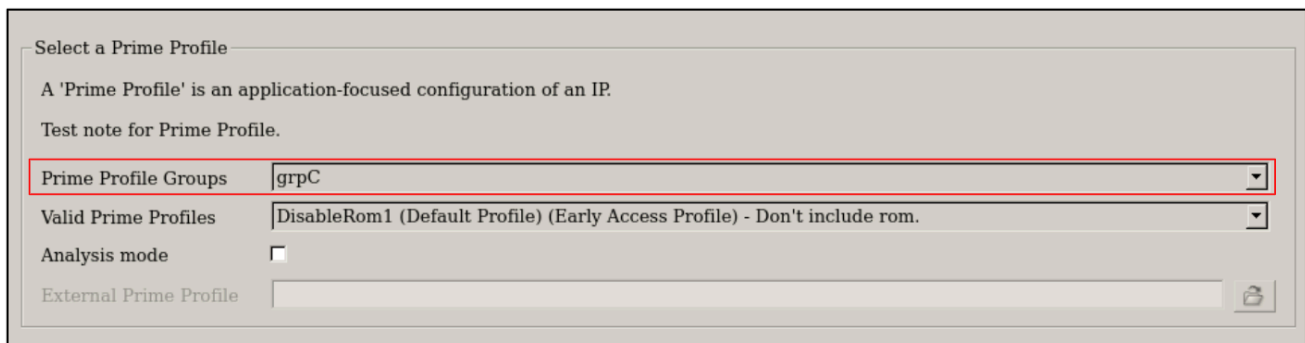
In the **Select Prime Profile** activity, the **External Prime Profile** option (<custom> - An external Prime Profile file) can be selected as shown in [Figure 3-8](#) on page 55.

Figure 3-8 External Prime Profile Selection

3.1.2.2 Prime Profile Groups

Prime Profile Groups facilitate ease of use during GUI led profile selection. A group selection drop-down appears if the profiles were created using the -group option. Selecting a group restricts the packaged prime profiles displayed. The default group is decided based on the first available default profile or alphabetical order if no default exist.

Figure 3-9 shows the Prime Profile Group option in the GUI.

Figure 3-9 Prime Profile Groups

3.1.2.3 Prime Profile Reports

Once the Prime Profile activity is completed, a HTML report is generated (PrimeProfiles.html) and is exported to the `<workspace>/report` directory.

Figure 3-10 on page 56 shows a sample HTML report generated.

Figure 3-10 Prime Profile Sample HTML Report

Design: mycore

Set Prime Profile	Analysis Mode	Valid Prime Profiles	PrimeProfileAllowNone
DisableRom1	1	DisableRom1 DisableRom2 DontLockProfile DummyCheckbox DummyCheckbox2 DummyExpr1 DummyRadio LockRamSize Test003	1

Dynamic Page (automatically refreshes).

Prime Profile	Label/Description	Default Profile	Early Access	External Profile
DisableRom1	Don't include rom. Don't allow ROM. Even in analysis mode.	1	1	0
DisableRom2	Don't include rom. Lock down ram size. Don't allow ROM. Lock down ram size in default mode.	0	0	0
DontLockProfile	Lock all except rom related params. prime profile using don't lock option	0	0	0
DummyCheckbox	Lock all - dummy check box. multiline long description for dummy checkbox prime profile	0	0	0
DummyCheckbox2	Lock expr params - dummy check box. multiline long description for dummy checkbox 2 prime profile	1	0	0
DummyExpr1	Lock all - dummy expression. dummy expression prime profile	0	0	0
DummyRadio	Lock all - dummy radio button. multiline long description for dummy radio prime profile	0	0	0
LockRamSize	Lock ram_size and ram_word. Lock ram_size and ram_word to 32. In analysis allow word to change.	0	1	0
Test003	Lock all params to 16	0	0	0

3.1.3 Specifying Configuration

Before you begin the Specify Configuration activity, you must have an open workspace with a selected core (for more information, see [“Creating a Workspace”](#) on page 43).

When you select Create RTL > Specify Configuration in the Activity List pane, the Activity View pane displays a core-specific series of parameter dialogs in which you review the options established by the core developer and specify your configuration for the core. When you finish specifying your configuration, coreConsultant writes (or overwrites) the configuration-dependent files into your workspace.

To specify your configuration, perform the following steps:

1. Select **Create RTL > Specify Configuration** from the Activity List pane. A core-specific Activity View pane appears, with a tree hierarchy of configuration categories on the left and the specification dialog on the right.
2. Review the default parameters in all configuration categories. Specify different parameter values if necessary for your design. Depending on the parameter type, you may have to enter a value or click on a checkbox.

Use the coreConsultant online help to look up the information you need to specify your parameter values by selecting the Help tab of the Activity View pane. Refer also to the databook and other documentation for the specific coreKit by selecting **Help > corename Data Book** from the toolbar menu.

- When you have reviewed the default values for all parameters and changed the specification if necessary for your design, click **Apply**. If all of your parameter specifications are valid, coreConsultant writes configured HDL source code to your workspace, and the Report tab of the Specify Configuration Activity View is displayed which allows you to generate and view report and generate optional views of the current configuration of the core.

To complete the Specify Configuration activity in command line mode, use the `set_configuration_parameter` and `autocomplete_activity` commands. The `autocomplete_activity <activity_name>` is equivalent to the graphical “Apply” button. For example:

```
set_configuration_parameter delay_cycles 8
autocomplete_activity SpecifyConfiguration
```

When the Specify Configuration activity completes, the <workspace>/src directory is populated with Verilog or System Verilog, and IP-XACT (.xml) descriptions of the component.

Next, complete the gate-level netlist activities described in “[Creating the Gate-Level Netlist for a Core](#)” on page 59.

3.1.4 Area Estimate Report

After you have completed the Specify Configuration activity, you can generate an Area Estimate report. This report is generated by collecting area data from the component, if available. For information on how this data is added to a component in coreBuilder (by the core developer), see the “Area Estimation Modeling” section in the *coreBuilder User Guide*.

The Area Estimate report is a report for the component in the workspace. This report includes links to individual component reports, and will provide values for each technology label. [Table 3-1](#) shows a typical Area Estimate report.

Table 3-1 Area Estimate Report

Area	Technology	Match Weight Percentage	Matching Parameters
10072	090	18.9	2/5
11111	090	17.0	2/4
8000	090	15.4	1/4

[Table 3-2](#) shows a detail report, linked from the “Matching Parameters” column.

Table 3-2 Area Estimate Detail Report - “Area: 8000, Technology: 090, Match Percentage:15.4”

Parameter	Weight	Estimation Value	Actual Value
P1	0	1	2

Table 3-2 Area Estimate Detail Report - “Area: 8000, Technology: 090, Match Percentage:15.4” (continued)

Parameter	Weight	Estimation Value	Actual Value
P2	7	7	1
P3		2	0

If no area estimation numbers are contained in the components, the report is empty, and a message informs you of this lack of data.

4

Creating the Gate-Level Netlist for a Core

The gate-level view of a core is a translation of the RTL view into a technology-specific netlist, which may include physical placement information.

This chapter contains the following sections:

- [“Specifying Target Technology \(TechSetup\)”](#) on page 60
- [“Technology Cell Binding”](#) on page 63
- [“Specifying Clocks”](#) on page 67
- [“Specifying Operating Conditions”](#) on page 68
- [“Specifying Port Constraints”](#) on page 70
- [“Specifying Power Intent”](#) on page 73
- [“Specifying Synthesis Methodology”](#) on page 75
- [“Specifying Test Methodology”](#) on page 78

4.1 Specifying Target Technology (TechSetup)

The Specify Target Technology activity specifies the technology libraries for ASIC synthesis. The collective settings are called the TechSetup.

When you select the Specify Target Technology item in the Activity List pane, the Library Setup tabs appear in the Activity View pane. Prior to showing this pane for the first time, your setup file (.synopsys_dc.setup) is read to determine initial values for the fields.

4.1.1 Logical Library Setup

The first tab of the specify Target Technology activity contains the search path and libraries associated with dc_shell or fc_shell. These parameters are described in [Table 4-1](#).

Table 4-1 Logical Library Setup Parameters

Activity Parameters	Description
SearchPath	Specify the paths to your technology library (db and NDM libraries) and any other libraries required for your design. The paths here work for Design Compiler and Fusion Compiler.
TargetLibrary	Specify the names of your technology library files (.db) and folders (.ndm). If your target .db library uses a separate file to characterize cells for best-case operating conditions (minimum timing), specify it in the MinLibrary parameter. Notes: ■ If no NDM libraries are specified in this field, the Fusion_Compiler_Reference_Methodology synthesis strategy will not be available, as covered in “Choosing a Synthesis Strategy” on page 94. ■ Only one NDM library can be specified in this field. ■ An NDM library specified here should be the first of any NDM libraries specified for the LinkLibrary parameter described in this table.
LinkLibrary	Specify the names of your technology library files and folders (.sldb, .db, and .ndm) and any other libraries required to link your design. For the names of specific libraries that must be linked to synthesize the core, see the documentation supplied with your core. Note: ■ If an NDM library is specified for the TargetLibrary parameter, it should be the first of any NDM libraries specified here.
MinLibrary	If your technology library uses a separate .db file to characterize cells for best-case (minimum timing) operating conditions, enter the name of it in this field. You can also specify this parameter as a list of the form {{worst-case_library_A best-case_library_A} {worst-case_library_B best-case_library_B} ...}. List specification of this parameter supports multiple min-timing libraries. A min-timing library can be associated with any library currently specified in the target or link library lists. Note: ■ There is no need to specify NDM library information in this field.

4.1.2 Physical Library Setup

The Physical Library Setup is used by DC-Topographical. You must set the `dc_shell` tool root in order to use DC-Topographical. These parameters are defined in [Table 4-2](#).

Table 4-2 Physical Library Setup Parameters

Activity Parameters	Description
Milkyway Design Library	This value corresponds to the Design Compiler variable <code>mw_design_library</code> , which is the full path to the Milkyway design library. The Milkyway design library must exist before specifying this parameter, since coreTools will not create it.
TechFileFormat	Select “tf” to use the techfile to create the Milkyway design library, or “plib” to use the physical library source (.plib) file.
TechFile	Technology file used to create the Milkyway design library.
CreateMwDesignOptions	Additional options used to create the Milkyway design library (see Milkyway documents).
MaxTluPlusFile	Max TLU+ file. Not required (defaults to synthesis strategy value)
MinTluPlusFile	Min TLU+ file. Not required (defaults to synthesis strategy value)
Milkyway Reference Library	Specify the full path to the Milkyway reference library (same value as the Design Compiler variable <code>mw_reference_library</code> . The library must exist before specifying this parameter.
Milkyway Logic 0 Net	Corresponds to the DC variable <code>mw_logic0_net</code> .
Milkyway Logic 1 Net	Corresponds to the DC variable <code>mw_logic1_net</code> .

4.1.3 Set Tool Installation Root

If the entry fields are not enabled, it means that you do not have a path defined for access to `dc_shell-t`. To resolve this problem, specify the tool installation areas using the **Edit > Tool Installation Roots** from the toolbar menu. This dialog box also contains an **Update Tool Roots** button that sets the tool roots to values determined from the current environment.



Note

Synthesis tools that are not suitable for a specific core are permanently disabled for that core and do not appear as options in coreConsultant.



Hint

To use 64-bit tool binaries within coreConsultant (64-bit platforms only):

- In the coreConsultant application, choose Edit -> Tool Installation Roots.
- Check the “64 Bit?” box for each tool to direct coreConsultant to use the 64-bit tool binaries. Most tools have a command line switch used by coreConsultant to enable 64-bit mode.
- Make sure that supporting tools like Vera also point to the 64-bit installation; otherwise Verification may fail. When using Vera, you must set the `VERA_HOME` such that it picks up the 64-bit binaries (see Vera documentation for details). Doing this also checks the 64-bit box for all appropriate simulation tools.

4.1.4 Library Setup in Command Line Mode

To complete the Specify Target Technology activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands. For example:

```
set_activity_parameter TechSetup SearchPath \  
  {./ /tools/synopsys/2000.11/libraries/syn /libraries/my_tech_lib}  
set_activity_parameter TechSetup dc_shellRoot /tools/synopsys/2000.11  
set_activity_parameter TechSetup LinkLibrary \  
  {* core_slow.db cells_slow.db dw_foundation.sldb}  
set_activity_parameter TechSetup fm_shellRoot /tools/formality  
set_activity_parameter TechSetup TargetLibrary {core_slow.db cells_slow.db}  
autocomplete_activity TechSetup
```

4.2 Technology Cell Binding

The optional Technology Cell Binding activity appears if your core includes the ability to map core cells to cells in your technology libraries.

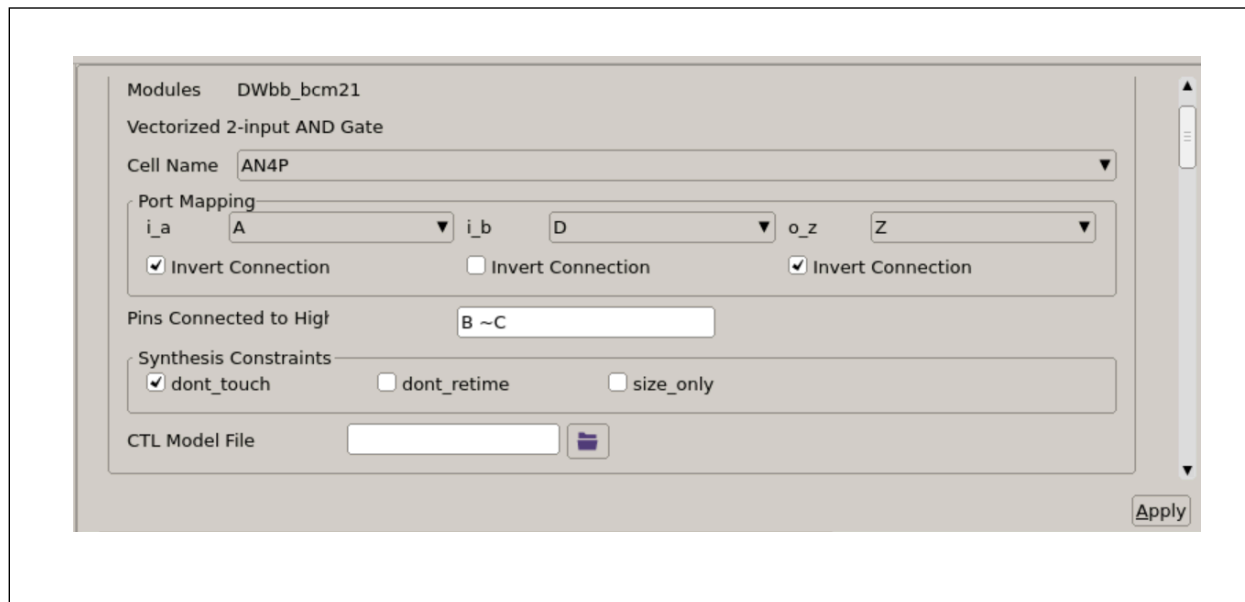
When you select the Technology Cell Binding activity, a form appears for you to map core cells to technology-hardened cells in the target libraries identified during the Specify Target Technology activity. A set of fields appears for each core cell that can be mapped. [Table 4-3](#) describes this set of fields.

Table 4-3 Technology Cell Binding Activity Fields

Field	Description
Macro	Macro name associated with the core cell that can be mapped For a description of the core cell, hover the mouse over the macro name or right-click on it and choose "What's this".
Cell Name	Choose the library cell to map to This drop-down menu contains cells from your technology library.
Port Mapping	Map the core cell ports to the library cell ports that appear in the drop-down after you select a technology cell The ports in the drop-down match the direction of the corresponding port in the core cell.
Invert Connection	Select to negate a selected port from the dropdown list or leave it unchecked if negation is not required.
Pins Connected to High or Low	Write to the list of unconnected tech cell pins that are to be connected to logic 1 or logic 0.
Synthesis Constraints	Select the synthesis constraints (such as dont_touch) you want for the technology cell; some synthesis constraints might be checked by default.
CTL Model File	CTL model file to describe the test behavior of the technology cell. This file is read in DC/FC along with the user specified CTL model files set in the designs or the strategy parameter in FCRM/DCRM .

Figure 4-1 shows the snapshot of the TCB GUI with the parameters listed in Table 4-3 .

Figure 4-1 TCB GUI Snapshot



When you complete the mapping entries and click Apply, the Technology Cell Binding report is shown in the Report tab. The reports include the following:

- Binding Commands:

A list of the `bind_tech_cell_mapping` commands that are executed in the background. For information about the `bind_tech_cell_mapping` command, see the online command reference.

- Files:

A table of information about the cells that were mapped. This includes a link to the RTL file where coreConsultant inserted a description and a macro line near the top that summarizes the mapping.

- Constraint File:

A link to the constraints file that is created at `<workspace>/export/tech_cell_binding_attributes.tcl`. This file is read and used during synthesis.

If you want to define your own cells to include in this mapping activity, you can use the `create_custom_tech_cell` command. For information about this command, see the online command reference.

4.2.1 Tech Cell Mapping Use Case

Technology cell definition commands (`define_tech_cell_mapping`) are issued in coreBuilder and are packaged in the coreKit. Technology cell binding commands (`bind_tech_cell_mapping`) are issued in coreConsultant and coreAssembler before the Technology Cell Binding activity starts. This activity inserts the tech cell definitions into the RTL files (specified in the definitions) as macros within `ifdef` or `ifndef` blocks. An example of the RTL transformation is shown in Table 4-4.

Table 4-4 Commands Issued

coreBuilder	coreConsultant/coreAssembler
<pre>define_tech_cell_mapping -module BCM61 \ -macro BCM61_map \ -instance I061 \ -description "Sample tech map" \ -port_names {r1 r2 r3} \ -file "compA.v" \ -port_descriptions {{r1 port} {r2 port} {r3 port}}</pre>	<pre>bind_tech_cell_mapping -module BCM61 \ -macro BCM61_map \ -tech_cell OR2 \ -port_map {{r1 A} {r2 B} {r3 Z}}</pre>
<pre>define_tech_cell_mapping -module BCM00 \ -macro BCM00_map \ -instance I000 \ -description "Sample tech map" \ -port_names {m1 m2 m3} \ -port_descriptions {{m1 port} {m2 port} {m3 port}} \ -enable_macro_args \ -extra_macro_args "a1 a2 a3" -file "master.v" \ -width "[WIDTH-1:0]" \ -filter "Name=~ADDF" \ -output_inversion_suffix "INVERTED_OUT" -required</pre>	<pre>bind_tech_cell_mapping -module BCM00 \ -tech_cell OR2 \ -port_map {{m1 A} {m2 B} {m3 Z 1}} \ -attribute {dont_touch size_only dont_retime}</pre>

**Note**

Command reference manual should be referred for additional details about the options and usage examples.

The `bind_tech_cell_mapping` commands should be issued at the top level in a `coreAssembler` subsystem. An example of the modifications are shown in [Table 4-5](#).

Table 4-5 Modifications in RTL for Tech Cell Mapping

Original Component Source	Modified Component Source After Tech Cell Mapping
<pre>module compA(r1, r2, r3); input r1, r2; output r3;</pre>	<pre>`ifdef ENABLE_TECH_CELL_MAPPING // Technology mapping for macro: BCM61 // Module: BCM61_map // Description: Sample tech map // Technology cell used: AN2</pre>

Table 4-5 Modifications in RTL for Tech Cell Mapping (continued)

Original Component Source	Modified Component Source After Tech Cell Mapping
<pre>// Technology cell insertion. `ifdef BCM51_map `BCM51_map `else assign r3 = r1 & r2; `endif endmodule</pre>	<pre>`define BCM61_map AN2 I061(.A(r1),.B(r2),.Z(r3)); `endif module compA(r1, r2, r3); input r1, r2;output r3; // Technology cell insertion. `ifdef BCM51_map `BCM51_map `else assign r3 = r1 & r2; `endif endmodule // undefining the defined tech cell macros defined earlier `ifdef BCM61_map `undef BCM61_map `endif</pre>

**Note**

To successfully synthesize your design with the Technology cell bound RTL, you to enable the 'ENABLE_Tech_CELL_MAPPING' macro during the synthesis run.

For Synopsys IP, the macro is undefined in the undef file from the component source; for custom IP, they are undefined in the file where the macro was introduced (for custom technology cells) or the files specified with the -file option in define_tech_cell_mapping commands.

To prevent the synthesis tool from optimizing away these inserted special cells or setting timing/area specific attributes on them, a tech_cell_binding_attributes.tcl file is exported during the Technology Cell Binding activity. A sample of the contents of a tech_cell_binding_attributes.tcl file is shown as follows:

```
set cells [get_cells -quiet -hierarchical -filter ref_name==AN2]
if {[sizeof_collection $cells]} {
set_dont_retime $cells
}
```

4.3 Specifying Clocks

The coreKit identifies and names all clocks in the design. As a core integrator, you need to specify the clock attributes according to:

- Actual clock period and waveform
- Clock network delays

When you select the Specify Clock(s) activity on the Activity List, the corresponding Activity View pane appears. On the Real and Virtual Clocks tab, specify values for each of the attributes listed in [Table 4-6](#). The coreConsultant online reference provides detailed descriptions of these attributes.

Table 4-6 Clock Attributes

Attribute	Description
CycleTime	The clock period time.
Waveform	Specifies the rise and fall edge times of the clock over an entire clock period. (By default, it uses library time units). The default value for Waveform is {0 CycleTime/2}.
TestClock	This clock is to be used in test mode.
TestClockCycleTime	The cycle time to be used for a clock in test mode.
TestClockWaveform	Waveform for this clock in test mode.
FixHold	Specifies whether to fix hold time violations for the selected clock. If FixHold is true, coreConsultant generates a set_fix_hold command, which causes Design Compiler to insert a delay, if necessary, to fix hold time violations at registers driven by the selected clock.
ClockRiseLatency/ ClockFallLatency	The time it takes for a clock signal to propagate from the clock definition point to the clock pin of a register, for a rising or falling transition, respectively, at the clock definition point.
ClockSourceRiseLatency/ ClockSourceFallLatency	The time it takes for a clock signal to propagate from its actual ideal waveform origin point to the clock definition point, for a rising or falling transition, respectively, at the waveform origin point.
ClockSetupUncertainty/ ClockHoldUncertainty	The setup and hold uncertainties, respectively, applied to all endpoints of a specified clock.
MinTransitionDelay/ MaxTransitionDelay	The minimum and maximum transition times, respectively, for an ideal clock.

Numeric values can be entered as fixed numeric values, as fixed numeric values with a time unit, or as a formula related to the cycle time. When time units are not specified, the units of the main target library are used. For example, assuming the technology library units are 'ns' and assuming a 10 ns cycle time, then each of the following entries would specify the same 10 nanosecond value: 10, 10ns, 10000ps, =percent_of_period 10.

On the Generated Clocks tab of the Specify Clock(s) activity, you specify attributes associated with each of the generated clocks in your design. At a minimum, you must define the CycleTime and/or WaveForm.

4.4 Specifying Operating Conditions

When you select the Specify Operating Conditions and Wireloads activity on the Activity List, the corresponding Activity View pane appears. Set values according to your knowledge of the environment in which the core is to be synthesized and integrated. At the very least, you must specify a value for `OperatingConditionsWorst`. [Table 4-7](#) defines the operating conditions and wireloads attributes.

Table 4-7 Operating Conditions and Wireloads Attributes

Attribute	Description
<code>ParentWireLoad</code>	Specifies the wireload model to be used for the design that is one level above the selected design. For a core integrated at the chip level, the <code>ParentWireLoad</code> for the core is the wireload model to be used for the chip.
<code>WireLoadMode</code>	Specifies the wireload mode for the selected design. Valid values are <code>top</code> , <code>enclosed</code> , and <code>segmented</code> . You select wireload models for the core and its subblocks as part of the Specify Synthesis Methodology activity. If you do not specify <code>WireLoadMode</code> , <code>coreConsultant</code> uses the library's default wireload mode (if one exists).
<code>WireLoadGroup</code>	Specifies the wireload model selection group for automatic wireload model selection. If you do not specify a <code>WireLoadGroup</code> , and none exists to inherit from the design at the next higher level, <code>coreConsultant</code> uses the library's default value (if one exists).
<code>WireLoadMinBlockSize</code>	Specifies the minimum block size for automatic wireload selection.
<code>Wireload</code>	The wireload model used for synthesis. If you do not specify a value explicitly, a wireload model is automatically selected as follows: ■ If <code>AreaEstimate</code> is set, the wireload for the initial compile is based on the values for <code>AreaEstimate</code> (if specified). It then switches to Design Compiler's automatic wireload selection for subsequent, incremental compiles. ■ If <code>AreaEstimate</code> is not set, Design Compiler's automatic wireload selection is used for all compiles.
<code>OperatingConditionsWorst</code>	Specifies the worst-case operating conditions to use for maximum delay (setup time) timing analysis. Your choice impacts both optimization and timing analysis.
<code>OperatingConditionsBest</code>	Specifies the best-case operating conditions to use for minimum delay (hold time) timing analysis. Your choice impacts both optimization and timing analysis.

To complete the Specify Context Information activity in command line mode, use the `set_activity_incomplete`, `current_design`, `set_clock_attribute`, `set_design_attribute`, and `autocomplete_activity` commands.

For example:

```
set_activity_incomplete SpecifyContextInformation
current_design TOP
set_clock_attribute clk CycleTime 20
```

```
set_design_attribute OperatingConditionsWorst slow_125_1.62  
autocomplete_activity SpecifyContextInformation
```

4.5 Specifying Port Constraints

The coreKit specifies technology-independent port constraint attribute values. These specifications are based on knowledge of the design and its likely target chip environment, and are the values you would normally see in a typical synthesis SDC file. coreConsultant uses the port attributes to generate port constraints for the core and its sub-blocks. When you select the Specify Port Constraints activity on the Activity List, the corresponding Activity View pane appears.

Table 4-8 provides brief descriptions of the port attributes.

Table 4-8 Port Attributes

Attribute	Description
MinInputDelay, MaxInputDelay, MinOutputDelay, MaxOutputDelay	Core developers specify delay values based on core timing performance and assumed chip environment. You may need to modify delay values to match the actual design environment in which you are synthesizing the component cores. NOTE: Depending on the setting of the Synthesis preference “Show Rise & Fall Delays Separately”, there will either be these four mentioned attributes or eight attributes will appear (MinRiseInputDelay, MinFallInputDelay, and so on.)
MinInputDelayFallingEdge[], MaxInputDelayFallingEdge[], MinOutputDelayFallingEdge[], MaxOutputDelayFallingEdge[]	The minimum and maximum input delays, respectively, for the input port with respect to the falling edge of the specified clock. If only one clock exists in the design, you do not need to name it through the subscript. NOTE: Depending on the setting of the Synthesis preference “Show Rise & Fall Delays Separately”, there will either be these four mentioned attributes or eight attributes will appear (MinRiseInputDelayFallingEdge, MinFallInputDelayFallingEdge, and so on.)
PinLoadType, PinLoadCount	Core developers specify technology-independent values based on the expected chip environment. If you have more specific values derived from your design and target technology, specify them before your initial synthesis.
DrivingCell	Core developers specify technology-independent values based on the expected chip environment. If you have more specific values derived from your design and target technology, specify them before your initial synthesis.
IdealPort	When true, this attribute implies infinite drive with no resistance. Therefore, directly attached nets are not checked for transition or capacitance violations. Set IdealPort on ports such as reset and test ports that you plan to buffer using post-synthesis tools.
DontTouchNetwork	If true, a Design Compiler set_dont_touch_network command is generated for the port. This prevents the modification or replacement of cells and nets in the transitive fanout of the port during optimization.
CriticalTiming	Core developers set CriticalTiming to true if the input port is likely to be part of a critical timing path. Depending on your initial synthesis results, you may need to change the value.

Table 4-8 Port Attributes (continued)

Attribute	Description
InputResistance	Specifies the resistance on the net driven by an input port.
MaxFanout, MaxCap, MaxTransition	Core developers do not specify values for these attributes, because they are specific to your design and target technology. If you do not specify values for these attributes, Design Compiler uses the design rules defined in your technology library. You should specify these values when you wish to impose stricter design rules than those specified by your technology library.
DftExistingSignalType	Test signal to be used for the 'existing_dft' view of set_dft_signal. These are signals that describe structures that already exist and that must be understood for the design to pass DRC (dft_drc): clocks, resets, constants, and existing scan chains. Default value: None
DftExistingSignalActiveState	Active logic state for the signal. Default value: None
DftExistingSignalTiming	Waveform to be used for the test signal. Default value: None
DftExistingSignalTestMode	The name of the test mode to which the dft signal specification applies. Default value: None
DftSpecSignalType	Test signal to be used for the 'spec' view of set_dft_signal. These are signals that describe structures that DFT Compiler should use for insert_dft: scan in, scan out, scan enable, and so on. Default value: None
DftSpecSignalActiveState	Active logic state for the signal. Default value: None
DftSpecSignalTestMode	The name of the test mode to which the dft signal specification applies.
TestHold	Obsolete
ScanSignalType	Obsolete
DftSignalType	Obsolete

The first time you synthesize the core, check that the following attributes are suitable for the target environment:

- Input Ports – MinInputDelay, MaxInputDelay, DrivingCell
- Output Ports – MinOutputDelay, MaxOutputDelay, PinLoadCount, PinLoadType

To view (and possibly modify) attribute values for individual bits of a bus, select the port in the port list and left-click the Set Bit-level Intent button.

To complete the Specify Port Constraints activity in command line mode, use the `set_activity_incomplete`, `current_design`, `set_port_attribute`, and `autocomplete_activity` commands as shown in the following example:

```
set_activity_incomplete SpecifyPortIntent
current_design TOP
set_port_attribute data_out {MaxOutputDelay[clk]} {=percent_of_period 20}
autocomplete_activity SpecifyPortIntent
```

4.6 Specifying Power Intent

The Specify Power Intent activity enables you to customize Unified Power Format (UPF) objects present in your design. The initial values of attributes on these UPF items are extracted from the UPF file associated with this activity. You can modify these attribute values based on the targeted technology.



Note

The Specify Power Intent activity is only visible when UPF items are present in the coreKit.

When you select the Specify Power constraint activity on the Activity List, the corresponding Activity View pane appears. There are three tabs in the pane: Cell Specification, Voltage Specification, and SupplyVoltage Specification. The last tab is only visible when there is any set_voltage command present in the packaged UPF file.

Table 4-9 provides brief descriptions of the UPF attributes.

Table 4-9 UPF Attributes

Attribute	Description
UPFStrategy	Valid on isolation, level_shifter, and retention UPF items
UPFDomain	Valid on isolation, level_shifter, and retention UPF items
UPFLibraryCells	Valid on isolation, level_shifter, and retention UPF items; you can select the library cells from a list of cells available in the current technology library
UPFSupply	Valid on port_state and power_state
UPFSupplyExpr	Valid on power_state
UPFState	Valid on port_state and power_state
UPFStateValue	Valid on port_state
UPFVoltageType	Valid on all UPF items; you can select one from the list: {nom min/max nom/min/max}
UPFMin	Float value valid on all UPF items
UPFMax	Float value valid on all UPF items
UPFNom	Float value valid on all UPF items
UPFVoltage	Float value valid on UPF supply voltage items

Upon completion of the activity, power intent files packaged with the coreKit are updated and written in the top-level folder <workspace>/upf.

A UPF_supply_<type>_<mode> item is created for every combination of voltage type and voltage mode coming from the get_supply_voltage command in the intent file. Voltage type can be either power or ground. Similarly, voltage mode can be either best or worst. In batch mode, you can modify the set_voltage value using the set_supply_voltage command as follows:

```
set_supply_voltage UPF_supply_power_worst 3.14
```

To complete the Specify Power Intent activity in command line mode, use the `set_upf_attribute` and `autocomplete_activity` command as follows:

```
set_upf_attribute <UPF item name> <UPF attribute name> <value>
```

```
autocomplete_activity SpecifyPowerIntent
```

4.7 Specifying Synthesis Methodology

Synthesis methodology is the set of attributes that you apply to designs within the core design hierarchy. These attributes guide the coreConsultant synthesis strategy generation. The coreKit specifies these attribute values based on detailed knowledge of the design and its likely target environment.



Note

Typically, you do not need to modify the synthesis methodology specifications unless the synthesis results fail to meet your design goals.

When you select the Specify Synthesis Methodology activity on the Activity List, the corresponding Activity View pane appears. The Synthesis Methodology Activity View contains the following tabs:

- “General Synthesis Attributes”
- “Advanced Synthesis Attributes” on page 76

4.7.1 General Synthesis Attributes

These are the attributes that you might want to change if synthesis results do not meet your design requirements. [Table 4-10](#) describes these attributes in detail.

Table 4-10 General Synthesis Attributes

Attribute	Description
MapBlockIndividually	Determines whether the selected design is compiled as an individual unit.
Optimization\ Priorities	Specifies the optimization priorities for the selected design: ■ timing – Focus on achieving the best timing, giving minimal consideration to area impact. ■ timing_area – Achieve timing objectives first, then try to reduce area. This is the default. ■ area_timing – The design should easily meet timing objectives, so focus optimization on reducing area. ■ area – Focus on achieving the smallest design that meets timing constraints. (In some cases, area tradeoffs make it impossible to meet timing goals).
ScanStyle	Indicates the scan style to use for the selected design. The specified value is used directly as the value for the Design Compiler <code>set_scan_configuration</code> command. The value of ScanStyle is relevant only when you set the Test Ready Compile synthesis strategy parameter to true.
ScanExclude	Specifies a list of items that are not to be replaced by scan cells. A “set_scan_element false” command is generated for each specified item. You must set ScanExclude on the design on the implementation frontier that contains the specified item. The value of ScanExclude is relevant only when you set the Test Ready Compile synthesis strategy parameter to true.

4.7.2 Advanced Synthesis Attributes

These are more advanced synthesis attributes that do not generally require modification. The most common circumstances for modifying these attributes is to add to your own auxiliary script. [Table 4-11](#) describes these attributes in detail.

Table 4-11 Advanced Synthesis Attributes

Attribute	Description
Wireload	The wireload model used for synthesis. If you do not specify a value explicitly, a wireload model is automatically selected as follows: ■ If AreaEstimate is set, the wireload for the initial compile is based on the values for AreaEstimate (if specified). It then switches to Design Compiler's automatic wireload selection for subsequent, incremental compiles. ■ If AreaEstimate is not set, Design Compiler's automatic wireload selection is used for all compiles.
TclAuxSynthesis Script[setup]	Executes the Design Compiler Tcl-mode script before any other processing.
TclAuxSynthesis Script[elab]	Executes the Design Compiler Tcl-mode script before elaboration.
TclAuxSynthesis Script[constrain]	Executes the Design Compiler Tcl-mode script before setting constraints.
TclAuxSynthesis Script[dcrmlInitial]	Executes the Design Compiler Tcl-mode script immediately after SDC constraints are applied. (used in the Design_Compiler_Reference_Methodology synthesis strategy)
TclAuxSynthesis Script [dcrmPathGroupScript]	Executes the Design Compiler Tcl-mode script after the existing group_path commands. (used in the Design_Compiler_Reference_Methodology and DCTCL_quick_map_strategy)
TclAuxSynthesis Script[cleanup]	Executes the Design Compiler Tcl-mode script after all processing is completed, just prior to quitting.
TclAuxSynthesis Script[report]	Executes the Design Compiler Tcl-mode script, which applies to report generation.
TclAuxSynthesis Script[dftInsertion]	Executes the Design Compiler Tcl-mode script, which applies to DFT insertion.
TclAuxSynthesis Script[dftCleanup]	Executes the Design Compiler Tcl-mode script, which applies to DFT cleanup.
TclAuxSynthesis Script[qmap]	Executes the Design Compiler Tcl-mode script prior to a specific optimization phase.
FormalVerification AuxScript	Specifies the script to be run in Formality after the reference and implementation designs have been loaded, but just before the verify command.

Table 4-11 Advanced Synthesis Attributes (continued)

Attribute	Description
StaticTimingAuxScript	Points to an auxiliary script to be executed after the design has been loaded and constrained in PrimeTime, but before the update_timing command runs the analysis.
AtpgTclAuxScript [setup]	Specifies the TestMAX TCL-mode scripts to be run just before the setup phase of test vector generation.
AtpgTclAuxScript [drc]	Specifies the TestMAX TCL-mode scripts to be run just before the design rules check phase of test vector generation.
AtpgTclAuxScript [run]	Specifies the TestMAX TCL-mode scripts to be run just before the run phase of test vector generation.
AtpgTclAuxScript [cleanup]	Specifies the TestMAX TCL-mode scripts to be run just before the cleanup phase of test vector generation.
FcAuxsScript [post_init_design]	Specifies a Fusion Compiler script at the very end of init_design stage before save_block.
FcAuxsScript [pre_read_rtl]	Specifies a Fusion Compiler script to be run just before reading the RTL.
FcAuxsScript [post_read_rtl]	Specifies a Fusion Compiler script to be run just after the RTL is read.
FcAuxsScript [pre_compile]	Specifies a Fusion Compiler script to be run just before compile.
FcAuxsScript [post_compile]	Specifies a Fusion Compiler script to be run just after compile.
FcAuxsScript [pre_compile_unified]	Specifies a Fusion Compiler script in the UNIFIED_FLOW before compile_fusion -from final_place.
FcAuxsScript [pre_place_opt]	Specifies a Fusion Compiler script just before placement optimization (place_opt).
FcAuxsScript [post_place_opt]	Specifies a Fusion Compiler script just after placement optimization (place_opt).
FcAuxsScript [write_data]	Specifies a Fusion Compiler script just before data is written out.
FcAuxsScript [report]	Specifies a Fusion Compiler script after data is written out; can be used to include report generation commands.
FcAuxsScript [cleanup]	Specifies a Fusion Compiler script after all processing is completed, just prior to quitting.

4.8 Specifying Test Methodology

In this activity, you set and/or review Design for Test attributes on designs. When you select the Specify Test Methodology activity on the Activity List, the corresponding Activity View pane appears. The Test Methodology Activity View contains the following tabs:

- [“General Test Methodology Attributes”](#)
- [“Scan Insertion”](#)
- [“Auto Fix”](#) on page 81
- [“Test Wrappers”](#) on page 82
- [“Wrapper Integration”](#) on page 83
- [“Built-in Self Test”](#) on page 84
- [“Testpoint Insertion”](#) on page 86

4.8.1 General Test Methodology Attributes

[Table 4-12](#) provides information on common scan test attributes.

Table 4-12 General Test Attributes

Attribute	Description
DftAuxSynthesis	Script Specifies the name of the script to use to perform insert_dft. The script may do all of the setup for DFT and run all of the commands, in which case InsertDft should be set to false. The script may also be used to issue more setup commands in addition to those selected in the DFT options below. If InsertDft is set to true and a script is specified, the script is run after all of the setup commands have been issued, but just before running insert_dft.
ScanSubblocksIndividually	Causes scan insertion to be performed on the subdesigns of this design. Default Value = 1 for top design, 0 elsewhere.
ScanBlockIndividually	Causes scan insertion to be performed on this design. Default Value = expr {[InheritValue up -attr ScanSubblocksIndividually 0] [get_attribute %item -attr WrapBlockIndividually] [get_attribute %item -attr BistBlockIndividually]}
TestClockDefaultCycleTime	Default cycle time for test clocks. Default value: 100

4.8.2 Scan Insertion

[Table 4-13](#) provides information on scan insertion attributes.

Table 4-13 Scan Insertion Attributes

Attribute	Description
ScanMethodology	Scan style to be used.

Table 4-13 Scan Insertion Attributes (continued)

Attribute	Description
	Enum values = full_scan, none. Default Value = InheritValue up full_scan
Scan Style	Sets the test scan implementation style for this design. Enum values = multiplexed_flip_flop, clocked_scan, lssd, aux_clock_lssd, combinational, none. Default Value = multiplexed_flip_flop
Scan Exclude	List of sequential cells, hierarchical cells containing sequential elements, references, library cells, or designs that should not be replaced by scan cells.
IdentifyShiftRegisters	When true, insert_dft/compile -scan identifies shift-registers in the design and keeps only the first flop as a scan flop. In preview_dft/insert_dft, Scan Architecting groups the flops that form shift registers as a single scan group that is named shift_register_identified_??. The preview_dft -show all command shows the shift registers that are identified. preview_dft -script writes out the shift-registers as scan groups. If the scan style is not multiplexed_flip_flop, insert_dft/compile -scan ignores this option.
EnableScanCompression	When true, enables scan compression. For details on DFTMAX compression options, see the following in the DFTMAX User Guide: ■ Using DFTMAX Compression ■ Managing X Values in Scan Compression For related information, see also ScanCompressionConfiguration.
Show 'advanced' attribute columns (checkbox enabled)	
NumberOfScanChains	The number of scan chains to be created. Setting the value to 0 allows DFT Compiler to determine the number of scan chains. Default Value = 0
MaximumScanChainLength	The maximum length a scan chain may be. Setting the value to 0 allows DFT Compiler to determine the maximum scan chain length. Default Value = 0
ClockMixing	Specifies whether cells from different clock domains may be put in the same scan chain. Enumerated Values: ■ no_mix: Cells must be clocked by the same edge of the same clock. ■ mix_edges: Cells must be clocked by the same clock, but the clock edges can be different.

Table 4-13 Scan Insertion Attributes (continued)

Attribute	Description
	■ mix_clocks_not_edges: Cells must be clocked by the same clock edge, but the clocks can be different. ■ mix_clocks: Cells can be clocked by different clocks and different clock edges. Default Value = InheritValue up no_mix
TestClocksFollowSystemDomains	When TRUE (1), dedicated test clocks (for example, test_scan_clock_a) are created according to different system clocks. Thus, scan cells that have different system clocks have different test clocks. When FALSE (0), test clocks are created without considering system clocks. Default Value = InheritValue up 0
AddLockUpLatch	When TRUE (1), lockup latches are inserted between clock domain boundaries on scan chains. Default Value = InheritValue up 1
InsertEndOfChainLockupLatch	When TRUE (1), lockup latches are inserted at the end of scan chains. Default Value = InheritValue up 0
DedicatedScanPorts	Determines whether dedicated scan out ports are created, or whether the functional ports are used as scan ports. Default Value = InheritValue up 0
BidirectionalMode	Determines whether bidirectional ports are treated as inputs, outputs, or left untouched during scan shift. Enumerated Values = input, output, no_disabling Default Value = InheritValue up input
ExternalTristates	Determines the disabling option during scan shift for all tristate nets that drive output ports of a design. ■ disable_all: disables all drivers ■ enable_one: disables all but one driver ■ no_disabling: specifies not to insert disabling logic Enumerated Values = disable_all, enable_one, no_disabling Default Value = InheritValue up disable_all
InternalTristates	Determines the disabling option during scan shift for all tristate nets that do not drive design output ports. Enumerated Values: ■ disable_all: disables all drivers ■ enable_one: disables all but one driver ■ no_disabling: specifies not to insert disabling logic

Table 4-13 Scan Insertion Attributes (continued)

Attribute	Description
	Default Value = InheritValue up disable_all
HierarchicalIsolation	When true, hierarchical isolation logic is built, so that dedicated subsystem scan-out signals are gated by the design scan enable signal. Default Value = 0
ScanInternalClocks	When true, internal clocks are treated as separate clocks for creating scan chains. Setting this attribute to false is equivalent to '-internal_clocks none'. When this attribute is set to TRUE it is used in conjunction with ScanJumpBuffersInverters. Default value: FALSE
ScanJumpBufferInverters	Only valid when ScanInternalClocks = TRUE. The value of this attribute is the value of the -internal_clocks option to set_scan_configuration. Default value = single
ScanReplace	Should non-scan registers be replaced by scannable registers? When true, insert_scan replaces sequential cells with scan cells, if the sequential cells are not violated by scan design rule checking. Default Value = 0
ReuseMVCells	Controls the reuse of existing MV isolation and level-shifter cells.
PowerDomainMixing	Specifies that insert_dft can mix scan elements from different power domains in the same scan chain. Default Value = 0
VoltageMixing	Allow insert_dft to insert scan elements from different voltage domains into the same scan chain.
ScanCompressionConfiguration	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, see the following in the DFTMAX User Guide: ■ Using DFTMAX Compression ■ Managing X Values in Scan Compression Note that when using set_design_attribute, a blank space must be the first character. When using set_attribute, a leading blank space is not necessary.

4.8.3 Auto Fix

Table 4-14 on page 82 provides information on auto fix test attributes.

Table 4-14 Auto Fix Attributes

Attribute	Description
EnableDftAutoFix	Enables the DFT Compiler Auto Fix utility. Default Value = InheritValue up FALSE
AutoFixClocks	Enables/disables automatic fixing of uncontrollable clock violations. Default Value = InheritValue up TRUE
AutoFixBidirectional	Enables/Disables automatic bidirectional disabling. Default value: InheritValue up TRUE
AutoFixAsync	Enables/disables automatic fixing of asynchronous preset/clear violations. Default Value = InheritValue up TRUE
AutoFixAsyncWithScanEnable	When TRUE, the test scan enable signal are used to autofix asynchronous set and reset violations. Default Value = InheritValue up FALSE
AutoFixAsyncLogicGate	When set to gate, specifies that the asynchronous sets and resets are autofixed with a gate driven by the test_mode signal. Enum Values = mux, gate Default Value = InheritValue up MUX
EnableDftShadowLogic	Enables the DFT Compiler Shadow LogicDFT for this design. Default Value = InheritValue up FALSE
AutoFixAsyncTestData	Port to be used as TestData for fixing set/reset violations. The port must have been defined with a DftExistingSignalType of Reset. Default value: none
AutoFixClockTestData	Port to be used as TestData for fixing clock violations. The port must be associated with a clock that has TestClock==1 or have been defined with a DftExistingSignalType of a clock type. Default value: none

4.8.4 Test Wrappers

Table 4-15 provides information on test wrapper attributes.

Table 4-15 Test Wrapper Attributes

Attribute	Description
WrapBlockIndividually	Causes test wrapper insertion to be performed on this design. Default Value = xdbist
WrapSubblocksIndividually	Causes test wrapper insertion to be performed on the subdesigns of this design.

Table 4-15 Test Wrapper Attributes (continued)

Attribute	Description
	Default Value = 0
WrapperDefaultSafeState	Specifies the default safe state for the wrapper cells. Enum Values = 0, 1, none Default Value = none
Show 'advanced' attribute columns (checkbox enabled)	
WrapperUseRegisterIO	Enables shared wrapper cells to be used for register-bound ports. The registers are reused in core wrapper chains. Default Value = 0
WrapperRegisterImplementation	If this attribute is set to 'swap,' registers connected to ports of the design are replaced with equivalent shared wrapper cells. If the value is set to 'in_place,' additional glue logic is added to registers attached to ports of the design to reuse these registers in core wrapper chains. Enumerated Values = swap, in_place Default Value = swap
DedicatedWrapperCell	Specifies the default dedicated wrapper cell type. Enumerated Values = WC_D1, none Default Value = WC_D1
SharedWrapperCell	Specifies the default shared wrapper cell type. Enumerated Values = WC_S1, none Default Value = WC_S1
OneWrapperClock	If set to TRUE (1), a single wrapper clock is used for all wrapper cells in the design. If shared wrapper cells are used in wrapping the design, the functional clocks of the shared wrapper cells are muxed with the wrapper clock in wrapper test modes by the core wrapper application. If the option is set to FALSE (0), a dedicated wrapper clock is used for all dedicated wrapper cells of the design and functional clocks are used for shared wrapper cells of the design. Default Value = 0
WrapperExcludePorts	List of ports that should not be wrapped. Since this reduces test coverage, you should exclude output ports only when necessary. You should never exclude input ports other than test signals, test holds, clocks, and other asynchronous signals from being wrapped. Values = list, port_1, port_2,...port_n

4.8.5 Wrapper Integration

Table 4-16 on page 84 provides information on wrapper integration attributes.

Table 4-16 Wrapper Integration Attributes

Attribute	Description
IntegrateTestWrappers	Specifies that all wrapped subdesigns of this design should be integrated using SOC methodology. If set to TRUE, all wrapper control signals of integration-ready subdesigns are hooked up to the top-level ports during the DFT insertion process. Also, a test controller is implemented to allow testing of integration-ready subdesigns in various test modes. Default Value = 0
IntegrationModeDecodingStyle	Specifies the type of port decoding to be used for the core integration test controller. Enum Values = default, one-hot Default Value = default

4.8.6 Built-in Self Test

Table 4-17 provides information on built-in self test attributes.

Table 4-17 Built-in Self Test Attributes

Attribute	Description
BistBlockIndividually	Causes insertion of BIST logic into this design. Default Value = InheritValue up -attr BistSubblocksIndividually
BistSubblocksIndividually	Causes BIST logic to be inserted into the subdesigns of this design. Default Value = 0
BistMode	BIST operations to be performed. Choosing “all” causes the design to be made bist-ready and the code to be inserted and integrated with the controller. For hierarchical BIST insertion, choose codec_insertion for the lower levels of hierarchy and integration at the level where the controller is to be integrated with the codecs. Choose bist_ready to make many BIST scan chains or channels within the design without X-propagation and without wrapping the design. Enum Values = all, bist_ready, codec_insertion, integration Default Value = all
BistType	Type of BIST to implement. Enum Values = dbist, xbist, sbist Default Value = dbist
Show 'advanced' attribute columns (checkbox enabled)	
BistCodecCount	Number of bist CODECs. Default Value = 1

Table 4-17 Built-in Self Test Attributes (continued)

Attribute	Description
BistMaxChainLength	Controls the size of the counter that increases the default upper limit on the length of the reconfigured scan chains. Setting BistMaxChainLength to 0 allows the DFT Compiler to determine the size of the counter. Default Value = 0
BalanceBistSegments	By default, SocBist rebalances the scan channels in BIST mode. Setting BalanceBistSegments to 0 (false) overrides this, and SocBist does not concatenate the scan segments. Default Value = 1
BistInvertPrpgClock	Invert the BIST clock that feeds the PRPG Registers. Default Value = 0
BistPrpgShadowSi	By default, DFT Compiler creates from one to twelve shadow scan chains for the PRPG shadow. The number of shadow scan chains created is optimized based on the length of the longest scan chain in the design. You can stipulate that DFT Compiler create a specific number of scan chains instead of setting BistPrpgShadowSi to 'n,' where n is an integer from 1 to 12. Setting BistPrpgShadowSi to 'default' allows DFT Compiler to determine the number of shadow scan chains. BistPrpgShadowSi does not affect the length of the longest scan chain, but only the number of shadow scan chains created. Enumerated Values = default, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 Default Value = default
BistSelectorShadowSi	The number of selector shadow chains. Setting BistSelectorShadowSi allows DFT Compiler to set the number of selector shadow chains based on the length of the longest scan chain. Enumerated Values = default, 1, 2, 3, 4 Default Value = default
BistObserveOutputs	Controls the number of scan output pins that are inserted into the design for BistMode == xdbist By default, the number of scan output pins created depends on the number of scan chains: scan_outputs = (scan_chains / 32). Setting this attribute to '0' lets DFT Compiler use the default value. Default Value = 0
BistPrpgLength	By default, SocBist creates a 478-bit PRPG. You can override this by setting BistPrpgLength to 257, creating a 257-bit PRPG. With DBIST, you can use up to 512 scan chains with the 257-bit PRPG. For X-tolerant DBIST, you can use up to 2048 scan chains with the 257-bit PRPG. Enumerated Values = 257, 479 Default Value = 479
BistDiagOutputs	The number of diagnostic outputs for DBIST.

Table 4-17 Built-in Self Test Attributes (continued)

Attribute	Description
	Enumerated Values = 1, 2, 4, 8, 16 Default Value = 1
BistUseTristateMux	Specifies that tristate muxes are used to construct the XDBIST codec. Default Value = 1
BistAutoFixBusses	Indicates floating bus and bus contention issues should be fixed automatically. Default Value = 1
BistAutoFixXprop	Indicates that fixing of x-propagation should be handled automatically. Default Value = 1 if (BistType=xbist dbist)

4.8.7 Testpoint Insertion

Table 4-18 provides information on testpoint insertion attributes.

Table 4-18 Testpoint Insertion Attributes

Attribute	Description
InsertTestPoints	Controls whether observe and control points are inserted into this design. Default Value = 0
TestabilityMethod	BIST testability to be used. Enum Values = bist, tdvr Default Value = bist
TestabilityControlSignal	Specifies the control signal to be used for test points. The signal should be predefined as having the TestMode signal type. If no control signal is specified, the test points are controlled by any signal with the TestMode signal type. If no TestMode signal is found, DFT Compiler creates a new TestMode port. Default value: none
TestabilityClockSignal	Specifies the clock to be used for control or observe registers. Default value: none
Show 'advanced' attribute columns (checkbox enabled)	
MaxControlPoints	Maximum number of control points to be inserted to enhance testability. When TestabilityMethod==control_and_observe, the larger of MaxObservePoints and MaxControlPoints are used for the -max_test_points value for set_testability_configuration. Default Value = 1000

Table 4-18 Testpoint Insertion Attributes (continued)

Attribute	Description
MaxObservePoints	Maximum number of observe points to be inserted to enhance testability. When TestabilityMethod==control_and_observe, the larger of MaxObservePoints and MaxControlPoints are used for the -max_test_points value for set_testability_configuration. Default Value = 1000
TestabilityMaxArea	Specifies the maximum percentage of area overhead due to the observe test point structure insertion in the design. Setting this attribute to 0 leaves this as unspecified in DFT Compiler. Default Value = 0
ControlPointsPerScanCell	Specifies the number of control points that can be shared per scan cell. When TestabilityMethod==control_and_observe the larger of ObservePointsPerScanCell and ControlPointsPerScanCell are used for the -test_points_per_scan_cell value for set_testability_configuration. Default Value = 8
TestpointPowerSavingOn	Adds logic and gate structure for power saving on the XOR tree. Default Value = 0
TestabilityClockType	Specifies the clock type to be used for testability. Enumerated Values = dominant, dedicated Default Value = dominant
ShareTestpointsAcrossHierarchy	Specifies that testpoint scan cells can be shared across hierarchy. Default Value = 0

5

Synthesizing a Core

This chapter explains how to synthesize the core and analyze the results to make sure they meet your design goals. It also explains how to refine your synthesis intent when the most recent synthesis results fail to meet your design requirements.

This chapter contains the following sections:

- [“Performing ASIC Synthesis”](#) on page 90
- [“Synthesis and Integration Methodologies”](#) on page 91
- [“Choosing Synthesis Activities in the GUI”](#) on page 94
- [“Launching a Synthesis Run”](#) on page 127
- [“Checking Synthesis Results”](#) on page 133
- [“Refining Your Synthesis Intent”](#) on page 134
- [“Generating ATPG Test Vectors”](#) on page 135

5.1 Performing ASIC Synthesis

To synthesize a core with coreConsultant, select the Perform ASIC Synthesis activity from the Activity List. The corresponding Synthesize Activity View appears. In this Activity View you can select a variety of synthesis options, including a synthesis strategy. After you select your options and click Apply, coreConsultant invokes the required synthesis tools to execute your selected strategy. You can also choose to generate the scripts for the selected strategy without actually initiating the synthesis job. If so, you can run the synthesis job later by invoking the top-level run script.

The options (parameters) for the Perform ASIC Synthesis activity fall into four general categories:

- **Strategy** – Enables you to choose from a list of available synthesis strategies. Each strategy requires you to select values for a set of options (strategy parameters), some of which are used to control different aspects of the synthesis process. See “[Choosing a Synthesis Strategy](#)” on page 94 for details.
- **Options** – The options related to synthesis execution, such as generating scripts only, and input/output file names. See “[Specifying Synthesis Execution Options](#)” on page 122.
- **Licenses** – Enables selection of licenses to be used during synthesis. Selecting a license means that coreConsultant can invoke synthesis tool features that require that license. See “[Viewing Licenses for Implementation Tools](#)” on page 123.
- **Reports** – Enables you to select to generate optional synthesis reports, in addition to the default set of reports that coreConsultant extracts from Design Compiler or Fusion Compiler. See “[Selecting Synthesis Reports](#)” on page 124.



Note

To use a project-specific Design Compiler setup file (.synopsys_dc.setup), place it in your *workspace/syn* directory.

When the synthesis run is complete, the results can be found in the *workspace/syn/stage* directory, where *stage* is the last completed synthesis stage. The stage names used depend on the strategy and strategy options. For example, a standard invocation based on the Design Compiler Reference Methodology/ Fusion Compiler Reference Methodology will leave results in the 'initial' directory. In all cases, a symbolic link named 'final' will link to the last generated stage directory.

The following section presents some information on using the available synthesis strategies to achieve the desired results.

5.2 Synthesis and Integration Methodologies

The methodologies for using the available synthesis strategies to achieve your desired results depend on the synthesis tools you have available. The following sections provide methodologies for:

- [“Working with Design Compiler Strategies”](#)
- [“Working with Fusion Compiler Strategies”](#)
- [“Working with Physical Synthesis Strategies”](#) on page 92
- [“Generating Timing Models”](#) on page 93
- [“Integrating Generated Scripts”](#) on page 93

5.2.1 Working with Design Compiler Strategies

The following steps show a typical coreConsultant activity flow for synthesizing and integrating a core, using Design Compiler as the synthesis tool.

1. Start with a configured core for which the synthesis intent has been specified.
2. Initially, use the [“Design_Compiler_Reference_Methodology”](#) on page 94 and the default QoR effort to establish a baseline set of results.
3. Synthesize the core.
4. Analyze the synthesis results (see [“Performing ASIC Synthesis Using QIK Reference Methodology for Synopsys Interface IP”](#) on page 129).
5. If results do not meet design objectives, modify intent or strategy values in the following order and re-synthesize:
 - a. Try setting QoR Effort to high.
 - b. Try relaxing overly pessimistic constraints.
 - c. Try over-constraining.
 - d. Try a newer version of Design Compiler (if applicable) to use the latest methodology and tool enhancements.
 - e. Iterate with step 3 as necessary (see [“Refining Your Synthesis Intent”](#) on page 134)
6. If results meet design objectives, modify intent or strategy values in the following order and re-synthesize:
 - a. Integrate the core into the application design (see [“Verifying Your Implementation”](#) on page 141).
 - b. Perform chip-level design verification and timing analysis (see [“Verifying Your Implementation”](#) on page 141).

5.2.2 Working with Fusion Compiler Strategies

The following steps show a typical coreConsultant activity flow for synthesizing and integrating a core, using Fusion Compiler as the synthesis tool:

1. Start with a configured core for which the synthesis intent has been specified.
2. Initially, use the [“Fusion_Compiler_Reference_Methodology”](#) on page 108 and the default QoR effort to establish a baseline set of results.

3. Synthesize the core.
4. Analyze the synthesis results (see [“Performing ASIC Synthesis Using QIK Reference Methodology for Synopsys Interface IP”](#) on page 129).
5. If results do not meet design objectives, modify intent or strategy values in the following order and re-synthesize:
 - a. Try setting QoR Effort to high.
 - b. Try relaxing overly pessimistic constraints.
 - c. Try over-constraining.
 - d. Try a newer version of Fusion Compiler (if applicable) to use the latest methodology and tool enhancements.
 - e. Iterate with step 3 as necessary (see [“Refining Your Synthesis Intent”](#) on page 134)
6. If results meet design objectives, modify intent or strategy values in the following order and re-synthesize:
 - a. Integrate the core into the application design (see [“Verifying Your Implementation”](#) on page 141).
 - b. Perform chip-level design verification and timing analysis (see [“Verifying Your Implementation”](#) on page 141).

5.2.3 Working with Physical Synthesis Strategies

The coreConsultant tool interfaces with Synopsys DC Graphical to implement a variety of physical synthesis strategies.

5.2.3.1 Setting Up for Physical Synthesis

For coreConsultant to use physical synthesis strategies, you must enable DC Graphical when you execute the Specify Target Technology activity and specify the following data for Physical Synthesis:

- Logical library (.db file) – The Physical Synthesis logical library is the same as the Design Compiler target library that you specify when you execute the Specify Target Technology activity. coreConsultant automatically uses your Design Compiler target library as your Physical Synthesis logical library.
- Physical synthesis – You specify a Milkyway Reference Library and Milkyway Technology File when you enable Physical Synthesis during the Specify Target Technology activity (Physical Library set-up tab), or the Physical Synthesis option of the Perform ASIC Synthesis activity. This may be a .tf or a .plib file. You may also provide optional files (TLUPlus) which allow the tools to model Deep Submicron effects with greater accuracy.
- Floorplan file – Floorplan information is optional and may be in DEF or TCL format.



Note

Fusion Compiler is a physically aware synthesis tool. If the required physical files have been provided, the tool runs physical synthesis. No additional settings are needed or needs to be enabled.

5.2.4 Generating Timing Models

Some synthesis strategies include an option to generate a timing model from the final synthesis results. coreConsultant uses Synopsys PrimeTime (license required) to generate the timing model. There are several options available to select how the model is generated. The modeling options are available on the Modeling tab of the Strategy Parameters dialog for the currently selected synthesis strategy (if supported by the strategy).

5.2.5 Integrating Generated Scripts

The typical approach for integration of a synthesized core into a chip is to synthesize in coreConsultant and then link the resulting database into the chip-level design. If desired, you can also integrate the core at the script level. In this case, you integrate the scripts generated by coreConsultant into your chip-level scripts. See the “Synthesis_API” man page for details.

5.3 Choosing Synthesis Activities in the GUI

The Perform ASIC Synthesis activity generates and optionally runs synthesis scripts. When you activate the activity by clicking on it in the activity list pane, the corresponding Activity View pane appears. The Synthesize Activity View pane contains the following tabs:

- Strategy (for details, see [“Choosing a Synthesis Strategy”](#))
- Options (for details, see [“Specifying Synthesis Execution Options”](#) on page 122)
- Licenses (for details, see [“Viewing Licenses for Implementation Tools”](#) on page 123)
- Reports (for details, see [“Selecting Synthesis Reports”](#) on page 124)

5.3.1 Choosing a Synthesis Strategy

On the Strategy tab you choose from a list of available synthesis strategies. The selected strategy can be configured to your needs by selecting the “Options” button to the right of the strategy name.

When you select a strategy from the drop-down list in the Strategy field, a detailed description of that strategy appears in the Description field to help you select the best strategy for your component. [Table 5-1](#) summarizes the available strategies and the remainder of this section contains a detailed review of each strategy.

Table 5-1 Summary of Synthesis Strategies

Strategy	Description
“Design_Compiler_Reference_Methodology” on page 94	Performs a one-pass, top-down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.
“Fusion_Compiler_Reference_Methodology” on page 108	Performs a one-pass, top-down compile from RTL using compile_fusion. This flow implements the Fusion Compiler Reference Methodology associated with the version of Fusion Compiler being used.
“DCTCL_quick_map_strategy” on page 120	Performs a quick, low-effort compile to generate a gate-level netlist of the core for analysis purposes only.

The specific set of strategies that are available to you depends on:

- The tools you enabled in the **Edit > Tool Installation Roots** dialog.
- Whether the core developer disabled the use of any tools that are not supported for the core.

Select a synthesis strategy from the pull-down menu and click the Options button to specify values for the strategy parameters. The following sections describe the available strategies (and their associated parameters), as well as the scenarios in which you should use them.

5.3.1.1 Design_Compiler_Reference_Methodology

This strategy performs a one-pass, top-down compile from RTL using compile_ultra. This flow implements the Design Compiler Reference Methodology associated with the version of Design Compiler being used.

Click the Options button to specify values for the Design_Compiler_Reference_Methodology parameters. These parameters are grouped as follows:

■ Basic

Parameters that control the QoR effort, Initial and Incremental DC Ultra compile options, and Design for Test options (see [Table 5-2](#)).

■ Netlist Generation

Parameters that control the Netlist Generation options (see [Table 5-3](#) on page 97).

■ Power

Parameters that enable Power Optimization, set power constraints, and report power information (see [Table 5-4](#) on page 98).

■ Physical Synthesis

Parameters that enable and control a Physical Synthesis run, enable Milkyway, and specify libraries and resources for that run. The Physical Synthesis parameters are the same as for the DCTCL_opto_strategy (see [Table 5-6](#) on page 101).

■ Clock Gating

Parameters that enable and control the use of clock gating during the design elaboration phase of the synthesis run (see [Table 5-7](#) on page 105). By default, clock gating is disabled.

■ Automotive

Parameters that enable and control the use of TMR and DCLS logic for automotive applications (see [Table 5-8](#) on page 107).



Note

The coreConsultant tool uses the correct Design Compiler Reference Methodology based on the version of Design Compiler used.

Table 5-2 Design_Compiler_Reference_Methodology Basic Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
QoR Effort (QorEffort)	Specifies how much relative CPU time Design Compiler uses for synthesis. (Higher QoR Effort implies longer compile runtime.) The possible values are low, medium, and high. The default is medium.
Optimization flow (OptimizationFlow)	Specifies the optimization flow, which is disabled by default. Specify one of the following: ■ hp lp: High Performance Low Power ■ hc: High Connectivity ■ rtm_exp: Runtime Exploration ■ tim: Timing Optimization

Table 5-2 Design_Compiler_Reference_Methodology Basic Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Reduced Effort Optimization Flow (ReducedEffortOptimizationFlow)	This parameter when enabled, adds -reduced_effort option to the set_qor_strategy command. The set_qor_strategy -reduced_effort option adjusts selected Optimization Flow (hplp or timing) for reduced runtime at the expense of timing QoR. Applicable when Physical Synthesis is enabled.
EnableOptimizeNetlist (EnableOptimizeNetlist)	Performs a high-effort optimization on the current design for better quality of results (QoR).
OptimizeNetlistArgs (OptimizeNetlistArgs)	Performs monotonic gate-to-gate optimization to improve area without degrading timing or leakage. This is valid only when the EnableOptimizeNetlist parameter is enabled. By default, this runs the optimize_netlist -area command, but it also accepts the following switches: ■ -no_boundary_optimization – Disables hierarchical boundary optimization ■ -no_seq_output_inversion – Disables sequential output inversion as supported by DC
Compile Ultra	
Enable Adaptive Retiming (AdaptiveRetiming)	Use adaptive retiming timing option '-retime' with the compile_ultra command. This option allows Design Compiler to move registers and latches to improve timing.
Additional Initial Options (CompileUltraInitOptions)	Additional options to pass to an initial compile_ultra command.
Additional Incremental Options (CompileUltraIncrOptions)	Additional options to pass to an incremental compile_ultra command.
SDC Generation	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
Technology Node	
Technology Node (TechnologyNode)	Specifies the technology node to be used in set_technology command in Design Compiler NXT. This helps in setting up the Design Compiler NXT for technology-node specific synthesis. The Design Compiler performs validation and coreTools does not validate the technology node value.
SDF Generation	
Write SDF after synthesis	Indicates whether an SDF file should be generated when synthesis completes. If enabled, the SDF file is written to <workspace>/syn/<stage>/db/<design>.sdf.

Table 5-2 Design_Compiler_Reference_Methodology Basic Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
(WriteSDF)	
ICC2 Write Files	
Write ICC2 files after synthesis (WriteICC2Files)	Indicates whether an ICC2 file is generated when synthesis finishes. If enabled, the ICC2 file is written to <workspace>/syn/<stage>/db/ICC2_files.
Design for Test	
Test Ready Compile (TestReadyCompile)	Enables test-ready compile. Primary effect of this parameter is to cause the -scan option to be added to the initial compile call.
Insert DFT (InsertDft)	Enables design for test circuitry to be inserted based on the settings from the Specify Test Methodology activity.
CTL Models (CTLModels)	CTL Model files are used to describe the test model required to generate test vectors for the design. This strategy parameter is read in the context of top-level synthesis (during both flat or bottom-up synthesis) whereas, the design attribute CTLModels is only used during bottom-up synthesis runs. This value set here is passed onto the DCRM variable. DCRM_DFT_CTL_FOR_ABSTRACT_BLOCKS along with the user specified CTLModels in the design attributes and the TechnologyCellBinding activity.
DFT Synthesis Optimization (DFTSynthesisOptimization)	Specify the value to perform optimizations during DFT insertion. 'all' to turn on all optimizations, and 'none' to turn off all optimizations. The attribute specifies the argument to 'set_dft_insertion_configuration -synthesis_optimization'. In the topographical mode, the only allowed value is none and it cannot be changed.

Table 5-3 Design_Compiler_Reference_Methodology Netlist Generation Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Netlist Generation	
Netlist Format (NetlistFormat)	Causes synthesis checkpoints to be written in the specified netlist format. The netlist is located in the "db" directory for the checkpoint.
change_names Name Rules (ChangeNamesRuleName)	Indicates the name of a change_names naming rule. Prior to writing the netlist in the specified format, change_names is called using the specified rules.

Table 5-3 Design_Compiler_Reference_Methodology Netlist Generation Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Remove unconnected ports (RemoveUnconnectedPorts)	Removes unconnected ports from the final netlist at the conclusion of synthesis before the design is written out. Default: 0
set_fix_multiple_port_nets	
Fix Multiple Port Nets Default (FixMultiplePortNetsDefault)	This parameter causes 'set_fix_multiple_port_nets -default' to be used in Design Compiler. No buffers will be inserted to fix multiple port nets. Default: 1. Unchecking this box enables Feedthroughs, Outputs, Constants, and Buffer Constants to be set independently.
Fix Multiple Port Nets Feedthroughs (FixMultiplePortNetsFeedthroughs)	Causes the "set_fix_multiple_port_nets -feedthrough' to be used in Design Compiler. Default: 0
Fix Multiple Port Nets Outputs (FixMultiplePortNetsOutputs)	Causes the "set_fix_multiple_port_nets -outputs' to be used in Design Compiler. Default: 0
Fix Multiple Port Nets Constants (FixMultiplePortNetsConstants)	Causes the "set_fix_multiple_port_nets -constants' to be used in Design Compiler. Default: 0
Fix Multiple Port Nets BufferConstants (FixMultiplePortNetsBufferConstants)	Causes the "set_fix_multiple_port_nets -buffer_constants' to be used in Design Compiler. Default: 0

Table 5-4 Design_Compiler_Reference_Methodology Power Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Power Optimization	
Enable Power Optimization (EnablePower)	Enables the power optimization within DC Ultra.

Table 5-4 Design_Compiler_Reference_Methodology Power Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Power Optimization Constraint (PowerOptimizationConstraint)	Enumerated type options are: ■ "set_max_leakage_power" which sets the max_leakage_power constraint to 0 ■ "set_max_dynamic_power" which sets the max_dynamic_power constraint to 0 ■ "set_max_total_power" which does both of the above.
Enable SAIF annotation (EnableSaifAnnotation)	Enable the annotation of switching activity from a SAIF file. Switching activity from SAIF is required to enable power optimization. Providing switching activity data also helps increase the accuracy of power analysis with PrimeTime-PX.
Saif File Path (SaifFilePath)	SAIF file to be used for power optimization.
Saif Instance Name (SaifInstanceName)	This is the path to the Device Under Test (DUT) in the testbench which generates the switching activity.
Report Power Options (ReportPowerOptions)	Dash options to be passed to the report_power command in DC Ultra
UPF	
Enable UPF Flow (EnableUpf)	When Enable UPF Flow is set to true, the UPF and power constraints files are used during synthesis. This feature is enabled for DC versions 2008.09 and later.
UPF Flow Type (UPFFlowType)	Use the default UPF flow (UPFPrime) or Golden UPF Flow (GoldenUPF) with golden UPF file and supplemental UPF file written after synthesis.
Incomplete UPF Allowed (IncompleteUPFAllowed)	Allow UPF synthesis flow with a partially configured UPF file.
UPF File (UpfFile)	The name of the Unified Power Format (UPF) file to be used with the Design Compiler load_upf command. This field is initialized with the value of the UpfFile attribute of packaged components, if specified by coreBuilder.
Power Constraints File (PowerConstraintsFile)	The power constraints file is sourced in Design Compiler after loading the UPF file. This file can be used to issue 'set_voltage' commands and remove 'dont_touch' elements on library cells, as needed.

Table 5-5 Design_Compiler_Reference_Methodology Design for Test Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Test Ready Compile (TestReadyCompile)	Enables test-ready compile. Primary effect of this parameter is to cause the -scan option to be added to the initial compile call.
Insert DFT (InsertDft)	Enables design-for-test circuitry to be inserted based on the settings from the Specify Test Methodology activity.
Internal_scan mode chain count. (NumberOfScanChains)	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports to be created. The default value is the design attribute NumberOfScanChains set on the implementation context.
DFT Synthesis Optimization (DFTSynthesisOptimization)	Specify the value to perform optimizations during DFT insertion. 'all' to turn on all optimizations, and 'none' to turn off all optimizations. The attribute specifies the argument to 'set_dft_insertion_configuration -synthesis_optimization'. In the topographical mode, the only allowed value is none and it cannot be changed.
CTL Models (CTLModels)	CTL Model files are used to describe the test model required to generate test vectors for the design. This strategy parameter is read in the context of top-level synthesis (during both flat or bottom-up synthesis) whereas, the design attribute CTLModels is only used during bottom-up synthesis runs. This value set here is passed onto the DCRM variable. DCRM_DFT_CTL_FOR_ABSTRACT_BLOCKS along with the user specified CTLModels in the design attributes and the TechnologyCellBinding activity.
Scan Compression Settings	
Enable scan compression (EnableScanCompression)	Enables ScanCompression_mode in DFT implementation. The default value is the design attribute EnableScanCompression set on the implementation context. For more information about scan compression, refer to DFTMax and DFTMax Ultra user guides.
ScanCompression_mode IO count. (ScanCompressionIOCount)	The number of chains for Internal_scan and ScanCompression_mode may be different. E.g. hybrid flow during bottom-up synthesis. This strategy parameter sets the number of scan IOs for compression mode. The default value is the design attribute ScanCompressionIOCount set on the implementation context.
Scan compression chain count. (NumberOfScanCompressionChains)	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally. The default value is the design attribute NumberOfScanCompressionChains set on the implementation context.

Table 5-5 Design_Compiler_Reference_Methodology Design for Test Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Scan compression usage (ScanCompressionUsage)	Sets the type of compression used. The default value is the design attribute ScanCompressionUsage set on the implementation context.
Scan compression test clock (ScanCompressionTestClock)	Sets a test clock for scan compression from the list of test clocks available.
Scan compression configuration (ScanCompressionConfiguration)	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.

**Note**

Most of the following Physical Synthesis parameter settings are also accessible from the "Physical Library setup" tab of the Specify Target Technology activity.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Physical Synthesis (PhysicalSynthesis)	Use DC-Topographical (2007.03 or later--this enables "topographical mode" in DC). Setting this parameter to true causes the physical synthesis placement engine to be used during optimization. Net loads and delays will be estimated by placement, and wire load models will not be used. Note that no placement data is saved with the ddc or Milkyway database.
Enable SPG (EnableSPG)	Enables the Synopsys Physical Guidance flow in DC-Topographical.
Floorplan Data Format (FloorplanFileDataFormat)	Adds support for the read_floorplan command. Legal values for FloorplanFileDataFormat are 'floorplan', 'def', and 'tcl'. - floorplan: (Default) floorplan data written by the 'write_floorplan' command. - def: file formats covered by the old 'DefPhysicalConstraintFile' parameters. - tcl: file formats covered by the old 'TclPhysicalConstraintFile' parameters.
Physical Constraint File (PhysicalConstraintFile)	Adds support for the read_floorplan command. String provides the path to the Physical Constraint File. The FloorplanFileDataFormat parameter sets the format for this file.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Enable Port Side Constraint (EnablePortSideConstraint)	Generates a port side file which is inserted into the synthesis flow for better pin placement. This file can only be generated when there is no floorplan file given and the SideConstraint attribute is set on the design port by the user. This can be used only in topographical mode. This feature is enabled only when the DC version is >= 2018.06.
Utilization Ratio (UtilizationRatio)	Specifies the utilization ratio as part of the synthesis intent. The default value is 0.7 and the values range from 0.0 to 1.0. The utilization ratio is applicable only when the floorplan data is not provided.
Min Routing Layer Name (MinRoutingLayer)	Specifies the minimum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers below the specified layer as fully blocked for RC and congestion estimation.
Max Routing Layer Name (MaxRoutingLayer)	Specifies the maximum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers above the specified layer as fully blocked for RC and congestion estimation.
Congestion Optimization Factor (CongestionOptimizationFactor)	Enables user to specify the maximum utilization factor for congestion optimization. This specifies how the Design Compiler can densely pack the cells in the uncongested regions. The default value is set to 0.9 in Design_Compiler_Reference_Methodology. This feature can only be configured when Physical Synthesis is enabled along with "OptimizationFlow" set to High Performance Low Power (hplp) or Timing Optimization (Tim).
Use Milkyway (UseMilkyway)	Use the Milkyway database format. Note that the Milkyway design library must have been created prior to specifying the Milkyway parameters. See Milkyway documentation for more information on the Milkyway format.
Milkyway Design Library (MilkywayDesignLibrary)	This parameter corresponds to the Design Compiler variable mw_design_library Specify the full path to the Milkyway design library. The Milkyway design library must exist before specifying this parameter. coreTools will not create it.
Create Milkyway Design (CreateMilkywayDesign)	If TRUE, coreConsultant will create the Milkyway design library. Default: 1 (TRUE)
Tech File Format (TechFileFormat)	Choose "tf" to create the tech file to create the Milkyway design library. Choose "plib" to use the physical library source file (.plib). Default: tf
Tech File (TechFile)	Technology file used to create the Milkyway design library. The file must match the format specified in "Tech File Format."

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Milkyway Reference Library (MilkywayReferenceLibrary)	This parameter corresponds to Design Compiler variable <code>mw_reference_library</code> . Specify the full path to the Milkyway reference library. The Milkyway reference library must exist before specifying this parameter. <code>coreTools</code> will not create it.
Max Tlu Plus File (MaxTluPlusFile)	The Max TLU+ file.
Min Tlu Plus File (MinTluPlusFile)	The Min TLU+ file.
Tech file to ITF Mapping File (Tech2ItfMap)	TECH2ITF mapping file name.
Extend MW Layers (ExtendMWLayers)	Inserts the <code>extend_mw_layers</code> command into DC topo scripts. Enable this parameter when the synthesis library requires more than the standard 255 layers in DC-Topographical synthesis. Default: 0 (FALSE)
Max Emulation TLU+ file name (MaxEmulationTluPlus)	Used for the <code>-max_emulation_tluplus</code> switch to <code>set_tlu_plus_files</code> command. See the Design Compiler documentation for information about the <code>set_tlu_plus_files</code> command.
Min Emulation TLU+ file name (MinEmulationTluPlus)	Used for the <code>-min_emulation_tluplus</code> switch to <code>set_tlu_plus_files</code> command. See the Design Compiler documentation for information about the <code>set_tlu_plus_files</code> command.
Create Mw Design Options (CreateMwDesignOptions)	Additional options for the <code>create_mw_design</code> command. See the Design Compiler or Milkyway documentation for options.
Milkyway Logic 0 Net (MilkywayLogic0Net)	This parameter corresponds to the Design Compiler variable <code>mw_logic0_net</code> . Contains the Milkyway logic0(1'b0) net for the design. Default: VSS
Milkyway Logic 1 Net (MilkywayLogic1Net)	This parameter corresponds to the Design Compiler variable <code>mw_logic1_net</code> . Contains the Milkyway logic1(1'b1) net for the design. Default: VDD
Save design in Milkyway Cell format (WriteMilkywayCell)	Milkyway Cell representation of the design will be written out into the directory specified by the <code>MilkywayDesignLibrary</code> parameter at the end of synthesis. Default: False (no save occurs)

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Create Congestion Map (CreateCongestionMap)	Create graphical congestion map when synthesis completes. To use this feature, you must synthesize with DC-Topographical 2010.03 or newer. This feature also requires DC-Graphical with a DC-Extension license. Your DISPLAY variable must be set to use this feature. Default: False
NDM Library	
Use NDM (UseNDM)	This switch enables NDM view to extract the physical information. NDM view is only supported in the DCRM version S-2021.06-SP4 or greater. Milkyway view is supported in the older versions of DCRM templates.
NDM Library Name (NDMLibraryName)	Specifies the name of the new NDM library. The path to the NDM library can be a relative, or an absolute path. If a relative path is given, the directory will be created in the workspace 'syn' directory. The name must not contain spaces or the colon (:) character, which is used to delimit the library and design name.
NDM Reference Library (NDMReferenceLibrary)	Specifies the reference libraries for the NDM library 'NDMLibraryName'. You can specify the reference libraries and physical source data using absolute or relative paths. If you specify the libraries with a relative path or with no path, the tool uses the TechSetup's activity parameter 'SearchPath' to locate the libraries. The NDM reference library must exist before specifying this parameter. coreTools will not create it.
Create Lib Design Options (CreateLibDesignOptions)	Additional options to the create_lib command which creates NDM library. Please see Design Compiler NXT or NDM library documentation for details.
Multi Vth Constraint	
Available Threshold Voltage Groups (AvailbleVthGroups)	The voltage groups that are available to the user.
Low Threshold Voltage Groups (LowThresholdVoltageGroups)	Specifies the threshold voltage groups considered to be low-threshold voltage groups. The groups argument is a list of threshold voltage group identifiers, which are names defined by attributes in the Liberty (.lib) descriptions of the cells and cell librarian.
Low Threshold Voltage Percentage (LowThresholdVoltagePercentage)	Specifies the maximum allowable percentage of low-threshold voltage cells for the current design. The percent_value argument is a floating-point number between 0 and 100.
Low Threshold Voltage Constraint Type	Specifies the constraint type, which can be either hard or soft.

Table 5-6 Design_Compiler_Reference_Methodology Physical Synthesis Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
(LowThresholdVoltageConstraintType)	The tool meets this constraint only if it does not adversely affect the timing constraint. Default: soft

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Insert Clock Gating (ClockGating)	Specifies whether to gate clocks during elaboration. The default value is false (do not gate clocks). If you select Gate Clocks During Elaboration, the clock gating options described below are enabled. If you select Gate Clocks During Elaboration, clock gating is implemented only on designs on which the ClockGatingAllowed attribute is true.
Insert Self Gating (SelfGating)	Self-gating is a clock gating technique that optimizes the dynamic power by gating the clock signal in those cycles in which the data saved in a register remains unchanged. An enable condition is computed by comparing the stored data with the new data arriving at the data pin, and that signal is used to drive the inserted self-gating cell. Enabling this adds the -self_gating option to the compile_ultra command. This option is available only when Physical Synthesis and Clock Gating are enabled. This parameter updates the control variable INSERT_SELF_GATING in ct_variable_setup.tcl present in the rm_dc_setup folder.
Gating Minimum Bit Width (GatingMinimumBitWidth)	Specifies the minimum bit-width of a register bank for which to gate the clock. (Corresponds to the -minimum_bitwidth option for Design Compiler's set_clock_gating_style command.) The default value is 3.
Latch Style Clock Gating or Combinational Clock Gating (GatingLatchCellName)	Provides a choice between latch-style and combinational gating. If you choose latch, you can enter the name of a library cell for latching the gated clock enable signal in the Gating Latch Cell Name text box. If you choose latch but do not enter a cell name, Design Compiler chooses a latch cell from the target library. (Corresponds to the -sequential_cell option for Design Compiler's set_clock_gating_style command.) If you select Combinational, GatingLatchCellName is set to none and generates a -sequential_cell none option.
Gating And Function (GatingAndFunction)	Specifies the circuitry to use to implement AND/NAND gating. The legal enum values are: integrated, and, buf and, and buf, buf and buf, nand inv, buf nand inv, inv or inv, inv nor, inv nor buf, and inv, buf and inv, nand, buf nand, nand buf, buf nand buf, inv or, inv or buf, and inv nor inv. The default is and.

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	Your selected AND function and your choice of Latch Style or Combinational Clock Gating are used to select the AND function value for the -positive_edge_logic and -negative_edge_logic options to Design Compiler's set_clock_gating_style command. For latch-style gating, Design Compiler inserts the selected AND circuitry into the clock network for positive edge-triggered registers. For combinational-style gating, Design Compiler inserts the selected AND circuitry into the clock network for negative edge-triggered registers. If the appropriate ICG cells are available in the library, this value defaults to 'integrated'.
Integrated And Cell (IntegratedAndCell)	When the Gating And Function is set to the value "integrated," the IntegratedAndCell parameter is enabled. This parameter can be set to the name of the cell to use from the technology library.
Gating Or Function (GatingOrFunction)	Specifies the circuitry to use to implement OR/NOR gating. The legal enum values are: integrated, or, buf or, or buf, buf or buf, nor inv, buf or inv, inv and inv, inv nand, inv and buf, or inv, buf or inv, nor, buf nor, nor buf, buf nor buf, inv and, inv and buf, and inv nand inv. The default is or. Your selected OR function and your choice of Latch Style or Combinational Clock Gating are used to select the OR function value for the -positive_edge_logic and -negative_edge_logic options to Design Compiler's set_clock_gating_style command. For latch-style gating, Design Compiler inserts the selected OR circuitry into the clock network for negative edge-triggered registers. For combinational-style gating, Design Compiler inserts the selected OR circuitry into the clock network for positive edge-triggered registers. If the appropriate ICG cells are available in the library, this value defaults to 'integrated'.
Integrated Or Cell (IntegratedOrCell)	When the Gating Or Function is set to the value "integrated," the IntegratedOrCell parameter is enabled. This parameter can be set to the name of the cell to use from the technology library.
Gating Setup Time (GatingSetupTime)	Specifies the setup time constraint at the enable input of the 2-input clock gate in the clock gating circuitry. (Corresponds to the -setup option for Design Compiler's set_clock_gating_style command.) The default is 0.
Gating Hold Time (GatingHoldTime)	Specifies the hold time constraint at the enable input of the 2-input clock gate in the clock gating circuitry. (Corresponds to the -hold option for Design Compiler's set_clock_gating_style command.) The default is 0.
Gating Control Point (GatingControlPoint)	Specifies if and where to insert an OR gate to control the gating signal for testability. (Corresponds to the -control_point option for Design Compiler's set_clock_gating_style command.) The legal values are none, before, and after. The default value is none.

Table 5-7 Design_Compiler_Reference_Methodology Clock Gating Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Gating Control Signal (GatingControlSignal)	Specifies the control signal for the gating control point during testing (ignored when Gating Control Point is set to none). The choices are scan_enable and test_mode. (Corresponds to the -control_signal option for Design Compiler's set_clock_gating_style command.)
Implement Observation Logic (GatingObservationLogicDepth)	Specifies whether to implement the circuitry necessary to observe gating behavior within the design. The default is unchecked (no value for GatingObservationLogicDepth; no observation logic implemented). (Corresponds to the -observation_point option for Design Compiler's set_clock_gating_style command.)
Gating Observation Logic Depth (GatingObservationLogicDepth)	Specifies the maximum depth of the XOR observability tree. (Corresponds to the -observation_logic_depth option for Design Compiler's set_clock_gating_style command.) Gating Observation Logic Depth is enabled only when Implement Observation Logic is checked.
Specify Clock Gating Cell Max Fanout (SpecifyClockGatingMaxFanout)	Specifies the maximum fanout of a single clock-gating cell. If no value is specified, the Design Compiler default is used.
Gating cell Max Fanout Value: (ClockGatingMaxFanoutValue)	Specifies the maximum fanout of a single clock-gating cell. If no value is specified, the Design Compiler default is used.

Table 5-8 Design_Compiler_Reference_Methodology Automotive Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
AutomotiveSynthesis Flow Type (AutomotiveSynthesisFlowType)	Specifies an automotive synthesis flows, such as triple module redundancy (TMR), dual core lockstep (DCLS), or TMR_and_DCLS. The list of valid values depends on which intents are packaged with the IP. To enable this parameter when using DC, you must set up your synthesis run to be “physical aware” as follows: ■ Enable the PhysicalSynthesis synthesis option (described in Table 5-6 on page 101) ■ Enable the UseDCNXT synthesis option (described in Table 5-22 on page 122)
TMR Separation Distance (TMRSeparationDistance)	Specifies the distance values for repelling bound constraint for redundancy group. This entry can be either an {x y} pair (for example, 1 1) or a single number denoting radial distance.

Table 5-8 Design_Compiler_Reference_Methodology Automotive Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
DCLS Distance (DCLSDistance)	Specify the DCLS distance as a pair of {width height}. This creates a group bound with the specified dimensions.

Table 5-9 Design_Compiler_Reference_Methodology Reports Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Generate supplementary Reports (GenerateRMReports)	Enables report generation commands specified in the Reference Methodology. Default: 0
Skip Congestion Report (DisableCongestionReport)	Disables the congestion report generation. It can be used for runtime improvements. Default: 1

5.3.1.2 Fusion_Compiler_Reference_Methodology

The Fusion_Compiler_Reference_Methodology strategy performs a one-pass, top-down compile from RTL using compile_fusion. This strategy is available in coreConsultant under the following conditions:

- Your core IP supports Fusion Compiler.
- You have specified an NDM library as explained in “[Logical Library Setup](#)” on page 60.
- You have specified a supported version of Fusion Compiler using the **Edit > Tool Installation Roots** menu, as explained in “[Set Tool Installation Root](#)” on page 61.

When you have selected the Fusion_Compiler_Reference_Methodology strategy, click the Options button to specify values for the Fusion_Compiler_Reference_Methodology parameters. These parameters are grouped as follows:

- Basic (see [Table 5-10](#) on page 109)
Parameters that control the flow, technology, Design for Test, and UPF options.
- Constraints (see [Table 5-12](#) on page 114)
Parameters that control the flow, technology, Design for Test, and UPF options.
- Automotive (see [Table 5-19](#) on page 119)
Parameters that control the use of TMR and DCLS logic for automotive applications.

Table 5-10 Fusion_Compiler_Reference_Methodology Basic Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Flow	
Enable QIK for DesignWare Interface IP (EnableQIK)	Enables the Quick Implementation Kit (QIK) for DesignWare Interface IP. This flow is a version of the Fusion Compiler Reference Methodology which has been tuned for Synopsys DesignWare Interface IP. Requires Fusion Compiler 2022.12 or later to be enabled. Default: 0
Final FCRM Flow Step (FinalFCRMFlowStep)	Indicates how far the FCRM flow should run. The specified step will be the last one run in the FCRM flow. The legal values are: ■ {place_opt (classic flow to final_opto)} ■ {compile (unified flow to final_opto)} ■ {route_opt (unified flow to final_opto + clock_opt and route_opt)} Default: Compile
QoR	
QoR Strategy (QorStrategy)	The selected strategy is passed into the set_qor_strategy -metric <strategy>. Appropriate application options are set to optimize the specified target strategy. The legal values are: ■ Timing (timing) ■ Leakage Poser (leakage_power) ■ Total Power (total_power) Default: Timing
Enable Reduced Effort (EnableReducedEffort)	Sets the -reduced_effort option for the set_qor_strategy command. Default: 0 (false)
FloorPlan	
Floorplan Data Format (FloorplanFileDataFormat)	Format of file containing floorplan constraints. The legal values are: ■ def: DEF file formats ■ tcl: TCL file formats
Enable Design Planning Flat Flow (EnableDesignPlanFlatFlow)	Enables the flat design planning flow. The goal of the flat design planning flow is the generation of a complete design floorplan. The primary inputs to the flow are RTL files, UPF constraints, technology node setup, and parasitic data. The flow will export a completed design floorplan. The exported floorplan is a primary input into the PnR flow. The default exported floorplan will be tcl based floorplan file. This feature is available with Fusion Compiler version >= 2022.03-SP1.

Table 5-10 Fusion_Compiler_Reference_Methodology Basic Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Physical Constraint File (TCLPhysicalConstraintFile)	Specifies the path to the Physical Constraint File ■ For the DEF format, populates the 'DEF_FLOORPLAN_FILES' setup parameter. ■ For the TCL format, populates the 'TCL_FLOORPLAN_FILE' setup parameter.
Enable Port Side Constraint (EnablePortSideConstraint)	Generates a port side file which is inserted into the synthesis flow for better pin placement. This file can only be generated when there is no floorplan file given and the SideConstraint attribute is set on the design port by the user.
Utilization Ratio (UtilizationRatio)	Specifies the utilization ratio as part of the synthesis intent. The default value is 0.7 and the values range from 0.0 to 1.0. The utilization ratio is applicable only when the floorplan data is not provided.
SDC Generation	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
TechFile	
Use Technology File (UseTechFile)	Use technology file to specify technology information. When enabled, the target library (TECH_LIB) setup is not used. Default: 0 (false)
TechFile (TechFile)	Technology file used to create the Milkyway design library (read-only). You can set the file path in the Specify Target Technology activity (TechSetup), as described in “Physical Library Setup” on page 61.
Enable Library Configuration Flow (EnableLibraryConfigurationFlow)	Use the library configuration flow, which calls the library manager under the hood to generate .nlibs. Default: 0 (false)
Technology Node	
Technology Node (TechnologyNode)	Specifies the technology node to be used in set_technology command in Fusion Compiler. This helps in setting up the Fusion Compiler for technology-node specific synthesis. Fusion Compiler performs validation and coreTools does not validate the technology node value.
Technology Reference Methodology Scripts (TechnologyRMScripts)	This parameter captures the path to the rm_tech_scripts delivered by Synopsys. When using QIK, it is recommended for the user to specify the path to the rm_tech_scripts.

Table 5-10 Fusion_Compiler_Reference_Methodology Basic Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Set technology after floorplan (SetTechnologyAfterFloorplan)	Setting of technology for nodes to be performed after floorplanning. The feature is available if Fusion Compiler version is 2022.03-SP1 or greater when TechnologyNode has been specified and either PhysicalConstraintFile is specified or Design Plan Flat Flow (EnableDesignPlanFlatFlow) is enabled. Default: 0 (false)
Technology Side File	
Technology Sidefile Settings (TechnologySideFileSettings)	Sets the Reference Methodology control variables. The parameter accepts a list of control variables and values. The information is exported in the 'syn/rm_fc_snps_ip_scripts/snps_ip_technology_sidefile_setup.tcl' file. The parameter is disabled when the QIK flow is enabled (EnableQIK is set to true). Default: Not Applicable
TLUPlus	
Early parasitic tech specification name (MinTluPlusName)	Specifies the early parasitic technology specification name.
Min TLUPlus file name (MinTluPlusFile)	TLU+ files to read for Min specification. If TechSetup's MinTluPlusFile activity parameter has been specified, the same file path is used for this parameter.
Min TLUPlus Layer map file (MinTluPlusLayerMapFile)	Layer mapping file between the ITF and Min technology specification.
Late parasitic tech specification name (MaxTluPlusName)	Specifies the late parasitic technology specification name.
Max TLUPlus file name (MaxTluPlusFile)	TLU+ files to read for Max specification. If TechSetup's MaxTluPlusFile activity parameter has been specified, the same file path is used for this parameter.
Max TLUPlus Layer map file (MaxTluPlusLayerMapFile)	Layer mapping file between ITF and Max technology specification.

Table 5-11 Fusion_Compiler_Reference_Methodology Design for Test Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Insert DFT (InsertDft)	Enables design-for-test circuitry to be inserted based on the settings from the Specify Test Methodology activity. Default: False
DFT Flow Type (DFTFlowType)	Sets the DFT_Flow control variable to in_compile or pre_compile. Default: pre_compile
Internal_scan mode chain count. (NumberOfScanChains)	The number of scan chains to be created. This value is passed to the set_scan_configuration command with the -chain_count option. For Internal_scan, this is also the number of scan IO ports to be created. The default value is the design attribute NumberOfScanChains set on the implementation context.
CTL Models (CTLModels)	CTL Model files are used to describe the test model required to generate the test vectors for the design. This strategy parameter is read in the context of top-level synthesis (during both flat or bottom-up synthesis) whereas, the design attribute CTLModels is only used during the bottom-up synthesis runs. This value set here is passed onto the FCRM variable CTL_FOR_ABSTRACT_BLOCKS along with the user specified CTLModels in the design attributes and the TechnologyCellBinding activity. Default: Null
Scan Compression Settings	
Enable scan compression (EnableScanCompression)	Enables ScanCompression_mode in DFT implementation. The default value is the design attribute EnableScanCompression set on the implementation context. For more information about scan compression, refer to DFTMax and DFTMax Ultra user guides.
Enable Distributed Codec (EnableDistributedCodec)	Enables Distributed Codec insertion in design. Can only be enabled when both EnableScanCompression and EnableTestMaxManagerFlow strategy parameters are enabled. For more information on scan compression, see the DFTMax and DFTMax Ultra User Guides. Default: 0
ScanCompression_mode IO count. (ScanCompressionIOCount)	The number of chains for Internal_scan and ScanCompression_mode may be different. E.g. hybrid flow during bottom-up synthesis. This strategy parameter sets the number of scan IOs for compression mode. The default value is the design attribute ScanCompressionIOCount set on the implementation context.
Scan compression chain count. (NumberOfScanCompressionChains)	In ScanCompression_mode, this parameter sets the number of scan chains to be created internally.

Table 5-11 Fusion_Compiler_Reference_Methodology Design for Test Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	The default value is the design attribute NumberOfScanCompressionChains set on the implementation context.
Scan compression usage (ScanCompressionUsage)	The value passed along with -usage option for ScanCompression_mode test mode in the define_test_mode command. See DFTMAX, DFTMAX Ultra and DFT SEQ Compression sections in the TestMAX DFT User Guide for more details. The default value for this strategy parameter is the value of the design attribute 'ScanCompressionUsage' set on the implementation context.
SEQ Compression Inputs (SeqCompressionInputs)	Total number of SEQ Compression input ports. Default: 7
SEQ Compression Outputs (SeqCompressionOutputs)	Total number of SEQ Compression output ports. Default: 1
Scan compression test clock (ScanCompressionTestClock)	Sets a test clock for scan compression from the list of test clocks available.
Scan compression configuration (ScanCompressionConfiguration)	The value of this parameter is passed directly to the 'set_scan_compression_configuration' command. For details on DFTMAX compression options, refer to the DFTMAX User Guide, Chapter 2, 'Using DFTMAX Compression' and Chapter 4, 'Managing X Values in Scan Compression'.
TestMAX Manager Settings	
Enable TestMAX Manager Flow (EnableTestMaxManagerFlow)	Enables TestMax Manager to introduce DFT infrastructure within the Designware RTL prior to synthesis. Such RTL with DFT can then be processed by the Fusion Compiler. To enable this feature the following conditions are to be met: ■ InsertDft set to 1 ■ The leaf level components must be TestMax Manager ready ■ Fusion Compiler version must be 2022.03 or greater ■ ScanCompressionUsage is not set to scan_compression ■ Test Max Manager version must be 2022.12-SP3 or greater ■ Spyglass version must be 2023.03 or greater ■ Not supported for encrypted license.
Enable Test point insertion (EnableTPI)	Enables the TestMax Manager to introduce the test points in the design RTL prior to synthesis. Such RTL with DFT can then be processed by the Fusion Compiler. The feature can only be enabled when ScanCompressionUsage is not set to scan_compression.

Table 5-12 Fusion_Compiler_Reference_Methodology Constraints Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Routing Layer	
Min Routing Layer Name (MinRoutingLayer)	Specifies the minimum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers below the specified layer as fully blocked for RC and congestion estimation.
Max Routing Layer Name (MaxRoutingLayer)	Specifies the maximum routing layer for the design. The router respects this setting. To ensure that RC estimation and congestion analysis stay synchronized with the router, when you set this option, the tool marks all routing layers above the specified layer as fully blocked for RC and congestion estimation.
Multi Vth Constraint	
Available Vth Groups (AvailableVthGroups)	The Voltage Groups available to the user
Low Threshold Voltage Groups (LowThresholdVoltageGroups)	A list of threshold voltage group names which are considered as low-threshold voltage groups, stored as threshold_voltage_group attributes present in the liberty or NDM description of the cell.
Low Threshold Voltage Percentage (LowThresholdVoltagePercentage)	Specifies the allowable low Vth cell percentage limit on the current design, based on cost type. This is a required setting. Default: 100
Low Threshold Voltage Cost (LowThresholdVoltageCost)	Specifies how low Vth percentage limit is calculated. The low Vth percentage can be based on cell count or cell area. Default: cell_count

Table 5-13 Fusion_Compiler_Reference_Methodology Power Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Power Optimization	
Saif File Path (SaifFile)	List of SAIF files for power aware synthesis.
Saif Instance Name (SaifInstanceName)	Name of the instance of the current design as it appears in SAIF file. This argument is passed on to the read_saif command with the -strip_path option.

Table 5-13 Fusion_Compiler_Reference_Methodology Power Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
UPF	
Enable UPF Flow (EnableUPFFlow)	When set to true, the UPF and power constraints files are used during synthesis. Default: False
UPF File (UPFFile)	Specifies the name of the Unified Power Format (UPF) file to be used with the Fusion Compiler.
Power Constraints File (PowerConstraintsFile)	Specifies the power constraints file. The power constraints file is sourced in Fusion Compiler after loading the UPF file. This file can be used to issue 'set_voltage' commands and remove do_touch elements on library cells, as needed.
Multi Voltage Setup File (MVSetupFile)	Tcl script placeholder for your multi-voltage setup file, which contains details such as create_voltage_area, placement bound, power switch creation, level shifter insertion, and so on.

Table 5-14 Fusion_Compiler_Reference_Methodology Parasitic Definition Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Parasitic n (where, n=1/2/3/4)	
Parasitic Model Name for Model n (ParasiticModelNamen)	Specifies the unique name which identifies this parasitic model. Default: Not Applicable
TLUPlus File for Model n (TLUPlusFileForParasitcn)	Specifies the names of the TLUPlus file or common nxtgrd file to read for this parasitic model. Default: Not Applicable
Layer Map File for Model n (LayerMapFilen)	Specifies the names of the layer map file between the technology file and the ITF file for this parasitic model. Default: Not Applicable

Table 5-15 Fusion_Compiler_Reference_Methodology Mode Definition Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Mode n (where, n=1/2/3/4)	
Name for Mode n (ModeNamen)	Defines a mode to be created using the create_mode command. Each mode must have a unique name. Default: Not Applicable
Mode Constraint File for Mode n (ModeConstraintFilen)	Constraint file used to set up mode n. Default: Not Applicable

Table 5-16 Fusion_Compiler_Reference_Methodology Corner Definition Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Index of the Primary Corner (PrimaryCorner)	Sets application option cts.compile.primary_corner to the selected corner. Default: 1
Corner n (where, n=1/2/3/4/5/6/7/8/9)	
Name for Corner n	Defines a corner to be created using the create_corner command. A corner being used for set-up time analysis must have SETUP in the name. Similarly, a corner used for hold time analysis must have HOLD in the name. Default: Not Applicable
Corner Constraint File for Corner n	Specifies the corner constraint file associated with corner n. Default: Not Applicable

Table 5-17 Fusion_Compiler_Reference_Methodology Scenario Definition Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Scenario n (where n=0/1/2/3/4/5/6/7/8/9)	
Name for Scenario n (ScenarioNamen)	Defines a scenario to be created with the create_scenario command. The entries must be named <MODE NAME>::<CORNER NAME>, where <MODE NAME> is a defined mode and <CORNER NAME> is a defined corner. Default: Not Applicable

Table 5-17 Fusion_Compiler_Reference_Methodology Scenario Definition Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Active Scenario List	
Active Scenario List for compile (ActiveScenarioListcompile)	Defines the set of active scenarios for the 'compile' stage. Values must match entries in the scenario definition parameters. If not specified, all scenarios are considered active. Default: Not Applicable
Active Scenario List for clock_opt_cts (ActiveScenarioListclock_opt_cts)	Defines the set of active scenarios for the 'clock_opt_cts' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable
Active Scenario List for clock_opt_opto (ActiveScenarioListclock_opt_opto)	Defines the set of active scenarios for the 'clock_opt_opto' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable
Active Scenario List for route_auto (ActiveScenarioListroute_auto)	Defines the set of active scenarios for the 'route_auto' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable
Active Scenario List for route_opt (ActiveScenarioListroute_opt)	Defines the set of active scenarios for the 'route_opt' stage. Values must match entries in the scenario definition parameters. If not specified, active scenarios from the prior stage will be used. If not specified for any stage, all scenarios are considered active. Default: Not Applicable

Table 5-18 Fusion_Compiler_Reference_Methodology Cell Pruning Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Don't Use Library Cells (DontUseLibCells)	Defines library cells that should not be used throughout the flow. Typically consists of very small and very large drive strength cells, and non-typical use cells. This parameter updates the FCRM control variable DONT_USE_LIB_CELL_LIST. Default: No Default Value

Table 5-18 Fusion_Compiler_Reference_Methodology Cell Pruning Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Valid Optimization Buffers (ValidOptBuffers)	Defines buffer cells to be used during all optimization stages except for 'cts'. Typically includes low and medium strength buffers across all voltage threshold classes. Buffer cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_OPT_BUFFER_LIST. Default: No Default Value
Valid Optimization Inverters (ValidOptInverters)	Defines inverter cells to be used during all optimization stages except for 'cts'. Typically includes low and medium strength inverters across all voltage threshold classes. Inverter cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_OPT_INVERTER_LIST. Default: No Default Value
Valid Hold Buffers (ValidHoldBuffers)	Defines buffer cells to be use for fixing hold time violations. Typically includes delay buffers and low drive strength buffers, usually of the slowest voltage threshold class available. This parameter updates the FCRM control variable VALID_HOLD_BUFFER_LIST. Default: No Default Value
Valid ICG Cells (ValidICGCells)	Defines the integrated clock gating (ICG) cells to be used. Typically includes ICG cells of a variety of drive strengths, usually of the fastest voltage threshold class available. ICG cells not listed here will be blocked from use. This parameter updates the FCRM control variable VALID_ICG_CELL_LIST. Default: No Default Value
Valid CTS Repeaters (ValidCTSRepeaters)	Defines valid buffer/inverter cells to be used as CTS repeater cells. Typically includes buffers and inverters of medium to high drive strength, usually of the fastest voltage threshold class available. Library cells not listed here will be blocked from use for the 'cts' stage. This parameter updates the FCRM control variable VALID_CTS_REPEATER_LIST. Default: No Default Value
Valid Skew Cells (ValidSkewCells)	Defines valid skew cells that should not be used for optimization. Cells in this list will be blocked from use for all optimization stages. This parameter updates the FCRM control variable SKEW_CELL_LIST. Default: No Default Value
Valid Tie Cells (ValidTieCells)	Defines the valid tie high and low cells to be used during optimization. Typically includes tie high/low cells of the lowest voltage threshold class available. Library cells in this list have any existing dont_touch designation removed and are defined for use in all optimization stages. This parameter updates the FCRM control variable VALID_TIE_CELL_LIST. Default: No Default Value
Clock Isolation Buffer (ClockISOBuffer)	Defines a single clock tree repeater cell of medium drive strength used to isolate clock ports. This parameter updates the FCRM control variable CLOCK_ISO_BUFFER.

Table 5-18 Fusion_Compiler_Reference_Methodology Cell Pruning Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	Default: No Default Value

Table 5-19 Fusion_Compiler_Reference_Methodology Automotive Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
AutomotiveSynthesis Flow Type (AutomotiveSynthesisFlowType)	Specifies an automotive synthesis flow, such as triple module redundancy (TMR), dual core lockstep (DCLS), or TMR_and_DCLS. The list of valid values depends on which intents are packaged with the IP.
TMR Separation Distance (TMRSeparationDistance)	Specifies the distance values for repelling bound constraint for redundancy group. This entry can be either an {x y} pair (for example, 1 1) or a single number denoting radial distance.
DCLS Distance (DCLSDistance)	Specify the DCLS distance as a pair of {width height}. This creates a group bound with the specified dimensions.

Table 5-20 Fusion_Compiler_Reference_Methodology Reports Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Generate supplementary reports (GenerateRMReports)	Enables report generation commands specified in the Reference Methodology. Default: 0
Generate Stage based QOR Reports (ReportQOR)	FCRM templates generate QOR data at end of each stage, enabled by this parameter. Default: 0
Generate QOR Histogram (ReportQORHistogram)	Report a histogram for QOR data using proc_histogram call in write_data_stage Default: 0
Skip Congestion Report (DisableCongestionReport)	Disables congestion report generation. It can be used for runtime improvements. Default: 1

5.3.1.3 DCTCL_quick_map_strategy

This strategy performs a quick, low-effort compile to generate a gate-level netlist of the core for analysis purposes only. Synthesis may optionally be carried out using DC-Explorer, provided that version G-2012.06-SP2 or newer of DC-Explorer is available.

Use DCTCL_quick_map_strategy to perform a quick feasibility check on a design. Do not use DCTCL_quick_map_strategy simply to reduce compile time on a design intended for manufacture because it does not perform all synthesis steps rigorously enough for production purposes.

Click the Options button to specify values for the DCTCL_quick_map_strategy parameters.

■ Basic

Parameters that control the basic level of effort for the estimation of the gate-level netlist (see [Table 5-21](#)).

■ Netlist Generation

Parameters that control the netlist format and rules, and the fixing of multi-port nets. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-3](#) on page 97).

■ PhysicalSynthesis

Parameters that enable and control a Physical Synthesis run, enable Milkyway and specify libraries and resources for that run. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-6](#) on page 101). The PhysicalSynthesis parameters are only enabled if “Use DC-Explorer” is set to true.

■ Clock Gating

Parameters that enable and control the use of clock gating during the design elaboration phase of the synthesis run. These parameters are the same as for the Design_Compiler_Reference_Methodology strategy (see [Table 5-7](#) on page 105).

Table 5-21 DCTCL_quick_map_strategy Basic Parameters

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
Use DC-Explorer (UseDcExplorer)	This defaults to false and is enabled only if DC-Explorer G-2012.06-SP2 or newer is available.
Stop at GTECH (StopAtGtech)	Analyzes and elaborates your configured core to the Synopsys technology independent GTECH library, without mapping the design to the target technology. Generating a GTECH version of the core is an easy way to create a simulation model for a core delivered as encrypted HDL. The default value is false (do not stop at GTECH). Disabled if the UseDcExplorer parameter is true.
Read Constraints (ReadConstraints)	When enabled, the SDC constraints are read in during the GTECH synthesis run. This option is enabled only when the “Stop at GTECH” option is checked. The default value is false (do not use only simple GTECH gates).

Table 5-21 DCTCL_quick_map_strategy Basic Parameters (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	Disabled if UseDcExplorer parameter is true.
Use only simple GTECH gates (OnlySimpleGtechGates)	Specifies whether to use only a minimal set of GTECH gates to implement the netlist. The cells used are: GTECH_NOR2, GTECH_NOT, GETCH_FD2, and GTECH_FD4. (This option is enabled only when the “Stop at GTECH” option is checked). The default value is false (do not use only simple GTECH gates). Disabled if the UseDcExplorer parameter is true.
Exclude cell set: (ExcludeGtechCellSet)	Set of GTECH cells that should not be used when mapping the design. Disabled if the UseDcExplorer parameter is true.
Include cell set: (IncludeGtechCellSet)	Set of GETCH cells to be used in addition to the cells used when OnlySimpleGtechGates is specified. Disabled if the UseDcExplorer parameter is true.
Ungroup all after compile (UngroupAllAfterCompile)	Indicates that the design should be totally ungrouped after the final compile stage. Checking this option will result in a completely flat netlist, regardless of any PreserveHierarchy settings. Default: 0
Compile Ultra	
Additional Initial Options (CompileUltraInitOptions)	Additional options to pass to an initial compile_exploration command. Only enabled if “Use DC-Explorer” is true.
SDF Generation	
Write SDF after synthesis (WriteSDF)	Indicates whether an SDF file should be generated when synthesis completes. If enabled, the SDF file is written to <workspace>/syn/<stage>/db/<design>.sdf.
ICC2 Write Files	
Write ICC2 files after synthesis (WriteICC2Files)	Indicates whether an ICC2 file is generated when synthesis completes. If enabled, the ICC2 file is written to <workspace>/syn/<stage>/db/ICC2_files/*.
Design for Test	
Test Ready Compile (TestReadyCompile)	Enables test-ready compile. Primary effect of this parameter is to cause the -scan option to be added to the initial compile call. Only enabled if “Use DC-Explorer” is true.

5.3.2 Specifying Synthesis Execution Options

Select your synthesis options on the Options tab of the Synthesize Activity View. Synthesis options differ from the Synthesis Strategy options in that, they do not determine how coreConsultant generates commands to the synthesis tools, but rather how the coreConsultant executes the strategy. For example, you can select a remote host on which to run the synthesis job.

For DCTCL_* strategies, you can also specify the number of CPUs on which to run parallel compiles. For large designs, running parallel compiles can significantly reduce synthesis runtime because subdesigns that are to be compiled individually can be compiled at the same time. [Table 5-22](#) defines the available synthesis options.

Table 5-22 Synthesis Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Generate Scripts Only (ScriptsOnly)	Specifies whether to execute synthesis immediately or just write out the scripts for later use. When Generate Scripts Only is false, the appropriate tool is automatically invoked to execute the scripts as soon as they are generated. When Generate Scripts Only is true, the synthesis scripts are generated and written but not run. You can run them later by executing run.scr at the UNIX command prompt in the workspace/syn directory. NOTE: You can use the wait_for_activity_subproc command in conjunction with ScriptsOnly=true to determine when subprocess started with launch_activity_subprocess completes or is stopped.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the Sun grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as -m machine name to direct the job to a particular machine) in this text box. The specified options are passed to the bsub command (for lsf) or the qsub command (for grd).
Parallel Compile Style (ParallelCompileStyle)	If you select Parallel Compile Style = standard, a synthesis strategy is generated that runs parallel compiles directly on the specified host machine, either local or remote. If you select Parallel Compile Style = lsf or Parallel Compile Style = grd, a synthesis strategy is generated that uses LSF or GRD for individual parallel compiles (enabled only if DC Tcl mode or DC Graphical is enabled). Regardless of your choice of Run Style, you can select to use LSF or GRD for parallel compiles for synthesis strategies that use DC Tcl mode or DC Graphical.
Parallel Compile Options (ParallelCompileOptions)	If you select Parallel Compile Style = lsf or grd, you can specify the options you want to pass to the bsub or qsub commands.
Parallel Job Cpu Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs on which to execute parallel compiles. If you select a value greater than 1, your selected strategy is implemented by performing parallel compiles of designs that are to be compiled individually. If you select 1, your selected strategy is implemented by performing individual

Table 5-22 Synthesis Options (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
	compiles sequentially on a single CPU. Parallel Job Cpu Limit is enabled only if DC Tcl mode or DC Graphical is enabled.
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when synthesis completes or is ended. The default for the text box labeled “To” is your (the current user's) user name.
Skip reading \$HOME/.synopsys_dc.setup (NoHomeInit)	Force tool to not read the .synopsys_dc.setup file from \$HOME Default: 0 (false)
Use Design Compiler NXT (UseDCNXT)	Uses Design Compiler NXT for synthesis. Note: Design Compiler NXT is available starting with the P-2019.03 version of Design Compiler.
Enable Autoread Flow (EnableAutoReadDC)	Uses Autoread mode in DC to analyze the RTL files. Note: To enable this option for the IP, use coreBuilder to select the Autoread Supported? option that is under the Advanced tab in the Load Designs activity.

5.3.3 Viewing Licenses for Implementation Tools

Licenses available for synthesis appear on the Licenses tab of the Synthesize Activity View. Selecting a license enables coreConsultant to invoke synthesis tool features that require that license. For example, selecting BOA-BRT enables the use of behavioral optimization of arithmetic (BOA) and behavioral retiming (BRT) algorithms during synthesis. You can specify license availability for the following tools:

- DC-Ultra-Opt
- DC-Ultra-Features
- DesignWare Foundation
- BOA-BRT
- Behavioral-Compiler
- Test-Compiler
- Test-Compiler-Plus
- Test-DFTC-TMAX
- DC-Expert
- PrimeTime

If licenses are not available for certain features, then coreConsultant does not write commands using those features in the generated scripts. In TCL mode only, you can force implementation tools to wait for licenses if they are not available using the Wait for Licenses parameter on the Licenses tab to indicate how many minutes to wait for licenses to become available. The default value is 0, meaning that the tool does not wait for an unavailable license. There is also available from the **Edit > Preferences > Synthesis** dialog “Wait

for licenses” for setting the default value of this parameter. (For more information about preferences, see [“Editing Tool Setup and User Preferences”](#) on page 177.)

5.3.4 Selecting Synthesis Reports

For some designs, the generation of certain reports consumes an unacceptably long amount of time. In the Reports tab of the Perform ASIC Synthesize activity, you can deselect any of the optional reports listed in [Table 5-23](#) that you do not want.

Table 5-23 Selecting Synthesis Reports

Synthesize Activity Parameter <GUI Label> (<Parameter Name>)	Description
Checker Reports	
Generate Through-Path report (DcThroughPathReport)	Specifies whether to generate a list of through paths (top-level inputs and outputs directly connected through combinational logic). Default: Do not generate this report.
Generate registered I/O report (DcRegisteredIOReport)	Specifies whether to generate a list of unregistered ports (ports not directly connected to a register at the input or output of the design). Default: Do not generate this report.
Generate DC setup report (DcSetupReport)	Specifies whether to generate a synthesis setup report. Default: Generate this report.
Generate check design report (DcCheckDesignReport)	Specifies whether to generate a DC design check report. Default: Generate this report.
Generate latch report (DcLatchReport)	Specifies whether to generate a DC latch report. Default: Generate this report.
Generate loops report (DcLoopsReport)	Specifies whether to generate a DC combinational loops report. Default: Generate this report.
Generate check test report (DcCheckTestReport)	Specifies whether to generate a DC check test report. Default: Generate this report.
Timing Reports	
Generate constraint report (DcConstraintReport)	Specifies whether to generate a DC constraint report. Default: Generate this report.
Generate internal timing report (DcInternalTimingReport)	Specifies whether to generate a DC internal timing report. Default: Generate this report.
Generate short I/O timing report (DcShortIOTimingReport)	Specifies whether to generate a DC short I/O timing report. Default: Generate this report.

Table 5-23 Selecting Synthesis Reports (continued)

Synthesize Activity Parameter <GUI Label> (<Parameter Name>)	Description
Generate long I/O timing report (DcLongIOTimingReport)	Specifies whether to generate a DC long I/O timing report. Default: Generate this report.
Generate Qor report (DcQorReport)	Specifies whether to generate a DC Qor report. Default: Generate this report.
Generate clock report (DcClockReport)	Specifies whether to generate a DC clock report. Default: Generate this report.
Generate timing exceptions report (DcTimingExceptionsReport)	Specifies whether to generate a DC timing exceptions report. This report contains false path and multi-cycle path information. Default: Do not generate this report.
Generate Path Group Report (PathGroupReport)	Specifies whether to generate a Path Group report. Default: Generate this report.
Utilization Reports	
Generate hierarchy report (DcHierarchyReport)	Specifies whether to generate a DC hierarchy report. Default: Generate this report.
Generate clock gating report (DcClockGatingReport)	Specifies whether to generate a DC clock gating report. Default: Generate this report.
Generate Self Gating Report (DcSelfGatingReport)	Controls whether or not a self-gating report will be generated. Only applicable for Design Compiler Reference Methodology. The report will be written out when both the SelfGating strategy parameter and DcSelfGatingReport parameters are enabled. The report gets generated as <design_name>.mapped.self_gating.rpt
Generate reference report (DcReferenceReport)	Specifies whether to generate a DC reference report. Default: Generate this report.
Generate power domains report (DcPowerDomainsReport)	Controls whether to generate a power domain report, and other reports for a multi-voltage synthesis flow using UPF files. The other reports are: power switches, supply nets, power state table, level shifters, isolation cells, and retention cells. Default: Generate this report.
Generate resources report (DcResourcesReport)	Specifies whether a resource report is generated, and whether it is for the top-level design only or for all designs. Default: Generate this report.
Generate register count report (RegisterCountReport)	Controls whether to generate a report detailing the number of registers present in the synthesized design.

Table 5-23 Selecting Synthesis Reports (continued)

Synthesize Activity Parameter <GUI Label> (<Parameter Name>)	Description
	Default: Generate this report.
Generate fanin/fanout report (DcFaninFanoutReport)	Controls whether to generate a report detailing fan-in and fan-out. Default: Do not generate this report.
Generate area report (GenerateAreaReport)	Controls whether to generate the area report for the current design. Default: Generate this report.
Physical Reports	
Generate ignored layers report (GenerateIgnoredLayersReport)	Reports the routing layers that are ignored during congestion analysis and RC estimation. The report gets generated for DCRM synthesis (only when PhysicalSynthesis is enabled) and FCRM synthesis run.
DFT Reports	
Generate DFT coverage estimate report (DFTCoverageEstimateReport)	Controls whether to generate a DFT coverage estimate report. This applies only if the DFT flow is enabled (for settings, see “Design for Test”) Default: Generate this report.
Automotive Reports	
Generate safety register rules report (ReportSafetyRegisterRules)	Controls whether to generate a safety register rules report. This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-8 on page 107 or Table 5-19 on page 119) Default: Do not generate this report.
Generate safety register groups report (ReportSafetyRegisterGroups)	Controls whether to generate a safety register groups report. This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-8 on page 107 or Table 5-19 on page 119) Default: Do not generate this report.
Perform & report safety related checks (ReportSafetyStatus)	Controls whether to perform and report checks related to safety_register_rules and safety_register_groups. This applies only if the AutomotiveSynthesisFlowType parameter is enabled (for settings, see Table 5-8 on page 107 or Table 5-19 on page 119) Default: Do not generate this report.

5.4 Launching a Synthesis Run

5.4.1 Performing ASIC Synthesis using Design Compiler

After you select the synthesis strategy, parameters, and options, click Apply in the Synthesize Activity View to complete the Perform ASIC Synthesis activity. coreConsultant generates custom, technology-specific synthesis strategy scripts to implement the reusable core according to your selected synthesis strategy.

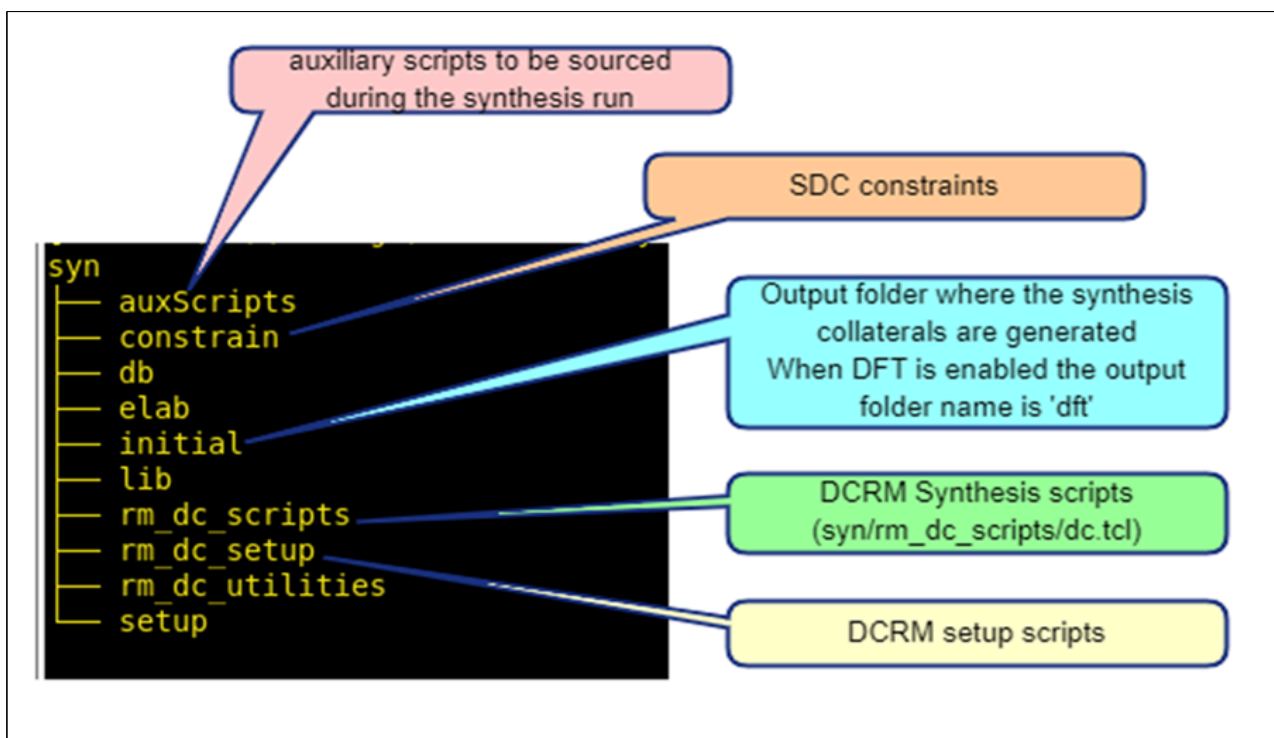
When you select Generate Scripts Only, coreConsultant generates the synthesis scripts for later use. To run the scripts, go to your workspace/syn directory and invoke the run.scr script from the shell.

To complete the Perform ASIC Synthesis activity in command line mode, use the `set_strategy_parameter`, `set_activity_parameter`, and `autocomplete_activity` commands. For example:

```
set_strategy_parameter Design_Compiler_Reference_Methodology QorEffort high
set_activity_parameter Synthesize StrategyName
Design_Compiler_Reference_Methodology
autocomplete_activity Synthesize
```

The synthesis folder structure post synthesis using Design Compiler is as shown in [Figure 5-1](#).

Figure 5-1 Synthesis Directory Structure – Design Compiler Synthesis



5.4.1.1 Synthesis Replay Scripts

Synthesis replay scripts capture the setup, variable settings, and synthesis commands used for a synthesis run. The scripts are in the directory `syn/.replay`. A shell script, `replay.sh`, residing in the `.replay` directory can be used to replay the complete synthesis process. The sole purpose of these scripts is to aid in packaging test cases to reproduce any synthesis errors using only Design Compiler or DC Graphical,

not coreTools infrastructure. Synopsys does not support modification and use of the replay scripts as an implementation flow.

5.4.1.2 Using the `restore_checkpoint` Command

Sometimes it is desirable to directly invoke Design Compiler on one of the complete synthesis stages within your workspace. The `restore_checkpoint` command is available in the workspace for this purpose. It does the following:

- Invokes the proper version of Design Compiler
- Loads libraries and the final DDC associated with the given phase
- Returns Design Compiler control to you at the specified phase for further analysis

It essentially enables you to 'open' a synthesis stage in Design Compiler for debugging, report generation and further analysis. To use this command from a coreTools workspace:

```
cd <workspace>/syn
restore_checkpoint.scr phase name
```

The *phase name* can be any complete synthesis phase beginning with 'constrain' (for example: initial, incr1, final).

5.4.2 Performing ASIC Synthesis using Fusion Compiler

To enable the Fusion Compiler option for synthesis, the NDM libraries and .db files are required. They are also required to perform the formal verification activity.

The IPs used in coreConsultant are categorized into three types. They are as follows:

- coreKit components
- IPXACT
- Raw RTL

IPXACT and Raw RTL are considered Fusion Compiler ready. But for coreKit, it is to be explicitly specified that it will run Fusion Compiler.



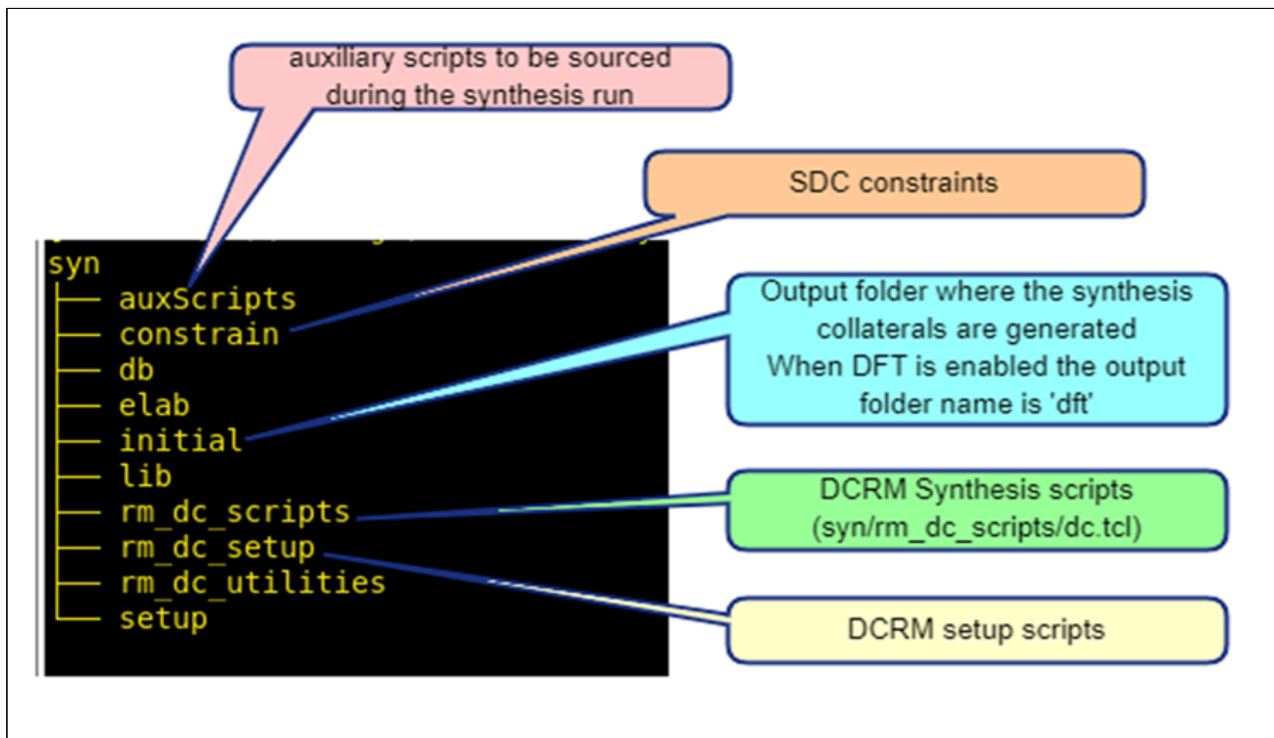
Note

All IPs are not Fusion Compiler ready. The FCRM strategy will not show up in the drop down menu if the IP is not Fusion Compiler ready.

For the Fusion Compiler option to be enabled, there are three conditions that are to be satisfied. They are as follows:

- IPs are to be Fusion Compiler ready.
- In the Specify Target Technology/ TechSetup, the appropriate NDM libs are to be specified. The physical libraries along with the logical libraries must also be specified for Fusion Compiler.
- Fusion Compiler version should be ≥ 2020.06 .

The synthesis folder structure post synthesis using Fusion Compiler is as shown in [Figure 5-2](#) on page 129.

Figure 5-2 Synthesis Directory Structure – Fusion Compiler Synthesis

5.4.2.1 Performing ASIC Synthesis Using QIK Reference Methodology for Synopsys Interface IP

The Quick Implementation Kit (QIK) for Synopsys Interface IP is a Reference Methodology for Fusion Compiler that has been designed with reference to the Synopsys Interface IP's.

QIK introduces support for Multi Corner Multi Mode (MCMM). Therefore, the setup required for QIK is different from the Fusion Compiler Reference Methodology. It also supports the ability for cell pruning (filtering the list of valid cells based on type).

5.4.2.1.1 Minimum Version Support

To use QIK, the minimum version for coreTools must be greater than or equal to V-2023.09 and the Fusion Compiler version must be greater than or equal to U-2022.12.

5.4.2.1.2 Methodology

To use QIK, the user needs to set the 'EnableQIK' parameter to '1' and provide definitions for the following:

- Modes
- Scenarios
- Corners
- Parasitics

Additionally, the user can specify the cells to be pruned and the list of active scenarios based on the synthesis stage.

5.4.2.1.3 Technology Reference Methodology Scripts

This parameter captures the path to the `rm_tech_scripts` delivered by Synopsys. When using QIK its mandatory for the user to specify the path to the `rm_tech_scripts`.

5.4.2.1.4 Parasitic Definition

The user can specify a maximum of four parasitic definitions. The information corresponding to each parasitic definition includes the following:

- Parasitic model name
- TLUPlus file name for the library
- Layer map file for the library



Note

- As a minimum, at least one Parasitic definition needs to be specified.
- If a parasitic model name is specified, then the corresponding TLUPlus file name is mandatory.
- The parasitic model name must be unique.

5.4.2.1.5 Mode Definition

The user can specify a maximum of four mode definitions. The information corresponding to each mode definition includes the following:

- Name of the mode
- Mode Constraint file (The mode constraint file contains the mode specific constraints)



Note

- As a minimum, at least one mode definition must be specified.
- The mode name must be unique.

5.4.2.1.6 Corner Definition

The user can specify a maximum of nine corner definitions. The information corresponding to each corner definition includes the following:

- Corner Name
- Corner Constraint File



Note

- The corner names must be unique and the corner constraint file must be specified by the user.
- The contents of the corner constraint file should include the PVT information that matches the library (`set_voltage`, `set_temperature`, `set_process_label`).

A sample Corner Constraint file content is as follows:

```
set_parasitic_parameters -corner [current_corner]
    -early_spec cworst_CCworst_T
    -late_spec cworst_CCworst_T
set_voltage 0.675 -object_list [get_supply_port VDD]
```

```
set_voltage 0.000 -object_list [get_supply_port VSS]
set_temperature -40
set_process_label ssgnp
```

5.4.2.1.7 Scenario Definition

The user can specify a maximum of nine scenario definitions. The information corresponding to each corner definition includes the scenario name.

The scenario name must follow the following convention:

<MODE NAME>::<CORNER NAME>



Note

- The mode and corner names must be part of the Mode and Corner definition
- The scenario name must be unique.

5.4.2.1.8 Active Scenario List

The user can specify the list of active scenarios as space separated values for a particular synthesis stage. If no active scenarios are specified, all scenarios will be active.

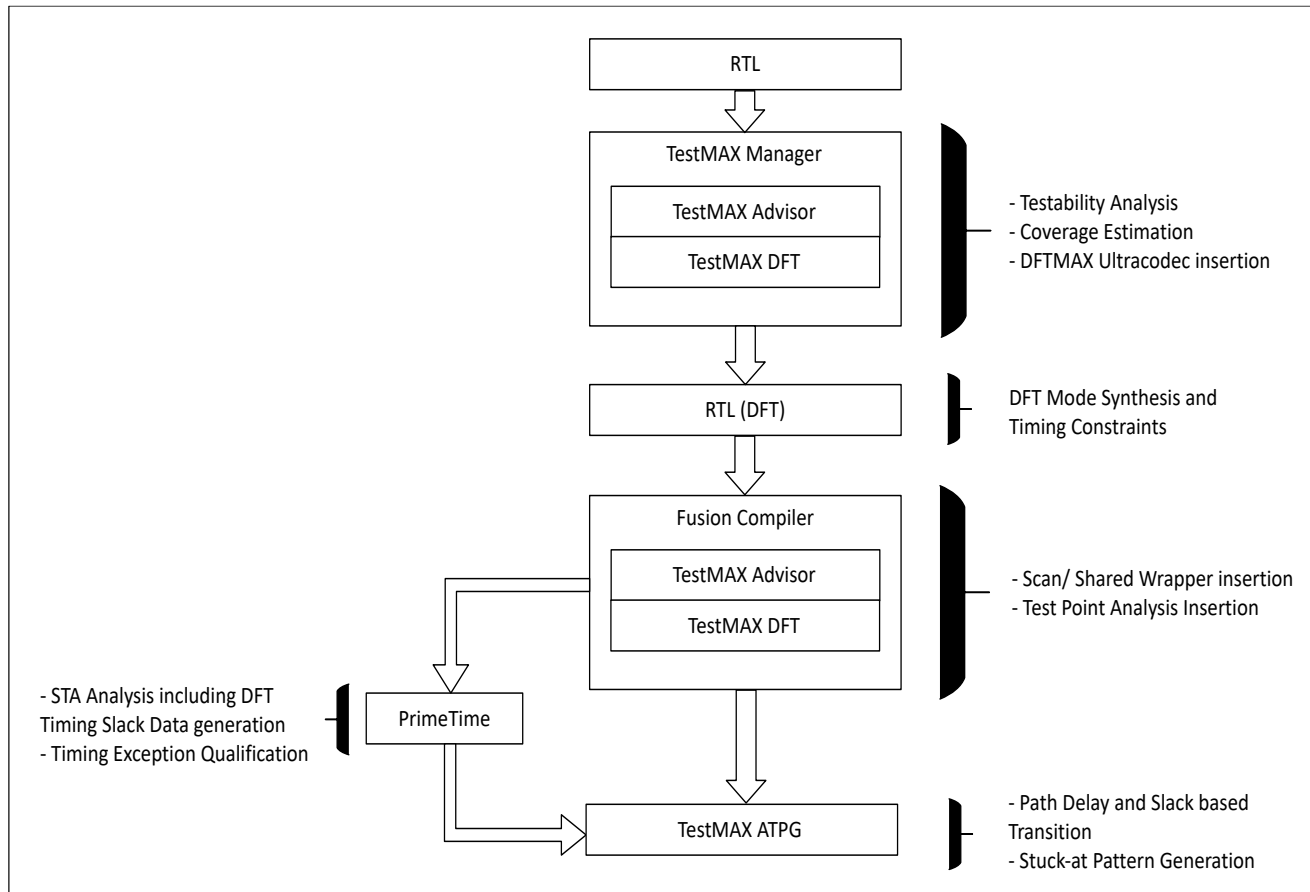
5.4.2.1.9 Cell Pruning List

The user can define a list of valid cells for a given type.

5.4.2.2 Performing ASIC Synthesis Using QIK Reference Methodology with TestMAX Manager

The TestMAX Manager tool provides a common interface for DFT implementation capabilities. Inserting the DFT feature at the RTL stage accelerates the validation of DFT with the design and provides a faster turnaround time before the physical design stage.

[Figure 5-3](#) on page [132](#) shows the TestMAX Manager flow chart.

Figure 5-3 TestMAX Manager Flow Chart

5.4.2.2.1 Data Input and Output Requirements for TestMax Manager (TMM) Flow

The input requirements for the TestMAX Manager (TMM) flow is as follows:

- RTL – Configured RTL
- Scan Setup Definition – Scan Configuration Definition
- Scan Ports Definition – Scan Input, Scan Output, Scan Enable definition

The output requirements for the TestMAX Manager flow that are generated by the TestMAX Manager is as follows:

- RTL with inserted DFT ports
- Updated scan configuration

5.5 Checking Synthesis Results

As part of the Perform ASIC Synthesis activity, coreConsultant extracts and analyzes several Design Compiler reports and design checks. To check synthesis status, go to the Report tab of the Synthesize Activity View pane.

The Reports tab shows the synthesis status for the current core, including the information listed in [Table 5-24](#). This Activity View also provides options to end the running synthesis job, to display analyzed results on completion of the job, and view Design Compiler summary reports.

Table 5-24 Activity Status Messages

Status Message	Description
Run Style	Identifies the type of synthesis run (local, lsf, or grd).
Results file	Name of the file that contains the top-level synthesis report (the summary index file).
Execution Host	Name of the machine on which the synthesis job is running (for the local run style only).
Job ID	The identifying job number for the current synthesis run.
Job Status	The state of the current synthesis run (running, done, or ended.)
Kill Job	Terminates the current synthesis job immediately. The Kill Job button is active (not greyed out) only while the synthesis run is in progress.

[Table 5-25](#) lists the Design Compiler Summary Reports available. Because these contain digested synopses of the raw information produced by Design Compiler during the current synthesis run, the coreConsultant summaries presented on the Report tab of the Synthesize Activity View provide a quick, effective means for you to view and analyze the results of your latest synthesis iteration (and prepare to improve on the results of the latest iteration). Wherever a hyperlink appears, you can “drill down” to progressively more detailed layers of information.

Table 5-25 Design Compiler Summary Reports

Report	Description
Compile Summary	Lists the name of the design, the target technology library, the amount of CPU time consumed, and the version of Design Compiler used.
Log File Summary	Lists the name and location of the log file, the error count, and a brief description of each error.
Completed Synthesis Stages	Lists completed stages, the error count, the time completed, and the current status.
Stage Results Summary	Lists the stage name, the critical path slack and total negative slack, the area, and the number of violating nets for each stage.

5.6 Refining Your Synthesis Intent

If the synthesis results do not meet your design goals (or if synthesis takes an excessive time to complete), you should first consult the core-specific documentation for your design. The core documentation may contain known problem areas or suggested special procedures for improving synthesis results.

If the core developer's tips do not yield satisfactory results, follow the guidelines for the specific problems described in the following sections. Experiment with a variety of solutions to determine the best set of synthesis intent and strategy specifications for your particular core configuration, technology library, and synthesis tool version.

To implement and check your changes more rapidly, you can also use the batch mode interface to automate the process. (For more information, see "[Batch Mode](#)" on page 175.)

5.6.1 Correcting Timing Problems

To correct timing problems, try setting QoR Effort to high. If that does not correct the problem, try the solutions for the specific problems described in the following sections.

5.6.1.1 Many Large Violations

Start by trying a high effort incremental strategy. Also, check the design-specific documentation to make sure you do not specify a clock speed that exceeds the advertised maximum speed of the core. Verify that your top-level MaxInputDelay and MaxOutputDelay specifications are reasonable.

5.7 Generating ATPG Test Vectors

Testability is an important issue for all IC designs. If you use our DFT features to generate test models during synthesis, you can automatically invoke TestMAX in order to generate test patterns. The Generate Test Vectors activity allows you to integrate the test strategy for the component into the overall synthesis flow of your chip. With this activity, you can create TestMAX-generated ATPG test vectors to be used in the chip simulation, or incorporate a set of test vectors supplied with the IP.



Note

The **Edit > Tool Installation Roots...** menu item and dialog box is where you can set the path to your TestMAX tool installation for this feature.

The TestMAX User Guide provides information for you on how to write scripts to generate test vectors for your subsystem.

Table 5-26 explains the major categories of customization for this activity.

Table 5-26 ATPG Test Vector Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Strategy	
Strategy (StrategyName)	Selects the strategy for generating test vectors. The following choices are presented: ■ TMAX Reference Methodology Implements the TestMAX reference methodology flow for stuck-at, path delay, and transition delay testing. These test modes can be run in any combination or individually. The options for this strategy are described in Table 5-27 on page 137. This strategy is only enabled for TestMAX version L-2016.03 or newer. ■ ATPG basic scan strategy Creates test vectors using the Basic_scan_only engine of TestMAX. This is the recommended starting point for generating vectors, as it provides fast, good coverage for full scan designs. There are no options for this strategy. ■ ATPG user defined strategy Runs a custom Tcl script to generate test vectors with TestMAX. The libraries and design netlist are loaded, and control is turned over to the user script, which is responsible for generating and saving the test vectors. The coreTools script generates reports and exits from TestMAX if the user script does not exit from TestMAX. The option for this strategy is described in Table 5-28 on page 140. Default: Depends on version of TestMAX: ■ For version L-2016.03 or newer: TMAX Reference Methodology ■ For older supported versions: ATPG basic scan strategy

Table 5-26 ATPG Test Vector Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Setup	
Output Stage Name (OutputStageName)	Specifies the synthesis stage to use for test vector generation. Default: None
Input Stage Name (InputStageName)	Specifies the synthesis stage netlist to be used for generating the test vectors. Default: final synthesis stage
Input Netlist Format (InputNetlistFormat)	Specifies the format of the design netlist to be read into TestMAX. Default: verilog
Test Libraries (TestLibraries)	Specifies files containing the libraries to be used by TestMAX. The value may be a list and may contain wildcards that are acceptable to TestMAX. The test libraries are the Verilog simulation models for the target libraries used during synthesis. They provide the description of the logical functionality of all the cells in the standard cell library. They are needed for the ATPG tool to comprehend the logic function implemented by each standard cell instantiated, to generate test vectors for the faults. Default: None
Libs Missing Cell Define (LibsMissingCellDefine)	If the libraries specified in TestLibraries do not have <code>`celldefine</code> and <code>`endcelldefine</code> around the module definitions, setting this parameter to 1 causes the <code>-library</code> switch to read the libraries. Default: 0
Test Pattern Format (TestPatternFormat)	Format for saving the test patterns. Default: None
TestMax Command File (UserStrategyFile)	Specifies the name of the user script containing the TestMAX commands to execute for the <code>ATPG_user_defined_strategy</code> . The user script should assume that the models are already loaded into TestMAX before the script is invoked. Default: None
Execution Options	
Parallel Job CPU Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs on which to execute parallel compiles. If you select a value greater than 1, your selected strategy is implemented by performing parallel compiles of designs that are to be compiled individually. If you select 1, your selected strategy is implemented by performing individual compiles sequentially on a single CPU. Parallel Job Cpu Limit is enabled only if DC Tcl mode or DC Graphical is enabled.
Generate Scripts Only (ScriptsOnly)	Specifies whether to execute synthesis immediately or just write out the scripts for later use. When Generate Scripts Only is false, the appropriate tool is automatically invoked to execute the scripts as soon as they are generated.

Table 5-26 ATPG Test Vector Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
	When Generate Scripts Only is true, the synthesis scripts are generated and written but not run. You can run them later by executing run.scr at the UNIX command prompt in the workspace/syn directory.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the Sun grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as -m <i>machine name</i> to direct the job to a particular machine) in this text box. The specified options are passed to the bsub command (for lsf) or the qsub command (for grd).
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when synthesis completes or is ended. The default for the text box labeled "To" is your (the current user's) user name.
Clean up files when repeating simulation (CleanupFiles)	If selected (default), this option deletes existing simulation files before redoing the Setup and Run Simulation activity. If not selected, no cleanup is performed.
Use TestMAX ATPG (UseTestMAX_ATPG)	Uses TestMAX ATPG for test vector generation. Note: TestMAX ATPG is available starting with the L-2016.03 version of TestMAX.

[Table 5-27](#) presents the options for the TMAX Reference Methodology strategy for generating test vectors and fault lists.

Table 5-27 Options for the TMAX Reference Methodology Strategy

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
TestSelection	
Stuck At Test (StuckAtTest)	When TRUE, stuck-at fault testing is performed. Default: TRUE
Transition Delay Test (TransitionDelayTest)	When TRUE, transition delay testing is performed. Default: FALSE
Transition Delay Coverage Goal (TransitionDelayCoverageGoal)	When TransitionDelayTest is TRUE, this parameter sets the transition delay coverage goal. Note: A coverage goal near 100% can cause significant processing time.

Table 5-27 Options for the TMAX Reference Methodology Strategy (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	Default: 100
At Speed Test (AtSpeedTest)	When TRUE, at-speed testing is performed. The AtSpeedScript parameter accepts a script for updating SPF waveforms for the testing. Default: 0
At Speed Script (AtSpeedScript)	Script to run in order to update the SPF waveforms for at-speed testing.
Path Delay Test (PathDelayTest)	When TRUE, path delay testing is performed. Default: FALSE
Random Incremental Atpg (RandomIncrementalAtpg)	When TRUE, random incremental ATPG is run for all enabled tests. The IncrementalRuns parameter sets the number of incremental runs for each test. Default: FALSE
Incremental Runs (IncrementalRuns)	When RandomIncrementalAtpg is TRUE, this parameter sets the number of incremental runs for each enabled test. Default: 0
Strategy Controls	
TMAX Testbench Config (MaxTbConfigFile)	Path to a file with options for the TestMAX reference methodology scripts. Default: <synDir>/<OutputStageName>/script/tmax_maxtb_config.tcl which has the default settings from the TestMAX reference methodology scripts.
Optimize Patterns (OptimizePatterns)	When TRUE, pattern count is optimized, at the expense of runtime. This optimization applies to the stuck-at and transition delay fault models; other fault models may not get better results, so they are not supported in the reference methodology. Default: FALSE
Persistent Fault Model (PersistentFaultModel)	When TRUE, persistent fault modeling is performed, which can result in smaller overall pattern sets, at the expense of runtime. For details on the persistent fault model, see the TestMAX User Guide. Default: FALSE, but automatically set to TRUE when TransitionDelayTest and StuckAtTest are both TRUE.
TMAX Debug Mode (TMAXDebugMode)	When TRUE, creates a more complete log file and additional reports for debugging. Default: FALSE
Integrate Path Delay Patterns (IntegratePathDelayPatterns)	When TRUE, path delay patterns are included with other tests and fault graded to reduce pattern count.

Table 5-27 Options for the TMAX Reference Methodology Strategy (continued)

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
	When FALSE, path delay patterns are used only for characterization or speed binning, and are kept as a separate test. Default: TRUE
Rules To Downgrade (RulesToDownGrade)	List of errors to downgrade without masking. Default: None
Rules To Mask (RulesToMask)	List of errors to mask. Default: None
Black Box Instances (BlackBoxInstances)	Instances to replace by TIEX primitives. Default: None
Exception Files (ExceptionFiles)	Paths to SDC files for timing exceptions. Each path must be relative to the synthesis directory. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Table 6-8 on page 153).
Delay Path File (DelayPathFile)	Path file for path delay testing. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Table 6-8 on page 153).
Add Pi Constraints (AddPiConstraints)	Add a constraint for a primary input The value of the parameter is a list of constraints in the form of: <port> <logic state> For example: <pre>set_strategy_parameter TMAX_Reference_Methodology AddPiConstraints [list test_se 0 rst_n 1]</pre>
Global Slack File (GlobalSlackFile)	Path file for path delay testing. Default: The output from static timing analysis when the LinkToATPG option was set to TRUE in the Static Timing Analysis activity (see Table 6-8 on page 153).
Use the scan compression Protocol (UseScanCompressionProtocol)	When TRUE, uses the scan compression configuration SPF file. If scan compression is not enabled (see “ Scan Insertion ” on page 78), this parameter is automatically disabled and the normal SPF file is used. Default: 0

Table 5-28 presents the options for the ATPG user defined strategy for generating test vectors and fault lists.

Table 5-28 Option for the ATPG User Defined Strategy

Strategy Parameter <GUI Label> (<Parameter Name>)	Description
TestMax Command File (UserStrategyFile)	Specifies the name of the user script containing the TestMAX commands to execute for the ATPG_user_defined_strategy. Your script must assume that the models are already loaded into TestMAX before the script is invoked. Default: None

6

Verifying Your Implementation

This chapter contains the following sections:

- [“Overview”](#) on page 142
- [“Formal Verification”](#) on page 144
- [“Static Timing Analysis Interface”](#) on page 149
- [“Explore RTL Architecture Activity”](#) on page 157
- [“Timing Constraints Manager”](#) on page 162
- [“Setup and Run Simulations”](#) on page 169
- [“Integrating the Core Into Your Design”](#) on page 170

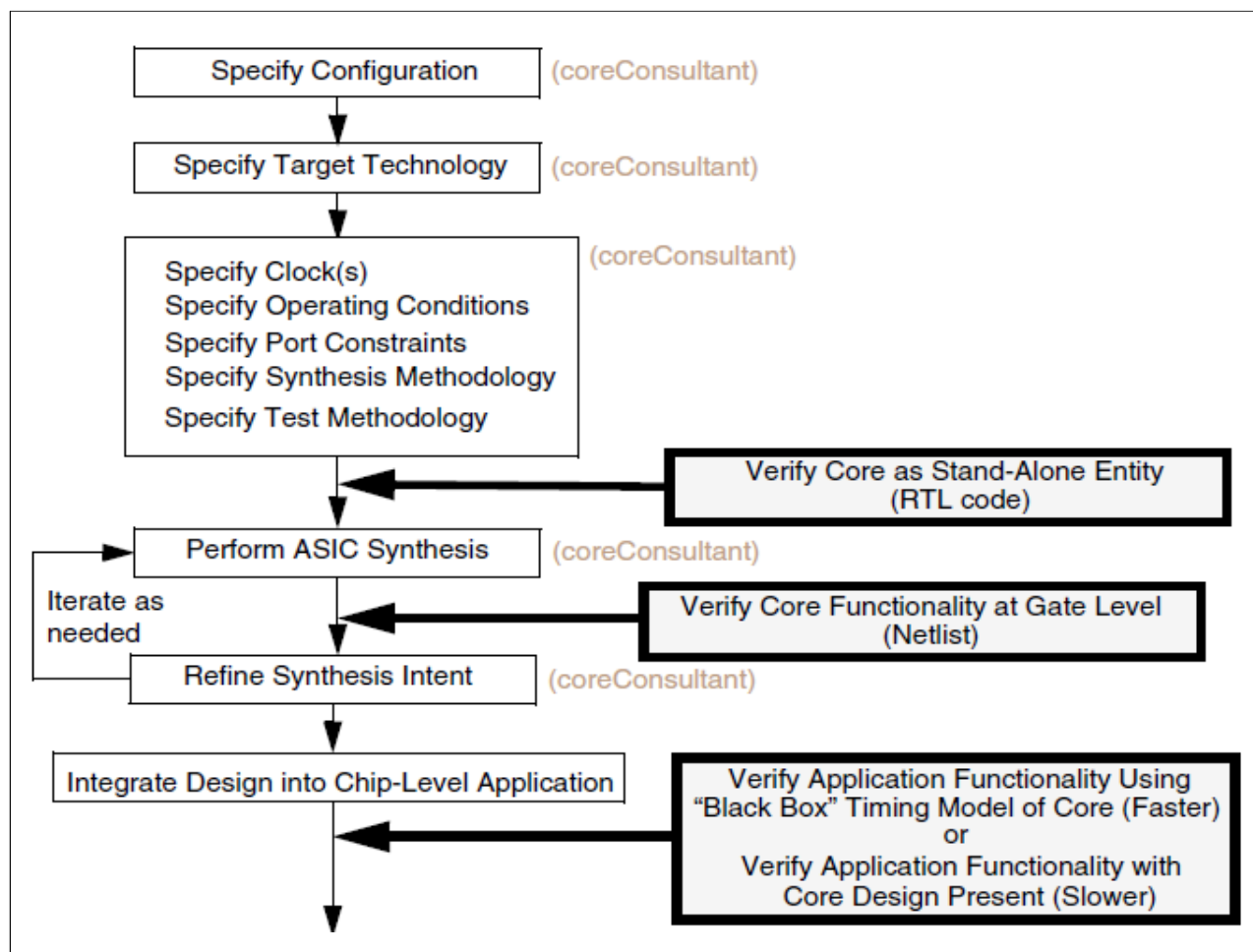
6.1 Overview

The coreConsultant tool does not directly support simulation functionality. The coreKit may include one or more testsuite regression environments (TREs) with which you can verify the functionality of the configured core at the component level. The type of testing available depends on whether the core's source files are encrypted, and on whether you have access to Synopsys simulation tools, which are the only simulators capable of reading the RTL encryption used by coreConsultant.

If the coreKit does not include a TRE, or if you wish to perform component functionality tests not covered by the coreKit's TREs, you can create your own test environment to verify the core.

Although the scope and relative sequencing of component-level verification activities in coreConsultant may vary from core to core, testing tends to occur at one or more of three logical junctures in the flow (as shown in [Figure 6-1](#) on page 143):

- After configuration, but prior to synthesis, the stand-alone core can be tested at the RTL level. This is the fastest way to test the functionality of a stand-alone core, since the design has not yet been mapped to a specific technology. This option is available only if the RTL source files are not encrypted, or if you have access to one of the Synopsys simulators (VCS) to read the encrypted RTL code.
- After synthesis, the functionality of the core can be tested at the gate-level with the netlist providing input for verification. Such testing is the only verification option when the core consists of encrypted RTL and you do not have the Synopsys simulators required to read the encrypted files. At this stage, you can also use formal verification to check your synthesized implementation and previously verified RTL design for functional equivalence. See [“Formal Verification”](#) on page 144.
- Once the configured and synthesized core is integrated into your chip-level application, you might want to test at one or both of the following levels:
 - Use a “black box” timing model of the core to check the timing of the entire application more quickly than if you had inserted the synthesized core.
 - Test the complete chip-level application (including your synthesized core) at the gate-level to make sure it functions entirely as expected.

Figure 6-1 Component Verification in the coreConsultant Activity Flow

6.2 Formal Verification

The Formal Verification activity uses Synopsys Formality (fm_shell) to check two designs for functional equivalence. You can check the gate-level design from a selected phase of a previously executed synthesis strategy against either the RTL version of the design or the gate-level design from another stage of synthesis (for more information on synthesis strategies, see [“Choosing Synthesis Activities in the GUI”](#) on page 94).

Running formal verification through coreConsultant involves:

- [“Executing the Formal Verification Activity”](#)
- [“Checking Formal Verification Results”](#) on page 148

6.2.1 Executing the Formal Verification Activity

To execute the Formal Verification activity, select Verify Component > Formal Verification in the Activity List pane. In this Activity View, you specify the Activity Controls (see [Table 6-1](#) for parameter definitions) and Execution Options (see [Table 6-4](#) on page 147 for parameter definitions) for your verification.

Table 6-1 Formal Verification – Activity Controls

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Reference Stage Name (ReferenceStageName)	Specify the design stage to use as the golden reference design. The legal values are RTL (use the RTL version of design as reference design) or any completed synthesis stage beginning with initial. (You cannot specify constrain or elab as the reference design stage.)
Use Reference Stage Netlist (UseReferenceStageNetlist)	Specify that the verification use the netlist from the reference stage instead of the database. Values are 1 (true) and 0 (false). Default: 0
Reference Stage Netlist (ReferenceStageNetlist)	Specify the name of the reference stage netlist.
Target Stage Name (TargetStageName)	Specify the stage of the design that you want to verify. Legal values are any completed synthesis stage beginning with initial. (You cannot specify constrain or elab as the target design stage.) coreConsultant uses Synopsys Formality to formally verify the Target Stage Name design using the Reference Stage Name design as the golden reference.
Use Target Stage Netlist (UseTargetStageNetlist)	Specify that the verification use the netlist from the target stage instead of the database. Legal values are 1 (true) and 0 (false). Default: 0
Use UPF PG Netlist (UseUPFPGNetlist)	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG information is in the UPF files. Default: False
Target Stage Netlist	Specify the name of the target stage netlist.

Table 6-1 Formal Verification – Activity Controls (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
(TargetStageNetlist)	Default: <i>component.v</i>
UPF Auxiliary Script Name (UpfAuxiliaryScriptName)	Defines the name of a script to be sourced when using a UPF flow. This script is sourced after the implementation design is loaded, but before issuing the 'set_top' command. It can be used to specify retention register models that match in both the reference and implementation designs.
Auxiliary Script Name (AuxiliaryScriptName)	Specify the name of any auxiliary fm_shell script that you want to execute before the fm_shell verify command. For example, you may want to insert commands to set or remove compare points.
Use Generated Svf (UseGeneratedSvf)	Controls whether the SVF generated during synthesis is used in Formality. Default: 1

Table 6-2 Formal Verification – Alternate Strategies

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Alternate Strategies	
Valid alternate strategies names (ValidAlternateStrategies)	List of valid alternate strategy names for Formality
Alternate strategies list (AlternateStrategies)	Run the Formality command run_alternate_strategies using the specified strategies. For a list of allowed strategies, see the Valid Alternate Strategies field. Use whitespace to separate each strategy in the list. To avoid running the run_alternate_strategies command, leave this field blank.
Run all valid alternate strategies (RunAllValidAlternateStrategies)	Run all valid alternate strategies defined in the version of Formality being used. This parameter is not related to the -run_all which is used in the run_alternate_strategies command. Default: 0 (false)
Parallel alternate strategies (ParallelAlternateStrategies)	Number of alternate strategies run in parallel Default: 4
Alternate strategies extra arguments (AlternateStrategiesExtraArguments)	Extra arguments appended to the run_alternate_strategies Formality command.

Table 6-2 Formal Verification – Alternate Strategies (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Alternate strategies run style (AlternateStrategiesRunStyle)	Defines the protocol for launching worker processes on remote .The protocol corresponds to the type of compute farm manager you are submitting to. When RunStyle is 'local' then AlternateStrategiesRunStyle defaults to 'grd' otherwise AlternateStrategiesRunStyle is set to the RunStyle.
Alternate strategies run options (AlternateStrategiesRunOptions)	Grid options for launching distributed jobs This attribute value is passed to the set_host_options command with the -submit_command argument.

Table 6-3 Formal Verification – DPX Flow

Activity Parameter <GUI Label> (<Parameter Name>)	Description
DPX Flow	
Enable DPX Flow (EnableDPXFlow)	DPX allows more partitions to be verified in parallel compared to the standard Formality equivalence checking solution. The DPX Manager (the main Formality process you interact with) saves a session file that contains the entire design space and verification context. Each worker process reads the session file and accepts tasks from the manager that consists of instructions to verify a partition using a specific solver strategy. Workers can accept one task per core and can run N number of tasks in parallel. When a task completes a specific effort level, the status is returned to the DPX Manager. The feature is can be enabled when the Formal Verification RunStyle activity parameter is set to 'grd' and the license check for 'Formality-DPX' is successful. Additional information on the DPX flow can be found in the Formality User Guide.
DPX run style (DPXRunStyle)	Defines the protocol for launching worker processes on remote or local machines. The protocol corresponds to the type of compute farm manager you are submitting to or, when not using a compute farm manager, to the method used to access the compute hosts. When RunStyle is 'local' then DPXRunStyle defaults to 'grd' otherwise DPXRunStyle is set to the RunStyle.
DPX run options (DPXRunOptions)	Grid options for launching distributed jobs. This attribute value is passed to set_dpx_options command with the -submit_command argument.
Specifies the number of machines	Specifies the maximum number of machines requested for processing distributed tasks.

Table 6-3 Formal Verification – DPX Flow (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
(MaxWorkers)	The default is four machines.
Number of parallel tasks per machine (MaxCores)	Specifies the maximum number of compute cores per machine assumed to be available for processing distributed tasks. The default is four cores per machine.

Table 6-4 Formal Verification – Execution Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Generate Scripts Only? (ScriptsOnly)	When this box is checked, coreConsultant generates and writes the formal verification scripts but doesn't run them. If the box is not checked (the default setting), coreConsultant automatically invokes fm_shell to execute the scripts as soon as they are generated.
Run Style (RunStyle)	Specifies whether to execute synthesis on the local host (the default), through the load sharing facility (LSF), through the Sun grid engine (GRD).
Run Style Options (RunStyleOptions)	If you select Run Style = lsf or Run Style = grd, you can enter options (such as -m <i>machine name</i> to direct the job to a particular machine) in this text box. coreConsultant passes the specified options to the bsub command (for lsf) or the qsub command (for grd).
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when formal verification completes or is ended. The default for the text box labeled "To" is your (current user's) user name.

Clicking Apply causes a script to be generated to run Formality comparing the synthesis results from the two stages (or possibly from one stage and the RTL). If the Scripts Only option is not chosen, Formality is launched and you can switch to the Report tab to see the job status and results.

At the completion of fm_shell execution, the fm_shell reports are in your workspace/syn/stage/report directory, where stage is your selected Target Stage Name. coreConsultant also generates an fm_shell save_session command. The resulting .fss file is in your workspace/syn/stage/db directory.

6.2.1.1 UPF Support in Formality

If the UPF flow was enabled for synthesis, the appropriate UPF files are automatically loaded into Formality for both the reference and implementation designs. If the reference design is the RTL, then the original UPF file is loaded. In all other cases, the UPF file written at the completion of the synthesis stage is loaded into Formality. See the "UPF Auxiliary Script Name" entry in [Table 6-1](#) on page 144 for more information.

6.2.2 Checking Formal Verification Results

The coreConsultant tool automatically executes the following commands to extract reports from fm_shell:

- `report_libraries -short`
- `report_failing_points`
- `report_compare_points` (generated only if you specify an Auxiliary Script)
- `diagnose`
- `report_error_candidates`

The output for each of the fm_shell report commands is in your *workspace/syn/stage/report* directory and is viewable through the Reports tab of the Formal Verification Activity View.

For more information about diagnosing and fixing failing points, see the Formality User Guide. If you need to work with Formality directly to diagnose failing points, you can load the saved session file (*workspace/syn/stage/final/db/Design.fss*) directly into Formality.

To complete the Formal Verification activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands. For example:

```
set_activity_parameter FormalVerification ReferenceStageName RTL
set_activity_parameter FormalVerification TargetStageName incr1
autocomplete_activity FormalVerification
```

6.3 Static Timing Analysis Interface

The **Static Timing Analysis** activity provides an interface to PrimeTime for performing static timing analysis. The activity is within the “Verification” group of activities, is not visible in testbench workspaces, and requires that a tool root is defined for PrimeTime.

Within this activity, you can choose to generate the following reports:

- Paths crossing clock domains
- Standard STA reports (as defined by the PrimeTime reference methodology)
- Power analysis with PrimeTime-PX

The report for clock domain crossing paths will be based on the SolvNetPlus article on that topic. The standard STA reports will come from integration of support for the PrimeTime Reference Methodology. In both cases, HTML reports are available from the “Reports” tab and also referenced with HTML links.

In addition to specifying report generation, you can specify the input data required to run the static timing analysis. This consists of either a synthesis stage name (using coreTools synthesis), or a list of netlist files (using a non-coreTools based synthesis process). You can also provide your own SDC file (optional) to override the one generated by Design Compiler, an SDF or SPEF file (optional), and a PrimeTime script file (optional). For some of the more configurable reports, the ability to specify command line options will be provided. You can also generate but not run the PrimeTime scripts, and launch scripts through load sharing programs.

Table 6-5 Static Timing Analysis -- Netlist Setup

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Use externally generated netlist (UseExternalNetlist)	Check this box to specify a netlist from a source outside this workspace. Leave unchecked to choose a netlist in this workspace.
Use UPF PG Netlist (UseUPFPGNetlist)	Use netlist with PG (power/ground) for Golden UPF flow. When set to false, PG information is in the UPF files. Default: false
Input stage name: (InputStageName)	Name of the stage that contains the design to be analyzed. Default: Last <incr2>
External netlist file: (NetlistFile)	Path to the target netlist, required if an externally-generated netlist is to be analyzed.
Constraints File (ConstraintsFile)	SDC Constraints File to be applied to the netlist. Default: (=sPt::getDefaultConstraintsFile)
Back-annotated data: (BackannotatedData)	Format of back-annotated data to apply to the netlist. Default: None
Back-annotated data file: (BackAnnotatedDataFile)	File containing data to back-annotate onto the netlist. Default: (=sPt::getDefaultBackAnnotationData)

Table 6-5 Static Timing Analysis -- Netlist Setup (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Back-annotated command string: (BackAnnotationOptions)	Specify any switches and options to pass to “read_sdf” or “read_spef” to guide the back annotation.

Table 6-6 Static Timing Analysis -- Analysis Setup

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Timing Analysis Setup	
Analysis Mode (AnalysisType)	Choose the type of analysis to be performed on the netlist Default: On-chip Variation
Set Delay Derating Factors (TimingDerateValue)	Set delay derating factors for either the design or a specified list of instances. Default: No derating
Timing Derate Options (TimingDerateOptions)	Arguments to be passed to the ‘set_timing_derate’ command in addition to ‘-early’ or ‘-late’.
Timing Derate Factor (TimingDerateValue)	Timing Derate value to be applied. Default: 1
Use Propagated Clocks (UsePropagatedClocks)	Setting this parameter to ‘true’ specifies that delays be propagated through the clock network to determine latency at register clock pins. A value of ‘false’ means that ideal clocking is assumed. Default: 0
Analyze netlist in mission mode (AnalyzeInMissionMode)	Perform timing analysis with the netlist in ‘mission’ mode, that is, with scan mode disabled. In this mode, any scan-enabled pin inserted by Design Compiler will be driven to a value which disables scan mode. Default: 1
Enable ECO Flow (EnableECOFlow)	Enables the ECO Flow in PrimeTime. Enabling it, PrimeTime would run the command ‘fix_eco_timing’. This fixes the setup timing violations, set as default, through Engineering Change Order (ECO) changes.
Options for fix_eco_timing (FixECOTimingArgs)	Defines arguments to be passed to the fix_eco_timing when writing the Static Timing Analysis intent file.

Table 6-6 Static Timing Analysis -- Analysis Setup (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Auxiliary script name: (AuxiliaryScriptName)	Path to optional Static Timing script. The script is executed after the design has been loaded and constrained in PrimeTime, before the update_timing command which performs the timing analysis.
Custom Script	
Custom PrimeTime Script (PTScriptFile)	Path to optional PrimeTime script. PrimeTime will execute this script at the end of its run, after all selected reports have been generated.
Power Analysis Setup	
Power Analysis Flow (PowerAnalysisFlow)	Power Analysis flow to use: select 'VCD/VCD+/FSDB' for analysis using waves, 'SAIF' for analysis based on a SAIF file, or 'Vector-free' to estimate power without simulation data. Default: Vector-free
Power Analysis Mode (PowerAnalysisMode)	Specify Power Analysis mode. Use this parameter to choose either averaged or time-based analysis. This feature is supported in PrimeTime-PX 2008.12 or newer. Default: Averaged
Switching Activity File (SwitchingActivityFile)	Switching activity files to be used by PrimeTime-PX for power analysis. For vector-based analysis, this is either a VCD/VCD+/FSDB or SAIF file. Important: If multiple files are given, all of the files specified for this option must be of the same type (so you cannot mix VPD and SAIF switching activity files in the same session). For the vector-free flow, this is an optional file containing toggle and switching commands for PrimeTime-PX.
Simulation Type (SimulationType)	Use this parameter to specify whether your VCD (or VCD+) wave file was generated from: ■ "RTL" simulation ■ "gate" - gate-level simulation with SDF ■ "zero_delay" - gate-level without SDF Default: RTL Simulation
SAIF Name Mapping File (SaifNameMapFile)	For Power Analysis using data (waves or SAIF) captured from an RTL simulation, this option file can be used to specify mapping between RTL and gate-level object names. As the names of RTL objects may change after synthesis, providing a mapping file may help to increase the accuracy of power calculations by helping PrimeTime-PX to annotate switching activity accurately to the gate-level netlist. Default: (=sPt::getDefaultSaifMapFile)

Table 6-6 Static Timing Analysis -- Analysis Setup (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Testbench Path to design inst. (StripPath)	If the current design is instantiated as an instance in a testbench used to generate a VCD/VCD+ or SAIF file, this parameter is used to specify the instance of the current design as specified in the testbench, for example, test_toplevel/U_instance.
Start Time for Power Analysis (StartTime)	Start time of time window for power calculation. A value of 0 means that the calculation will start from the beginning of the wave file. The 'StartTime' parameter is now a list. If you specify one value in each box, that start time will be applied to all of the switching activity files listed. If you want to have different start times for different switching activity files, just enter a list (the number of entries in the list must correspond to the number of switching activity files specified). Default: 0
End Time for Power Analysis (EndTime)	End time of time window for power calculation. A value of -1 means that the calculation will cover the time window from the specified start time to the end of the wave file. The 'EndTime' parameter is now a list. If you specify one value in each box, that end time will be applied to all of the switching activity files listed. If you want to have different start and end times for different switching activity files, just enter a list (the number of entries in the list must correspond to the number of switching activity files specified). Default: -1

Table 6-7 Static Timing Analysis -- Override Tech Setup

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Override Tech Setup	
Override Search Paths (OverrideSearchPath)	A separate group of search paths for libraries which are required in Static Timing Analysis. These search paths are set in the variable :: search_path reflect in the <StageName>_pt.ptsh file. Default: null
Override Link Libraries (OverrideLinkLibraryMapping)	A separate group of link libraries which are specifically used for Static Timing Analysis. This attribute when enabled will override the previous Techsetup link libraries and setup the new libraries. These libraries will be set in the variable ::link_libraries reflected in the <StageName>_pt.ptsh file. This parameter uses a mapping technique which is used to run Static Timing Analysis with the updated SDC.

Table 6-7 Static Timing Analysis -- Override Tech Setup (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
	To provide the inputs to this parameter, you must set it up in a list of list sequence. For example, {{Link_Library_Tech_Setup_1 Override_Link_Library_1} {Link_Library_Tech_Setup_2 Override_Link_Library_2}} When the Static Timing Analysis is fired, coreTools checks for the Link_Library_Tech_Setup_* attributes in the SDC and updates it to the parameters of Override_Link_Library_*. Only Override_Link_Library_1 will be used to extract the Operating Condition and update the Link_Library_Tech_Setup_1 OPCON in the SDC and pt intent file. Default: null
Override Default Operating Condition (OverrideDefaultOperatingCondition)	An optional parameter when using different sets of libraries in Static Timing Analysis. It is used to change the Operating Condition during STA. If not provided, the tool will automatically extract the corresponding Operating Condition from the provided reference library given in OverrideLinkLibraryMapping. Default: null

Table 6-8 Static Timing Analysis -- Select and Configure Reports

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Report Design Attributes (ReportDesign)	Setting this parameter to 'true' runs the 'report_design' command. Default: 1
Report Clocks (ReportClocks)	Setting this parameter to 'true' runs the 'report_clocks' command. Default: 1
Report Register Information (ReportRegisterInfo)	Creates a report which lists the number of registers in the IP. The number of positive- and negative-edge flops per domain, and the number of latches per domain, along with a list of all such registers found. Default: 0 (false)
Report Synchronization Flops (ReportSyncPaths)	Creates a report showing the synchronization flops in the design. Default: 0 (false)
Report Case Analysis (ReportCaseAnalysis)	Runs the 'report_case_analysis' command. Default: 0
Report Timing Exceptions (ReportExceptions)	Runs the 'report_exceptions' command. Default: 1

Table 6-8 Static Timing Analysis -- Select and Configure Reports (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Report Timing Exception Options (ReportExceptionsOptions)	Options and switches to pass to the 'report_exceptions' command.
Report Analysis Coverage Check Type (ReportAnalysisCoverageCheckType)	Specifies arguments to the -check_type option for the report_analysis_coverage command for PrimeTime. Default: setup recovery removal min_pulse_width clock_gating_setup If the synthesis strategy is Fusion_Compiler_Reference_Methodology and the FCRM Strategy parameter (EnableQIK) is set, the DefaultValue list is setup recovery removal min_pulse_width clock_gating_setup hold.
Report Timing (Max Delay) (ReportMaxTiming)	Runs the 'report_timing' command to report 'max' path delays. Default: 1
Report Timing (Max Delay) Options (ReportMaxTimingOptions)	Additional options and switches to pass to 'report_timing -delay max' Default: -delay_type max -nworst 10 -max_paths 100 -input -net -trans -cap -nosplit -cross -path full -sig 5
Report Timing (Min Delay) (ReportMinTiming)	Runs the 'report_timing' command to report 'min' path delays. Default: 0
Report Timing (Min Delay) Options (ReportMinTimingOptions)	Additional options and switches to pass to 'report_timing -delay min' Default: -delay_type min -nworst 10 -max_paths 100 -input -net -trans -cap -nosplit -cross -path full -sig 5
Report Constraints (ReportConstraints)	Runs the 'report_constraints' command. Default: 1
Report Constraints Options (ReportConstraintsOptions)	Options to pass to the 'report_constraints' command. Default: -all_violators -max_delay -sig 5
Report Cross-domain Paths (ReportCrossDomainPaths)	Report paths which cross clock domains. Default: 0
Number of paths to report (NumCrossDomainPaths)	Maximum number of paths to report (from clock group to clock group) Default: 100
Report Format (CrossDomainReportFormat)	Format of cross-domain report. 'Summary' returns the start and end points of each path, 'short' reports show the cells at the start and end of each path, 'Full (detailed) paths' reports show each cross-domain path in its entirety. Default: Full (detailed) paths
Suppress Duplicate Bus Paths (SuppressDuplicateBusPaths)	Affects the reporting of cross-clock-domain paths. When enabled, duplicate paths on buses going between clock domains are not reported.

Table 6-8 Static Timing Analysis -- Select and Configure Reports (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
	Default: true
Run Bottleneck Analysis (ReportBottleneck)	Report timing bottlenecks. Default: 0
Bottleneck Analysis Options (ReportBottleneckOptions)	Specify the arguments to pass to the report_bottleneck command.
Run Clock Timing Report (ReportClockTiming)	Run the report_clock_timing report Default: 0
Run Minimum Pulse Width rpt. (ReportMinPulseWidth)	Run the report_min_pulse_width report Default: 0
Minimum Pulse Width Report Options (ReportMinPulseWidthOptions)	Arguments to pass to the report_min_pulse_width command.
Run clock-gating report (ReportClockGatingCheck)	Run the report_clock_gating_check report Default: 0
Clock Gating Report Options (ReportClockGatingCheckOptions)	Arguments to pass to the report_clock_gating_check report Default: -sig 5
Run Power Analysis (ReportPower)	Run Power Analysis using PrimeTime-PX Default: 0
Power Analysis Options (PowerAnalysisOptions)	Options to pass to the set_power_analysis_options command in PrimeTime-PX. This parameter is only supported in PrimeTime-PX 2008.12 or newer. Power Report Options (ReportPowerOptions) Options to be passed to the report_power command in PrimeTime-PX Default: -hierarchy
Report Switching Activity (ReportSwitchingActivity)	Run the report_switching_activity command to generate a report of annotated switching activity on the design. Default: 1
Switching Activity Report Opt. (ReportSwitchingActivityOptions)	Options and switches to pass to the report_switching_activity command.
Report Clock Gate Savings (ReportClockGateSavings)	Generate a set of reports using the report_clock_gate_savings report command. Default: 0 (false)

Table 6-8 Static Timing Analysis -- Select and Configure Reports (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Link To TestMax (LinkToATPG)	When TRUE, creates timing files needed by TestMAX for the TMAX Reference Methodology strategy. For more information, see Table 5-27 on page 137. Default: 1 (true)

Table 6-9 Static Timing Analysis -- Execution Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the script run.scr to be generated in the <workspace>/syn directory, but the script is not executed. You can then run the script at a later time. The path to the run.scr script is available using the "get_logical_dir syn/run.scr" command. Default: 0
Run Style (RunStyle)	If "Generate scripts only?" is not checked, this item selects how to run the script file, which can be run locally, through LSF, through GRD. Default: local
Run style options (LsfOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example '-q normal'). For qsub, you can specify a project or other options (for example '-P normal -l arch=glinux'). The -V option is included automatically for qsub. Default: (=get_activity_parameter Synthesize RunOptions)
Send e-mail / To: (EmailAddress)	If this parameter is set to a value other than null, then email is sent when the command script completes or is ended. Default: <current username>

6.3.1 Checking Static Timing Analysis Results

To complete the Static Timing Analysis activity in command line mode, use the `set_activity_parameter` and `autocomplete_activity` commands. For example:

```
% autocomplete_activity StaticTimingAnalysis
```

If you need to work directly with PrimeTime after running the activity, you can load the saved session directory `workspace/syn/stage/final/db/design_pt_session` into PrimeTime.

6.4 Explore RTL Architecture Activity

The Explore RTL Architecture activity incorporates the Synopsys RTL Architect™ tool. The RTL Architect tool enables early, physically aware RTL exploration to achieve a better starting point for RTL synthesis and better convergence during physical implementation.

In coreConsultant, you can use this activity to perform what-if analyses of various design configurations to speed the development of RTL and design constraints without waiting for feedback from the physical (back-end) design team. This activity produces numerous reports that show PPA and timing estimations.

For multiple configuration runs (FlowType = explore), comparison reports are included. For more information about the RTL Architecture tool, see the RTL Architecture product documentation.

Before running this activity, you must set paths to the following tools in **Edit > Tool Installation Roots**:

- PrimeTime (pt_shell)
- RTL Architect (rtl_shell)

The major settings for this activity are described in the following tables:

- [“Explore RTL Architecture -- Activity Controls”](#)
- [“Explore RTL Architecture -- Flat Flow Settings”](#) on page 159
- [“Explore RTL Architecture -- Explore Flow Settings”](#) on page 160
- [“Explore RTL Architecture -- Execution Options”](#) on page 161

[Table 6-10](#) describes the activity control settings for the Explore RTL Architecture activity.

Table 6-10 Explore RTL Architecture -- Activity Controls

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Flow Settings	
RTL Architect Flow Type (FlowType)	Selects the RTL Architect flow type for this block ■ flat: Run fast synthesis (optionally with auto floorplanning) and generate congestion, timing, and power metrics ■ explore: Run data exploration based on utilization, aspect ratio, and max routing layers Default: flat
RTL Architect Design Style (DesignStyle)	Sets the DESIGN_STYLE variable for this block ■ flat: Flat place and route flow ■ hier: Hierarchical place and route flow Default: flat
Insert DFT (InsertDft)	Enables design for test circuitry to be inserted based on the settings from the Specify Test Methodology activity Default: 0 (false)

Table 6-10 Explore RTL Architecture -- Activity Controls (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
DFT Flow Type (DFTFlowType)	Sets the RTL Architect DFT_FLOW control variable to in_compile or pre_compile Default: in_compile
Use Technology File (UseTechFile)	Use technology file to specify technology information. When enabled, the target library (TECH_LIB) setup is not used. Default: 0 (false)
Floorplan	
Floorplan Data Format (FloorplanFileDataFormat)	Format of file containing floorplan constraints ■ DEF: Design exchange format file Default: DEF
Physical Constraint File (PhysicalConstraintFile)	Path to the floorplan file; if unspecified, a default set of ■ A path to a TCL floorplan file populates the TCL_FLOORPLAN_FILE setup parameter ■ A path to a DEF floorplan file populates the DEF_FLOORPLAN_FILE setup parameter
Constraint Settings	
Options for write_sdc (WriteSDCArgs)	Defines arguments to be passed to the write_sdc command when writing SDC constraints for synthesis.
TLUPlus	
Early parasitic tech specification name (MinTluPlusName)	Specifies the early parasitic tech specification name Default: minTLUP
Min TLUPlus file name (MinTluPlusFile)	Specifies the TLUPlus files to read for Min specification
Min TLUPlus Layer map file (MinTluPlusLayerMapFile)	Path to the layer mapping file between ITF and Min tech specification
Late parasitic tech specification name (MaxTluPlusName)	Specifies the late parasitic tech specification name Default: maxTLUP
Max TLUPlus file name (MinTluPlusFile)	Specifies the TLUPlus files to read for Max specification

Table 6-10 Explore RTL Architecture -- Activity Controls (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
MAX TLUPlus Layer map file (MaxTluPlusLayerMapFile)	Layer mapping file between ITF and Max tech specification
UPF	
Enable UPF Flow (EnableUPFFlow)	When Enable UPF Flow is set to true (1), the UPF and power constraints files are used during synthesis. Default: 0 (false)
UPF Flow Type (UPFFlowType)	Use the default UPF flow Default: UPF
UPF File (UPFFile)	The name of the Unified Power Format (UPF) file to be used with the Fusion Compiler
Power Constraints File (PowerConstraintsFile)	Power constraints file that is sourced in Fusion Compiler after loading the UPFFile This file can be used to issue 'set_voltage' commands and remove ouch elements on library cells, as needed.
Utilization Ratio	
Utilization Ratio	Specifies the utilization of the core area. The utilization is the total area of the core occupied by all standard cells and macro cells divided by the total core area. You can specify a value between 0 and 1. For example, a core utilization of 0.8 specifies that 80 percent of the core area is used for cell placement at this stage. By default, the core utilization is 0.7. The value specified is used to update the control variable 'AUTO_FLOORPLAN_UTILIZATION' in the RTL Architect reference methodology scripts. Enabled when FlowType == flat Default: 0.7

Table 6-11 describes the flat flow settings for the Explore RTL Architecture activity.

Table 6-11 Explore RTL Architecture -- Flat Flow Settings

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Enable Congestion Metrics	Enable congestion metrics in flat flow Default: 1 (true)

Table 6-11 Explore RTL Architecture -- Flat Flow Settings (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
(EnableFlatFlowCongestionMetrics)	
Enable Timing Metrics (EnableFlatFlowTimingMetrics)	Enable timing metrics in flat flow Default: 1 (true)
Enable Power Metrics (EnableFlatFlowPowerMetrics)	Enable power metrics in flat flow Default: 0 (false)

Table 6-12 on page 160 describes the explore flow settings for the Explore RTL Architecture activity.

Table 6-12 Explore RTL Architecture -- Explore Flow Settings

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Utilization values (ExploreUtilizationList)	Set the EXPLORE_UTILIZATION_LIST values for the explore flow
Aspect ratio values (ExploreAspectRatioList)	Set the EXPLORE_ASPECT_RATIO_LIST values for the explore flow
Max routing layer value(s) (ExploreMaxRoutingLayerList)	Set the EXPLORE_MAX_ROUTING_LAYER_LIST/MAX_ROUTING_LAYER variables for the explore flow ■ Specifying more than one value sets the EXPLORE_MAX_ROUTING_LAYER_LIST variable in the rm scripts. ■ Specifying one value sets the MAX_ROUTING_LAYER variable in the rm scripts.
Min routing layer value (ExploreMinRoutingLayer)	Set the MIN_ROUTING_LAYER value for the explore flow
Explore Flow Run Style (ExploreFlowRunStyle)	Defines the protocol for launching worker processes on remote. The protocol corresponds to the type of compute farm manager you are submitting to. When RunStyle is 'local' then ExploreFlowRunStyle defaults to 'grd' otherwise ExploreFlowRunStyle is set to the RunStyle.
Explore Flow Grid Options (ExploreFlowGrdOptions)	Grid options for launching distributed jobs Distributed run is set to true for explore flow and false for flat flow. This attribute value is passed to the set_host_options command with the -submit_command argument. The set_host_options command is used when the 'DISTRIBUTED' variable is set to true in rtl_setup.tcl.

Table 6-13 describes the execution options for the Explore RTL Architecture activity.

Table 6-13 Explore RTL Architecture -- Execution Options

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Generate scripts only? (ScriptsOnly)	Setting this causes the script run.scr to be generated in the <workspace>/syn directory, but the script is not executed. You can then run the script at a later time. The path to the run.scr script is available using the “get_logical_dir syn/run.scr” command. Default: 0
Run style (RunStyle)	If “Generate scripts only?” is not checked, this item selects how to run the script file, which can be run locally, using LSF, GRD. Default: local
Run style options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example ‘-q normal’). For qsub, you can specify a project or other options (for example ‘-P normal -l arch=glinux’). The -V option is included automatically for qsub. Default: (=get_activity_parameter Synthesize RunOptions)
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified address when formal verification completes or is ended. The default for the text box labeled “To” is your (current user) user name.

6.5 Timing Constraints Manager

Constraint verification refers to the verification of the contents of an SDC/ TCL file to flag situations where the specified constraints are either incorrect or incomplete, both of which if not addressed could result in silicon failure.

Running lint checks on the RTL is recommended for identifying and resolving problems before simulation or synthesis. Similarly, linting SDC constraints before synthesis or STA helps to quickly isolate and correct issues avoiding unnecessary delays in timing closure constraints before they emerge in the time consuming implementation analysis. The best constraint verification flow is the one which has a high ability to detect issues with minimum noise, that is, issues that upon the user's review, results in no change to the design or constraints.

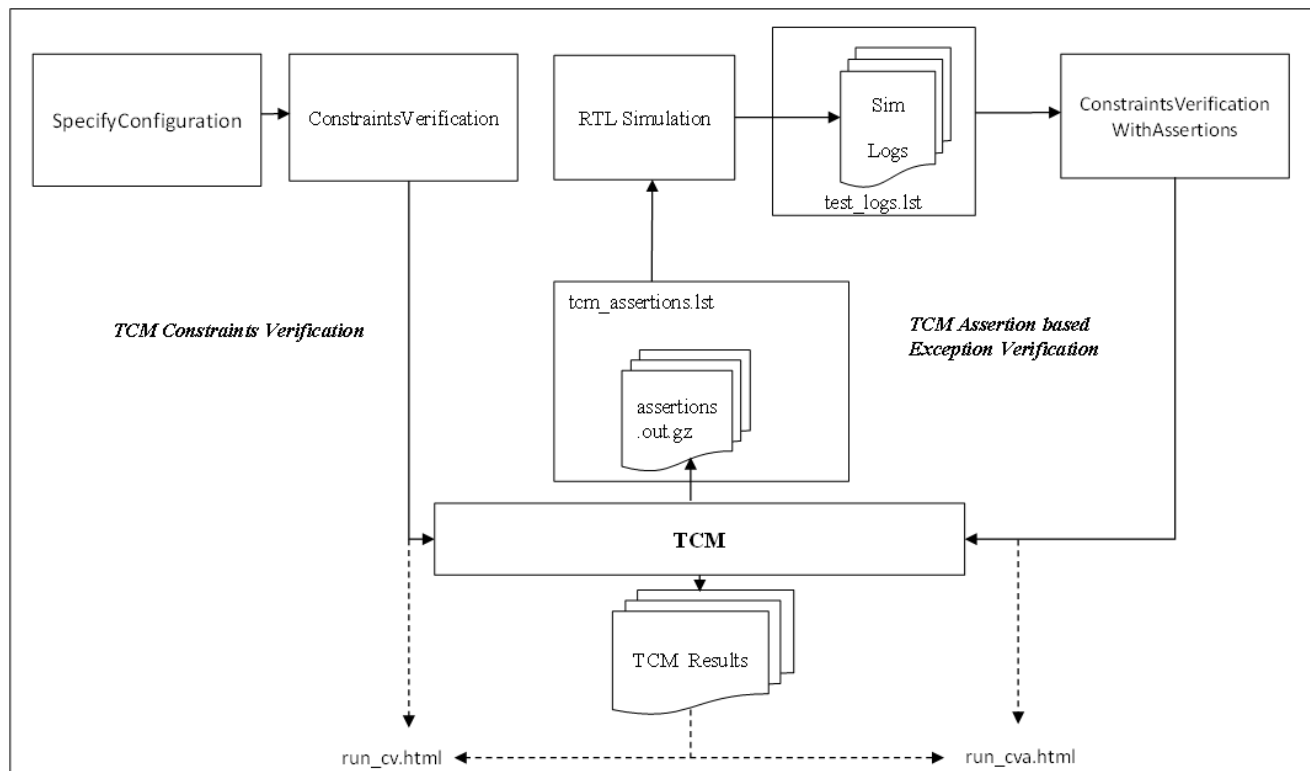
The Timing Constraint Manager (TCM) is an effective tool while performing the constraint verification. It enables the user in applying minimal effort due to minimal noise and detects the constraint syntax, mapping, and verification issues efficiently. The tool performs lint checks to verify a certain group of constraints, whereas the timing exception related constraints are checked formally, followed by failed paths with assertion-based verification. If the constraints still fail post running it through the TCM flow, they will need to be reviewed.

The TCM flow is performed with the following two activities:

- “[Constraints Verification \(CV\) Activity](#)” on page 164
- “[Constraints Verification with Assertions \(CVA\) Activity](#)” on page 166

Figure 6-2 shows the TCM flow diagram.

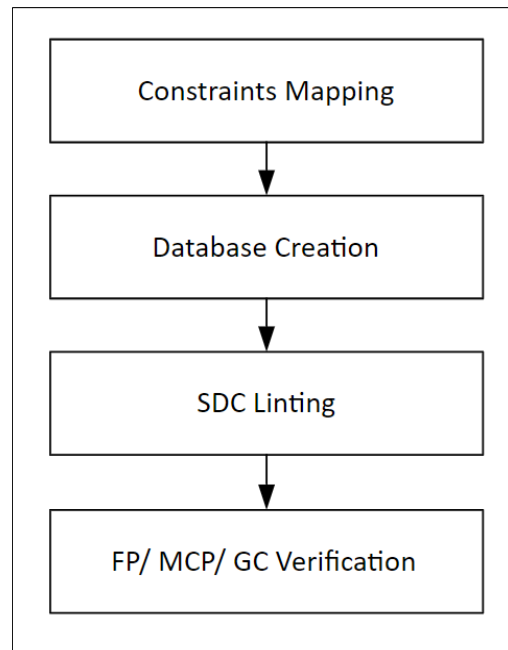
Figure 6-2 TCM Flow Diagram



6.5.1 Constraints Verification

The TCM flow supports the stages as shown in [Figure 6-3](#) to perform the constraints verification.

Figure 6-3 Constraint Verification Stages



6.5.1.1 Constraints Mapping

This stage reads the RTL/ constraints and applies the constraints to the design. The constraint syntax/ semantic and RTL parsing errors are also reported.

The stage has the ability to auto handle the synthesis transformation such as “_” to “/” conversion. For example, the netlist hierarchy a_b_c can be mapped to a/b/c.

At the mapping stage, the main files are shown in [Figure 6-4](#) which can be found at `<workspace>/constraintsverification/tcm_rtl_verification`.

Figure 6-4 Constraint Mapping Stage Files

```

- setup
  | files.f
  | interactive.tcl
  | map_sdc_and_check_setup
  | map_sdc.tcl
  | refocus_interactive.tcl
  | setup.tcl
  | waive.sdc
  | waive.tcl
  
```

6.5.1.2 Database Creation

In this stage, the database model is created. The design.fdb is the model that can be reused if the design is unchanged in the future runs. Reusing the design.fdb file also helps in reducing the runtime. [Figure 6-5](#) on page 164 lists the database creation stage files.

Figure 6-5 Database Creation Stage Files

```

-- ver_db
|
| create_ver_db
| create_ver_db.tcl

```

6.5.1.3 SDC Linting

The linting stage performs the formal SDC linting. The SDC linting stage files are as shown in [Figure 6-6](#).

Figure 6-6 SDC Linting Stage Files

```

-- setup
| files.f
| interactive.tcl
| map_sdc_and_check_setup
| map_sdc.tcl
| refocus_interactive.tcl
| setup.tcl
| waive.sdc
| waive.tcl

```

6.5.1.4 FP/ MCP/ GC Verification

The FP/ MCP/ GC verification run validates the timing exceptions and generated clocks. If there are assertion failures, they are written in the same directories with the file name 'assertions.out.gz'. [Figure 6-7](#) shows the FP/ MCP/ GC verification stage files.

Figure 6-7 FP/ MCP/ GC Verification Stage Files

```

-- fp_verification
| generate_html
| generate_html.tcl
| incremental.tcl
| interactive.tcl
| run_confirm.tcl
| verify_fp
-- gc_verification
| generate_html
| generate_html.tcl
| incremental.tcl
| interactive.tcl
| run_confirm.tcl
| verify_gc
-- generate_all
-- generate_summary
-- mcp_verification
| generate_html
| generate_html.tcl
| incremental.tcl
| interactive.tcl
| run_confirm.tcl
| verify_mcp

```

6.5.2 Constraints Verification (CV) Activity

In coreConsultant, the TCM Constraints Verification is run with the set of activity parameters as shown in [Table 6-14](#) on page 165.

**Note**

This flow is supported only for the Synopsys IPs and the IPs must be TCM enabled.

Table 6-14 Constraints Verification Activity Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Setup	
Verify All (VerifyAll)	When VerifyAll is selected, other parameters are disabled. VerifyAll runs all the constraint verification activities.
Verify False Paths (VerifyFalsePath)	Performs False Path verification
Verify Generated Clocks (VerifyGeneratedClocks)	Performs Generated Clocks verification
Verify Multi Cycle Path (VerifyMultiCyclePath)	Performs Multicycle Path verification
Execution Options	
Generate Scripts Only (ScriptsOnly)	Setting this causes the run_cv.scr script to be generated in the work directory, but the script does not get executed. Users can then run the script at a later time. The path to the run_cv.scr script can be obtained by using the following command: "get_logical_dir syn/run_cv.scr" Default: 0
Run Style (RunStyle)	If "Generate scripts only" is unchecked, then this selects how to run the command, which can be run locally, through LSF, through GRD.
Run Options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example '-q normal'). For qsub, you can specify a project or other options (for example '-P normal -l arch=linux'). The -V option is included automatically for qsub.
Parallel Job CPU Limit (ParallelJobCpuLimit)	Specifies the maximum number of CPUs to be allowed in TCM. It is set on the TCM variable max_parallel_processes which determines the maximum number of processes that are allowed to run the verification. If you select a value greater than 1, TCM will split up the exceptions to be verified into separate groups, depending on the value of max_parallel_processes and is verified on a separate processor.
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when synthesis completes or is ended. The default for the text box labeled "To" is your (the current user's) user name.

The input files for the Constraints Verification (CV) activity are as follows:

- Packaged waiver file (waivers.tcl)

This file is provided as the read-only global waivers.tcl file that remains unmodified.

- External waiver file (waivers.external.tcl)

The TCM process writes the data to the external waiver TCL file (waivers.external.tcl) during the interactive mode that is enabled by running the `view_fishtail_results` command in one of the verification folders (setup, fp_verification, mcp_verification, or gc_verification), and after saving any changes (added waiver).

6.5.3 Constraints Verification with Assertions (CVA) Activity

Assertions that are created if there are unverified exceptions after running the “[Constraints Verification \(CV\) Activity](#)” on page 164 need to be validated during the simulation. Post simulation, you can extract the SDC status in the Constraint Verification with Assertions (CVA) activity.

[Table 6-15](#) shows the Constraints Verification with Assertions activity parameters.

Table 6-15 Constraints Verification with Assertions Activity Parameters

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Setup	
Simulation Log List File (SimulationLogs)	File containing a list of the simulation log files. Supports both absolute and relative path. Relative path must be calculated from the logical directory.
Pass Static (PassStatic)	The -pass_static option reports inconclusive (vacuous) paths as Pass Static instead of Inconclusive. Default: 0
Remove Feedback Paths (Remove Feedback Paths)	The -remove_feedback_paths option does not report paths where the start point and end point are the same design object. Default: 1
Execution Options	
Generate Scripts Only (ScriptsOnly)	Setting this causes the run_cva.scr script to be generated in the work directory, but the script does not get executed. Users can then run the script at a later time. The path to the run_cva.scr script can be obtained by using the following command: “get_logical_dir syn/run_cva.scr” Default: 0
Run Style (RunStyle)	If “Generate scripts only” is unchecked, then this selects how to run the command, which can be run locally, through LSF, through GRD.

Table 6-15 Constraints Verification with Assertions Activity Parameters (continued)

Activity Parameter <GUI Label> (<Parameter Name>)	Description
Run Options (RunOptions)	Additional options can be provided for a given run style. For LSF and GRD, this specifies options to the bsub and qsub commands, respectively (for example '-q normal'). For qsub, you can specify a project or other options (for example '-P normal -l arch=linux'). The -V option is included automatically for qsub.
Send e-mail (EmailAddress)	If selected, this option sends an e-mail message to the specified user when synthesis completes or is ended. The default for the text box labeled "To" is your (the current user's) user name.

6.5.4 Running the TCM Flow

For the Synopsys TCM supported Synopsys IP's, the following are the steps to be implemented to run the TCM flow:

1. The TCM is to be loaded by setting the tool root path as follows:

```
set_tool_root tcm -root <PATH to TCM tool root or FISHTAIL_HOME>
```
2. Perform and complete the Specify Configuration activity.
3. Perform the verification by selecting any of the goals listed in [Constraints Verification Activity Parameters](#) on page 165 in the “[Constraints Verification \(CV\) Activity](#)” on page 164.
4. View the results (run_cv.html) under the reports tab post completion of the “[Constraints Verification \(CV\) Activity](#)” on page 164.
5. A tcm_assertions.lst list file is generated if there are unverified exceptions. The following is the path for the list file:

```
<workspace>/constraintsverification/tcm_assertions.lst
```
6. Perform the Simulation activity (See “[Setup and Run Simulations](#)” on page 169) after enabling the EnableTCM parameter. The simulation log list file will be provided to the SimulationLogs activity parameter of the “[Constraints Verification with Assertions \(CVA\) Activity](#)” on page 166 when the simulation is completed.
7. Perform and complete the “[Constraints Verification with Assertions \(CVA\) Activity](#)” on page 166.
8. View the results (run_cva.html) under the reports tab post completion of the “[Constraints Verification with Assertions \(CVA\) Activity](#)” on page 166.

Note

- The prerequisite for running the Timing Constraint Manager is that the minimum version for TCM must be U-2022.12-SP5.
- Once the “[Constraints Verification with Assertions \(CVA\) Activity](#)” on page 166 is completed, the run_cv.html file becomes outdated and is updated with a message. See the run_cva.html for the TCM verification results.

6.5.5 Constraints Verification Directory Tree Structure

Figure 6-8 shows the list of the main files generated by coreConsultant for the TCM flow. Additionally, the TCM generates its own set of files in the `<workspace>/tcm/tcm_rtl_verification` folder.

Figure 6-8 Constraints Verification Directory Tree Structure

```
.
├── constraints
│   └── scripts
│       └── <design>.tcl
├── run_cv.bash
├── run_cv.scr
├── scripts
│   ├── constraintsverification_mod.bash
│   └── report_constraintsverification.pl
└── tcm
    ├── rtl_verification.cfg
    ├── waivers.external.tcl
    └── waivers.tcl
```

6.6 Setup and Run Simulations

When you select **Verify Component > Setup and Run Simulations** from the Activity List, the corresponding Activity View appears. For non-Synopsys cores, this activity may look substantially different than it does for Synopsys cores, or it may not be available. In this Activity View you select the simulator, the view to be simulated, various execution options, and testbench parameters.



Note

The **Edit > Tool Installation Roots...** menu item and dialog box is where you can set or change the paths to your simulators. The initial value of these paths comes from the simulator environment variables. See your simulator documentation for required environment variables.

6.7 Integrating the Core Into Your Design

After completing the coreConsultant design flow, you have a customized implementation of the reusable core that has been optimized to meet your design objectives. You must now integrate the core into your application design and verify the core in the context of your application, according to your verification strategy.

6.7.1 Connecting to Surrounding Logic

See the core-specific documentation provided by the core developer for descriptions of the core functionality and the behavior and timing of its ports. The core-specific documentation also includes examples of how to instantiate the block. Connect the I/O ports of the core to your application design logic according to your chip-level design specifications.

You can use the coreAssembler tool to assemble your configured components into a subsystem design if these components have the pre-defined AXI, AHB, or APB interfaces. For information on using coreAssembler, see the *coreAssembler User Guide*.

6.7.2 Performing System-Level Synthesis

When you are ready to perform system-level synthesis on components configured by coreConsultant, you may need to have the top-level Synopsys Design Constraints (sdc) available in a constraint file for the configured core. This file is available from `export/design_name.sdc`. See the “Synthesis_API” man page for details.

6.7.3 Verifying Functionality

In most design flows, you will already have verified the functionality of the RTL version of the core. To verify the functionality of the optimized gate-level version, use any or all of the following methods:

- Formal verification (for example, Synopsys Formality) to verify the gate-level implementation against the already-verified RTL version. Formal verification checks that the two versions of the design are functionally identical. (See “[Formal Verification](#)” on page 144)
- Simulation of the gate-level version of the design in the verification environment supplied by the core developer.
- Simulation of the gate-level version of the design in the context of your application design according to your design verification strategy.

7

Advanced User Modes and Features

The preceding chapters explain how to implement a reusable core using the GUI mode.

This chapter contains the following sections:

- [“Command Line Interface”](#) on page 172
- [“Batch Mode”](#) on page 175
- [“Editing Tool Setup and User Preferences”](#) on page 177
- [“Working with Verification IP”](#) on page 190
- [“Working with Hard Macros”](#) on page 191
- [“Dumping Debug Information”](#) on page 192
- [“Documentation Generation”](#) on page 193
- [“Table-Driven Data Import/Export”](#) on page 194

7.1 Command Line Interface

coreConsultant provides a command-line interface so that you can enter commands directly from the command line, or write the commands to a file for batch-mode execution.

7.1.1 Accessing the Command Line Interface

You can invoke the command line interface in two ways:

- Enter commands at the coreConsultant> prompt in the coreConsultant console.
- Invoke coreConsultant in shell mode by entering the following command at the UNIX prompt:

```
% <installation_dir>/bin/coreConsultant -shell
```

The following sections show how to use the command line interface to perform a variety of activity-related tasks. See the coreConsultant online command reference for complete descriptions of all of the available commands.

7.1.2 Setting the Value of a Design Configuration Parameter

To set the value of a top-level design configuration parameter, use the `set_configuration_parameter` command:

```
set_configuration_parameter param value
```

For example, to set the value of top-level design parameter `extd_intr` to 1, enter the following command:

```
coreConsultant> set_configuration_parameter extd_intr 1
```

The above command automatically sets the state of the Specify Configuration activity to incomplete. To complete the Specify Configuration activity, use the `autocomplete_activity` command:

```
coreConsultant> autocomplete_activity SpecifyConfiguration
```

coreConsultant writes out the configuration-dependent files for your new design configuration and enables the activities that depend on Specify Configuration.

To get the current value of a top-level design configuration parameter, use the `get_configuration_parameter` command:

```
coreConsultant> get_configuration_parameter extd_intr 1
coreConsultant>
```

7.1.3 Reporting Activity Parameters

When you select values for activity parameters through the coreConsultant GUI, coreConsultant applies the information as a parameter associated with the activity. For example, when you select `DCTCL_opto_strategy` as your synthesis strategy, coreConsultant applies the value `DCTCL_opto_strategy` to the `StrategyName` parameter on the **Perform ASIC Synthesis** activity.

To list the names and brief descriptions of the parameters associated with an activity, use the `report_activity_parameters` command:

```
report_activity_parameters ActivityName
```

To get the current value of an activity parameter, use the `get_activity_parameter` command:

```
get_activity_parameter activity_name parameter_name
```

To get all parameters, use the command:

```
find_item -type param blob_name
```

7.1.4 Setting the Value of an Activity Parameter

To set the value of a parameter on a coreConsultant activity, use the `set_activity_parameter` command:

```
set_activity_parameter activity_name parameter_name parameter_value
```

For example, to select DCTCL_opto_strategy as the synthesis strategy:

```
coreConsultant> set_activity_parameter Synthesize StrategyName  
DCTCL_opto_strategy
```

When you use `set_activity_parameter` to set the value of an activity parameter, coreConsultant automatically changes the status of the associated activity to incomplete.

7.1.5 Setting the Value of a Synthesis Strategy Parameter

To set the value of a synthesis strategy parameter, use the `set_strategy_parameter` command:

```
set_strategy_parameter strategy_name parameter_name parameter_value
```

For example, to set the DCTCL_opto_strategy QorEffort parameter to high:

```
coreConsultant> set_strategy_parameter DCTCL_opto_strategy QorEffort high
```

When you use `set_strategy_parameter` to set the value of a synthesis strategy parameter, coreConsultant automatically sets the status of the Perform ASIC Synthesis activity to incomplete.

To get the current value of a synthesis strategy parameter, use the `get_strategy_parameter` command:

```
get_strategy_parameter strategy_name parameter_name
```

7.1.6 Re-Executing the coreConsultant Design Flow

When you modify one or more activity parameters, you need to re-execute the coreConsultant design flow to implement those changes. The command below automatically completes various sets of activities, depending on the options specified, and [Table 7-1](#) describes the available options:

```
autocomplete_activity activityName [-enable | -recur]
```

Table 7-1 Options for autocomplete_activity

Option	Description
none	Automatically complete all pre-requisite activities, then complete the specified activity. (Default)
-enable	Enable, but do not complete, the specified activity by automatically completing all pre-requisite activities.
-recur	Automatically complete the specified activity and all activities that depend on the specified activity.

For example, if you change specifications for the Perform ASIC Synthesis activity, use `autocomplete_activity` to re-execute Synthesize automatically, this time using the new specifications:

```
coreConsultant> autocomplete_activity Synthesize
```

As previously indicated, using the `set_activity_parameter` to set the value of an activity automatically sets the state of that activity to incomplete. However, changes that you make by executing `set_clock_attribute`, `set_design_attribute`, and `set_port_attribute` do not cause any changes to the current state of coreConsultant activities.

7.1.7 Abbreviating Commands

You can abbreviate coreConsultant commands and options to the shortest unambiguous string. For example, you can abbreviate the `set_port_attribute` command to “`set_por.`” Command “tab-completion” is also available.

You cannot abbreviate built-in Tcl commands.

Do not use command abbreviation in batch files. Also, remember that existing command abbreviations may become ambiguous if and when future versions of coreConsultant add to or change the list of commands.

For a list of all commands that match the ambiguous abbreviation, use `help` with wildcard pattern matching:

```
coreConsultant> help cur*
current_design # Set/get the current design
current_instance # Set/get the instance
current_kb # Make the named knowledge base the current one.
coreConsultant>
```

7.2 Batch Mode

You can execute coreConsultant commands in batch mode by writing the required sequence of commands to a file using Tcl syntax. coreConsultant supports both standard Tcl commands and coreConsultant-specific Tcl commands. Batch mode execution is useful for:

- Regression testing
- Experimentation with different design alternatives

7.2.1 Batch Command File Examples

A common use for batch mode in coreConsultant is to change values for one or more parameters for an activity, then re-execute that and all subsequent activities. This scenario assumes that you have already used the GUI to complete the design flow at least once, but this is not a strict requirement.

The following example changes a configuration parameter on a core, then re-executes configuration and all subsequent activities up to and including Synthesize, using the currently stored activity parameter values.

```
#Change parameter value
set_configuration_parameter extd_intr 1
#Re-execute configuration and subsequent activities
autocomplete_activity Synthesize
#Quit coreConsultant
quit
```

The next example changes a parameter for the Perform ASIC Synthesis activity, then re-executes the Perform ASIC Synthesis activity.

```
#Change activity specifications
set_activity_parameter Synthesize StrategyName DCTCL_opto_strategy
#Re-execute this activity and all subsequent activities
autocomplete_activity Synthesize
#Quit coreConsultant
quit
```

See the coreConsultant online reference for details about the commands used in the above examples.

7.2.2 Automatically Generating a Batch File

coreConsultant provides a utility to automatically write a batch script file that contains the commands required to re-create the current state of your workspace. You can then use the resulting batch script in any of the following ways:

- As an example of how to write a batch script
- To create another identical workspace in batch mode
- To modify the batch file and create a new workspace with different options

To automatically generate a batch script in command line mode, use the `write_batch_script` command. For example:

```
coreConsultant> write_batch_script my_script.tcl
```

See the online command reference for the complete set of `write_batch_script` options.

To automatically generate a batch script in GUI mode, select **File > Write Batch Script** in the coreConsultant menu bar.

7.2.3 Batch File Options

The script generator has all interfaces connected explicitly, and not rely on automatic interface connection. In addition, the following list describes the `write_batch_script` options and conditions:

- When generating a batch script through the GUI, checking is first performed to prevent overwriting an existing batch file.
- If the workspace already exists, you can answer “Yes” to overwrite the existing workspace, or provide a “path” where to create the new workspace.
- Selecting the “Include Design Constraints” option writes constraints into the batch file. Otherwise, these constraints are not written.

7.2.4 Executing a Batch Command File

To invoke coreConsultant and automatically execute a batch command file, specify your workspace name and use the `-f` option to specify the name of the batch command file. For example:

```
% coreConsultant -shell workspace_name -f batch.tcl
```

The example shown above performs the following sequence:

1. Starts coreConsultant in shell mode.
2. Loads your workspace (*workspace_name*).
3. Executes the batch mode command file `batch.tcl`.

You can also execute a batch command file from the command line interface by using the Tcl source command, for example:

```
coreConsultant> source batch.tcl
```

7.2.5 Batch Mode coreKit Installation

If the installation procedure for your core does not require a core-specific license, you can install the coreKit in batch mode by using the coreConsultant `batch_install` command:

```
batch_install coreKit install_dir
```

coreKit is the name of the coreKit file (typically `name.coreKit`). *install_dir* is the target directory into which you want to install the coreKit.

For example, to invoke coreConsultant in shell mode and automatically install a coreKit in batch mode, execute the following command:

```
% coreConsultant -shell -x "batch_install coreKit install_dir; quit"
```

This example performs the following sequence:

1. Starts coreConsultant in shell mode.
2. Installs the specified coreKit into *install_dir*.
3. Exits coreConsultant.

7.3 Editing Tool Setup and User Preferences

You can control certain tool aspects that are set automatically when the tool opens. This is done using:

- “Setup Variables”
- “User Preferences” on page 178

7.3.1 Setup Variables

To establish a custom environment, you can set tool variables and define tcl commands in .synopsys_rt.setup files. When launched, coreConsultant looks for three .synopsys_rt.setup files in the following order:

1. *Site-wide* coreConsultant variable settings in:

`<install_dir>/admin/setup/.synopsys_rt.setup`

You can view the contents of this file to see the variables that are most commonly set.

2. *Personal* settings in:

`$HOME/.synopsys_rt.setup`

For a short sample of a personal setup file, see `<install_dir>/admin/setup/user.synopsys_rt.setup`.

3. *Project-specific* settings can be placed in a .synopsys_rt.setup that resides in the directory where you invoked the coreConsultant tool.

When you invoke coreConsultant, it reads and executes the setup files before executing any command line arguments.

The variable settings and tcl command definitions in the most recently read file take precedence over those in previously read files. In other words, the setup file in your \$HOME directory overrides the tool setup in `<install_dir>/admin/setup/.synopsys_rt.setup`, and a project-specific setup file overrides all others.

To define useful tcl commands in the setup file, use the ‘proc’ statement. Several examples are as follows:

```
proc fpa { args } { return [eval find_item -type param $args] }
proc fpi { args } { return [eval find_item -type pin $args] }
proc fn { args } { return [eval find_item -type net $args] }
proc fc { args } { return [eval find_item -type cell $args] }
```



Note

All coreTools applications use the same setup file.

Table 7-2 describes some of the variables that you can set in .synopsys_rt.setup files. For other variables you can set, see the `<install_dir>/admin/setup/.synopsys_rt.setup` file.

Table 7-2 Setup Variables

Variable	Description	Default
component_search_path	List of directories that contain installed coreKits; used to determine available	[getenv\DESIGNWARE_\HOME]/iip/latest

Table 7-2 Setup Variables (continued)

Variable	Description	Default
	components for coreConsultant.	
search_path	The list of directories to be searched when coreConsultant looks for workspace-specific files.	"\$synopsys_root/libraries/dware/rt."
scratch_dir	The directory used as a staging area during installation of a coreKit and for the storage of temporary files.	/tmp
collection_print_item_limit	The maximum number of item names to print as part of a command result.	10
default_browser_command	Allows specifying the HTML browser used. Note: PDF files will open from either a direct \$PATH to acroread, or as a plugin to your default browser.	firefox / mozilla / netscape
netscape (Path)	If you use Netscape, specify the path to your installed version of Netscape.	"netscape"
netscape (LaunchArgs)	Arguments passed to Netscape when coreConsultant invokes Netscape with show_url_netscape.	

7.3.2 User Preferences

The coreConsultant tool reads a set of tool interface preferences (if they exist) when it opens in either GUI or command-line mode. When set, the preferences reside in the following file:

\$HOME/.synopsys_rt_prefs.tcl

All preferences are user-specific and are used (as appropriate) for each coreTools application you run.

To set your preferences, choose **Edit > Preferences** from the toolbar menu. Note that the settings are saved when coreConsultant is closed.

**Note**

All coreTools applications use the same preferences file.

Table 7-3 describes the available preferences.

Table 7-3 Edit > Preferences Menu

Preference	Description
Appearance > Global > Fonts	
GUI: Default font Name: app_default_font_family Default: Bitstream Vera Serif, 10	Specifies display font family and size used for most application text at startup. This font must be proportional.
GUI: Console font Name: app_fixed_width_font_family Default: Courier, 10	Font family and size used for console text.
Appearance > Global > Spreadsheet	
GUI: Show spreadsheet Boolean values as a checkbox Name: show_spreadsheet_bool_as_checkboxes Default: 1 (true)	Show spreadsheet Boolean values as a check box when true; otherwise, display as a combination box.
GUI: Show all spreadsheet columns Name: show_all_grid_columns Default: 0 (false)	Show all attributes in spreadsheets, not just the most common set.
Appearance > External Browser	
GUI: External browser launch command Name: default_browser_command Default: firefox %s	The default browser invocation command string replaces %s with the URL.
GUI: External PDF viewer: Name: default_pdf_viewer Default: acroread	The default viewer for PDF files. The PDF file name is appended to whatever string is specified for the viewer.
Appearance > Windows	
Dialog Window GUI: Show the main application dialog tabwindow on tool startup Name: show_tabwindow_on_guistart Default: 1 (true)	Show the main application dialog tabwindow on tool startup.
Console Window	Relative height of console at tool startup.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
GUI: Initial console height (in percent) Name: initial_console_height Values: 15, ..., 50 Default: 19	This value is a percentage of window space that the console should occupy. The value 0 means do not show the console, which can then be shown using View > Console Window.
Activity Window GUI: Initial activity window width (in percent) Name: initial_activity_window_width Values: 10, ..., 50 Default: 25	Relative width of activity window at tool startup. This value is a percentage of window space that the activity window should occupy. The value 0 means do not show the activity window, which can then be shown using View > Activity Window.
Synopsys IP Updates (see also, “Component Update Checking” on page 40)	
GUI: Enable automatic Synopsys IP update checks Name: allow_automatic_update_checks Default: 1 (true)	Enable automatic update checks for used Synopsys IP. If disabled, you can perform a manual IP update check anytime using the Help > Check For IP Updates menu item.
GUI: Synopsys IP update check frequency in days Name: automatic_update_check_frequency Default: -1	The frequency to check for available updates of the tool and the Synopsys IP being used. The default setting of -1 causes coreConsultant to automatically prompt you to set this preference when you open a workspace that contains Synopsys IP.
GUI: Inform Synopsys about opting out of automatic IP update checks Name: inform_optingout_automatic_update_checks Default: 1 (true)	Inform Synopsys about opting out of automatic IP update checks.
Workspace Options (see also, “Setting Workspace Options” on page 43)	
GUI: Include synthesis related activities Name: ws_options_synthesis Default: 1 (true)	Include activities in the activity list which relate to the generation of a gate-level implementation of the component or subsystem.
GUI: Write batch script after each assembly related activity Name: ws_options_batch Default: 0 (false)	Write a batch script for recreating the subsystem as each activity in the 'Create RTL' group completes. The batch script is written to <workspace>/export/batch.tcl.
GUI: Include component specific unit-level test activities Name: ws_options_unittest Default: 1 (true)	Include activities associated with unit-level testing of DesignWare implementation IP. This includes GTECH model generation and unit-level simulation. Note: This option applies only in coreAssembler.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
GUI: Show “Memory Map Specification” activity Name: ws_options_memorymapspecification Default: 1 (true)	Shows the Memory Map Specification activity and its associated dialog in the workspace activity flow. Note: This option applies only in coreAssembler.
Document Generation (see also, “Documentation Generation” on page 193)	
GUI: Generated Documentation Format Name: documentation_format Default: DocBook	Generate reports/documentation in the specified format: ■ DocBook: User configurable reports generated in DocBook XML and then automatically translated to HTML. ■ HTML: Generate non-configurable reports directly in HTML format.
GUI: Compress Documentation Name: docbook_compress_documentation Default: 1 (true)	When set to true, items that are grouped with DocGroup will not be reported individually. For more about DocGroup and DocMaster, see the man pages.
GUI: Separate Files Per Parameter Group Name: docbook_file_per_param_group Default: 0 (false)	■ 0 (false): All parameters are written to a single table. ■ 1 (true): Each page from the configuration dialog has a separate table, and each table is written to a different file.
GUI: Separate Files Per Address Block Name: docbook_separate_address_block_files Default: 1 (true)	■ 0 (false): All register and register field information for an address block is written to a single file. ■ 1 (true): Information for each register is written to a file and register field information is written to a separate file.
GUI: Separate Files Per Port Group Name: docbook_separate_file_per_port_group Default: 1 (true)	■ 0 (false): All ports are written to a single file. ■ 1 (true): Each port group is written to a separate file.
GUI: Port Documentation Attributes Name: docbook_port_attr Default: PortDirection *DocGenerateIf *DocSynchronousTo *DocRegisteredBy *DocPowerDomain *DocDescriptionField *DocDescription	Indicates which attributes appear in DocBook documentation generated for top-level ports (signals) of a component. Attributes whose names are preceded by ‘*’ appear in a single column in the generated table. Attributes beginning with ‘Doc’ are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. If the DocGenerateIf attribute is included, it is excluded from reports generated for configured designs.
GUI: Parameter Documentation Attributes Name: doc_param_attr	Indicates which attributes appear in DocBook documentation generated for top-level parameters of a component. Attributes whose names are preceded by ‘*’ appear in a single column in the generated table. Attributes beginning with ‘Doc’ are replaced with the

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Default: Value *Name *Values *DocDefaultValue *DocEnabled *DocDescriptionField *DocDescription	corresponding non-Doc version if the Doc* version of the attribute is unset. If the Value attribute is included, it is excluded from reports generated for unconfigured designs. Note: The name attribute is included by default because the first column of the parameter tables uses the GUI label instead of the parameter name.
GUI: Register Documentation Attributes Name: docbook_register_attrs Default: DocGlobalAddressOffset *DocShortDescription	Indicates which attributes appear in DocBook documentation generated for registers within a memory map of a component. Attributes whose names are preceded by '*' appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset.
GUI: Register Field Documentation Attributes Name: docbook_field_attra Default: Name DocMemoryAccess *DisplayName *Reserved *DocRegisterResetValue *DocTestable *DocRegisterResetMask *DocVolatileMemory *WriteConstraint *MinWriteConstraint *MaxWriteConstraint *DocDescription *DocDescriptionField *BitRangeVariables *MemoryAccessDescription *ReadActionDescription *WriteBehaviorDescription	Indicates which attributes appear in DocBook documentation generated for fields within a register. Attributes whose names are preceded by '*' all appear in a single column in the generated table. Attributes beginning with 'Doc' are replaced with the corresponding non-Doc version if the Doc* version of the attribute is unset. Note: The name attribute is included by default because the first column of the field tables uses the bit range as a descriptor.
Formal Verification	
GUI: Target stage netlist Name: formver_use_target_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseTargetStageNetlist.
GUI: Reference stage netlist Name: formver_use_reference_stage_netlist Values: Use db (false), Use netlist (true) Default: Use db	Sets the default value for formal verification activity parameter UseReferenceStageNetlist.
Tech Setup	
GUI: Run DC to init variables Name: run_dc_to_init_variables Default: 0 (false)	Forces coreTools to run Design Compiler to determine the values of synthesis setup variables.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Assembly	
GUI: Default Port Name Format (via exported interface) Name: auto_port_name_format Default: %e_%p	The format to be used for generating port names for exported interfaces. Port name format usage is as follows: ■ %c: Component instance name ■ %p: Interface instance port name ■ %e: Exported interface name ■ %i: Interface instance name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Default Port Name Format (via manual export) Name: manual_port_name_format Default: %c_%p	The format to be used for generating port names for manually exported ports. Port name format usage is as follows: ■ %c: Component instance name ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Default Signal Name Format Name: signal_name_format Default: %c_%p	The format to be used for generating signal names when writing RTL. Signal name format usage is as follows: ■ %c: Component instance name ■ %l: Use load component name instead of driver (single load only) ■ %p: Component port name ■ %d: Component port direction (i, o, b) ■ %D: Component port direction (I, O, B)
GUI: Automatically apply when leaving a dialog or page Name: auto_apply Default: 0 (false)	Controls how the tool handles changes to dialog boxes or pages that have not been applied. ■ 1 (true): Automatically apply changes in the dialog or page ■ 0 (false): Query the user whether to apply or cancel the changes
GUI: Allow incomplete subsystem Name: allow_incomplete_subsystem Default: 0 (false)	Controls activity completion and RTL generation for an incomplete subsystem.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
GUI: Allow undriven pins (see help) Name: allow_undriven_pins Default: 0 (false)	When true, errors related to undriven pins are downgraded to warnings. Note: The resulting netlist will not include these connections because they are effectively undriven, so not including the connections is functionally equivalent to including an undriven wire.
GUI: Remove Unneeded Ports Name: remove_unneeded_ports Default: 0 (false)	When true, ports created as part of the interface-based connection process that are not needed, are removed. Ports are not needed in the following scenario: When an interface is exported out of a lower-level of hierarchy and the connection on one side or the other is missing. This results in exported ports that are either 'open' or connected to a constant (if the interface definition provided a default value). If connected to open, the ports will be removed. If connected to a constant, the port is removed and the constant is connected to any required pins in the lower-level hierarchy.
GUI: Use Merged Glue Logic Source Name: use_merged_glue_logic_source Default: 0 (false)	When true, the generated file, <subsystem_name>_sim.lst, will not reference individual Synopsys glue logic component source. The glue logic will be compiled from the shared glue logic source in <coreTools image root>/auxx/dware/glue.
GUI: Don't Auto-connect Interfaces Name: disable_autoconnect Default: 0 (false)	When true, interfaces will not auto-connect in GUI mode.
Process Execution	
GUI: Run style preference Name: run_style_preference Values: local, lsf, grd Default: local	Controls how to run external jobs. Jobs can be run locally through LSF, or through GRD.
GUI: Run style options Name: run_style_options Default: N/A	Specifies additional options for a given run style (run_style_preference). ■ For lsf and grd, this specifies options for the bsub and qsub commands.
Synthesis	
GUI: Bus naming style Name: bus_naming_style Default: N/A	Sets the bus_naming_style variable for synthesis.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
GUI: Show rise & fall delays separately Name: synth_show_rise_and_fall_delays Default: 0 (false)	■ 1 (true): Separate columns are shown for rise and fall delay constraints. ■ 0 (false): Constraints are shown merged with minimum value shown for minimum delays and the maximum value shown for maximum delays.
GUI: Wait for licenses Name: syn_wait_for_licenses Default: 0 (minutes)	Sets the number of minutes to wait for Design Compiler licenses. Causes Design Compiler to be invoked using: '-checkout Design-Compiler -wait <minutes>/' In Design Compiler, scripts also wait before giving up when checking out needed licenses, such as Ultra, Test-Compiler, and so on.
GUI: Alib library analysis path Name: alib_library_analysis_path Default: ./	Sets the alib_library_analysis_path variable for synthesis. For more information, see the man page in Design Compiler.
GUI: Alib library analysis cache Name: alib_library_analysis_cache Default: ./	Sets the alib_library_analysis_cache variable for synthesis. For more information, see the man page in Design Compiler.
GUI: Include params in top design name Name: include_params_in_top_design_name Default: 1 (true)	■ 1 (true): Parameters on the top-level design cause the design name to be modified as specified in the Design Compiler variable, template_parameter_style. ■ 0 (false): The top-level design name is unmodified by its parameter settings.
Multiple Port Nets GUI: Do not fix multiple port nets Name: fix_multiple_port_nets_default Default: 1 (true)	When true, causes "set_fix_multiple_port_nets -default" to be used in Design Compiler, so no buffers will be inserted to fix multiple port nets.
Multiple Port Nets GUI: Buffer feedthroughs Name: fix_multiple_port_nets_feedthroughs Default: 0 (false)	When true, causes "set_fix_multiple_port_nets -feedthroughs" to be used in Design Compiler.
Multiple Port Nets GUI: Buffer pins driving multiple outputs Name: fix_multiple_port_nets_outputs Default: 0 (false)	When true, causes "set_fix_multiple_port_nets -outputs" to be used in Design Compiler.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Multiple Port Nets GUI: Duplicate logic constants Name: fix_multiple_port_nets_constants Default: 0 (false)	When true, causes “set_fix_multiple_port_nets -constants” to be used in Design Compiler.
IP-XACT	
Output Controls GUI: Schema version for IP-XACT output files Name: xml_schema_version Default: 1685-2014	Indicates SPIRIT IP-XACT schema version against which the files are written. Possible values are as follows: ■ 1.0: Uses SPIRIT 1.0 ■ 1.1: Uses SPIRIT 1.1 ■ 1.2: Uses IP-XACT 1.2 ■ 1.4: Uses IP-XACT 1.4 ■ 1.5: Uses IP-XACT 1.5 ■ 1685-2009: Uses IEEE 1685-2009 ■ 1685-2014: Uses IEEE 1685-2014
Output Controls GUI: Include component synthesis constraints Name: include_synthesis_constraints_in_components Default: 0 (false)	When false, synthesis constraints are not included in generated component files.
Output Controls GUI: Include defaulted parameters in design files Name: include_defaulted_params_in_design Default: 0 (false)	■ 1 (true): Parameters that are set to default values are written to the appropriate IP-XACT design file. ■ 0 (false): Only parameters that are set to non-default values are written to the appropriate IP-XACT design file.
Output Controls GUI: Include unused bus interfaces Name: include_unused_interfaces Default: 0 (false)	When true, unused (unconnected) bus interfaces are included in the component definition.
Output Controls GUI: Allow Broadcast Interface Connections Name: allow_broadcast_connections Default: 0 (false)	When true, allow one-to-many interface connections in the IP-XACT design files, where the pins on the one-side are all outputs.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Output Controls GUI: Include fileSet information Name: include_fileset_information Default: 0 (false)	When true, fileSet information is included in the component definition.
Output Controls GUI: Include unused signals in bus interface Name: include_unused_ports_in_interface Default: 0 (false)	When true, unused bus interface signals (signals with no corresponding component signal) are included in the signalMap of the corresponding busInterface element. These appear as signalName elements that have no value.
Output Controls GUI: Write IP-XACT components Name: write_spirit_components Default: 0 (false)	When false, IP-XACT components are not rewritten when writing a hierarchical IP-XACT component or design file. Instead, references to these components are handled through the standard VLVN (vendor, library, name, version) mechanism.
Output Controls GUI: Include accessHandle information Name: include_accesshandle_information Default: 1 (true)	When true, accessHandle information is included in the component definition.
Output Controls GUI: Use absolute path writing file/fileSet dependencies Name: write_absolute_dependency_path Default: 0 (false)	When true, use an absolute path writing file/fileSet ipxact:dependency.
Output Controls GUI: Include Defines in IP-XACT component Name: include_defines_in_fileset Default: 0 (false)	If the value of this preference parameter is true, the defines that are set are written out in IP-XACT. The defines can be set in the design attribute 'StaticDefineExprs' for coreBuilder, coreConsultant, and coreAssembler. The defines can also be set in the LoadDesigns activity parameter 'CompileDefines' in coreBuilder.
Output Controls GUI: Include undefined macros in IP-XACT Name: include_undefined_macros_in_ipxact Default: 0 (false)	If the value of this preference parameter is true, then undefined macros will also be written out in IP-XACT.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Output Controls GUI: Include Doc attributes Name: include_doc_attrs Default: 0 (false)	The Control variable enables exporting of Documentation attributes in the component XML. The Doc attrs are only exported for Designware IP in coreConsultant and coreAssembler. ■ DocGroup: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocMaster: coreBuilder (addressBlock, register, port, param), coreConsultant (addressBlock, register, port) ■ DocOffset: coreBuilder (register), coreConsultant (register) ■ DocGenerateIf: coreBuilder (port), coreConsultant (N/A) ■ DocRegisterResetValue: coreBuilder (register, registerField), coreConsultant (register, registerField) ■ DocDescriptionField: coreBuilder (register, registerField, port, param), coreConsultant (register, registerField, port, param)
Schema Validation GUI: Input errors should be Name: xml_schema_validation Default: 1 (warnings)	Indicates how to deal with XML schema validation errors when reading XML files: ■ 0 (suppressed): All errors are suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Schema Validation GUI: Output errors should be Name: xml_schema_output_validation Default: 0 (suppressed)	Indicates how to deal with XML schema validation errors when writing XML files: ■ 0 (suppressed): Validation is suppressed ■ 1 (warnings): Treat errors as warnings; continue processing ■ 2 (errors): Treat errors as errors; processing stops
Assembly GUI: Enable Automatic Connections for IP-XACT Bus Definitions Name: auto_connect_spirit_definitions Default: 1 (true)	When true, indicates that interfaces whose definition comes from an IP-XACT bus definition (as opposed to a coreBuilder TCL interface definition) can be connected automatically.
Assembly GUI: Use IP-XACT ad-hoc Connection Names as Signal Names Name: use_ipxact_adhoc_names_as_signal_names	When true, indicates that signals defined through ad-hoc connections should be named based on the name specified in the ad-hoc connection element.

Table 7-3 Edit > Preferences Menu (continued)

Preference	Description
Default: 0 (false)	
Assembly GUI: Treat single pin ad hoc connections as a connection to 'open' Name: ipxact_treat_single_pin_adhoc_as_open Default: 0 (false)	When true, indicates that single pin ad-hoc connections will be treated as a connection to 'open' for both reading and writing IP-XACT design files.
Assembly GUI: Treat unconnected interfaces as unused in IP-XACT designs Name: ipxact_mark_unconnected_interfaces_as_unused Default: 0 (false)	When true, unconnected interfaces on an IP-XACT component will be marked as unused when reading a design file which does not make a connection to the interface.

7.4 Working with Verification IP

Your coreKit may contain verification IP. The coreConsultant design flow for verification IP is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and Synthesis activities are irrelevant.

The verification IP may be in the form of an HDL model, C model, timing model, or some other format. Non-HDL verification IP requires a top-level VHDL or Verilog wrapper.

The coreConsultant design flow for a verification IP block typically consists of the following steps:

1. Install the coreKit.

Use coreConsultant to install the coreKit as described in [“Installing coreKits”](#) on page 39.

2. Create a workspace as described in [“Creating a Workspace”](#) on page 43.

3. Execute the Specify Configuration activity.

If the verification IP is configurable through HDL design parameters, the Specify Configuration activity configures the HDL and writes out configuration-specific files to your workspace/src directory. These files can also be found in the workspace/export directory.

The verification IP may also have configuration options that are not implemented as HDL parameters. In such cases, your IP may provide custom coreConsultant configuration activities. Complete any custom configuration activities according to the options specified by your IP documentation.

4. Implement the verification IP as needed for your design verification strategy.

7.5 Working with Hard Macros

Your coreKit may contain a hard macro, which by nature is not configurable and not synthesizable. Therefore, the coreConsultant design flow for a hard macro is a subset of the steps required for a synthesizable soft core. In particular, the Specify Target Technology and Synthesis activities are irrelevant. The top-level view of the IP block must be in VHDL or Verilog format. Therefore, a hard macro requires a top-level VHDL or Verilog wrapper.

The coreConsultant design flow for a hard macro consists of the following steps:

1. Install the coreKit.

Use coreConsultant to install the coreKit as described in [“Installing coreKits”](#) on page 39.

2. Create a workspace as described in [“Creating a Workspace”](#) on page 43.

3. Execute the Specify Configuration activity.

Although individual hard macros are not configurable, the core may allow you to select from a set of implementations of the hard macro. If the selection is implemented with HDL parameters, the Specify Configuration activity configures the HDL and writes out configuration-specific files to your workspace/src directory. This file can also be found in your workspace/export directory.

4. Execute verification activities.

The coreKit may provide a TRE or other custom verification-related activities for the macro.

Verification may involve a verification model for the macro. Execute any design-specific verification activities as instructed by the core’s documentation.

5. Implement the hard macro in your application design.

7.6 Dumping Debug Information

The menu entry, **File > Build Debug Tar-file**, is used to capture debug information for the Synopsys Support Center. This menu item creates the file <working_dir>/debug.tar.gz. This debug file includes this optional information (set in the Build Debug Tarfile dialog):

- A batch script for recreating the workspace
- Inclusion of the current workspace contents
- Synthesis and simulation log files (if available)
- User environment variable settings
- Inclusion of DESIGNWARE_HOME listing
- Output from the existing .debug_info. command
- The results of an environment check (if available)
- Tool version information
- Indication of whether design source files are encrypted or in “source” form
- A “Notes:” description of the problem

7.7 Documentation Generation

The “report” generation process is based on DocBook instead of HTML. This allows documentation for I/O definition, parameter definition, and memory maps to be generated in HTML, RTF, and MIF formats from the DocBook source. The coreConsultant tool automatically generates HTML reports from this source, and allows users to output to other formats using standard DocBook utilities.

DocBook output is enabled by default. To disable DocBook file generation, choose the **Edit > Preferences... “Documentation Generation”** menu item and dialog to control the elements shown in [Table 7-4](#) (for more information about preferences, see “[Editing Tool Setup and User Preferences](#)” on page 177):

Table 7-4 Documentation Generation Tool Preferences

GUI Label	Description
Generated Documentation Format Default: DocBook	If the DocBook format is chosen, documentation is generated in DocBook format and then translated to HTML for coreAssembler reports. Both file types (.html, .xml) appear in the report directory. If the HTML format is chosen, only the old non-configurable HTML-style reports are generated.
Port Documentation Attributes	Default Values: DocPortWidth PortDirection *DocDescription *Registered *SynchronousTo *DocGenerateIf *DocDescriptionField
Parameter Documentation Attributes	Default Values: Value *Name *DefaultValue *Values *DocEnabled *DocDescriptionField *DocDescription
Register Documentation Attributes	Default Values: GlobalAddressOffset RegisterSize MemoryAccess *DocDescription *RegisterResetValue *DocEnabled
Register Field Documentation Attributes	Name MemoryAccess *DisplayName *RegisterResetValue *DocEnabled *DocDescription

The attribute lists control the contents of the DocBook documentation. Attributes appearing without a leading asterisk “*” get their own column in the generated tables. Attributes with “*” are all included in a single “Description” column with appropriate labels, to prevent wide tables.

7.8 Table-Driven Data Import/Export

The coreTools support import and export of data in a table based format to enable user flows that are driven from spreadsheets. This provides both an ease-of-use advantage (no customer conversion to coreTools commands required) as well as a performance benefit as table processing is generally faster than command processing. The table read/write commands are available in batch mode and are also available through the **File > Table...** menu.

7.8.1 General Table Formatting (All coreTools Applications)

Table files can contain one or more tables. Within a table file, tables can be named or anonymous. Named tables provide two additional capabilities that are not available with anonymous tables:

- Named tables can contain blank lines
- Named tables can be extracted from multi-table files and processed without processing of other named or anonymous tables.

All tables have the following general format: `<table name><header row><data>`, where the `<table name>` is optional. When reading a table file, a table is assumed to begin at the first non-blank row. If that row contains exactly two cells, the first of which has a value of "Table", then it is assumed to be a table name. Otherwise it is assumed to be the row containing the column headers (that is, the `<header row>`). If there is a table name, the next non-blank line is assumed to be the `<header row>`. The columns which have entries in the `<header row>` define the set of columns for the table. The `<data>` section of the table begins at the next non-blank line following the `<header row>`. The `<data>` section ends at the next blank line beyond its starting point, unless the table is named and the table is being read with the `-blanks` option specified.

All of the table reading commands locate and process tables in a consistent fashion, and share a number of common arguments:

- **<table file>**: The name of the table file to be processed.
- **-tables <table names>**: Indicates the names of the tables to be processed in the given table file. Only tables with the indicated names will be processed. If this option is not specified, then all tables in the table file will be processed (and must therefore be supported by the same table reading command).
- **-columns <column data>**: Defines static columns that should be assumed to exist in all processed tables. The `<column data>` is a list of two-element lists where the first element is a column name and the second element is the value to appear in that column for every table row. If a column is defined in this manner and explicitly exists in the table, then a table reading error occurs.
- **-map <column map>**: Defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names seen in the table to the names expected by the table reading command. This enables user-defined column names while still providing the table reader with known names. The `<column map>` is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table reading command.
- **-separator <column separator>**: Defines the character used to separate columns in the table file. If not specified, a tab character is assumed.

The table writing commands also share a number of common arguments:

- **<table file>**: The file into which the table should be written.
- **-name**: The name for the table file. If specified, a `<table name>` row will be included to identify the table.

- **-append:** If specified, the table is appended to the indicated file. Otherwise the file would be overwritten.
- **-map <column map>:** Defines column header name mapping. Different tables have different requirements regarding the names of columns in the table. This option can be used to map from the names desired in the table to the names expected by the table writing command. This enables user-defined column names within the written tables. The <column map> is a list of two-element lists where the first element is the name of a column in the table and the second element is the name expected by the table writing command.
- **-separator <column separator>:** Defines the character used to separate columns in the table file. If not specified a tab character is assumed.
- **-columns <column names>:** Indicates a list of columns whose values are expected to be identical across all table rows and which should therefore be excluded from the table. If the table writer determines that the values are not identical, then table writing fails.

7.8.2 Table Commands

7.8.2.1 read_attribute_table

This command provides basic attribute table read capability:

```
read_attribute_table <table filename> \
[-tables <table names>] \
[-columns <column name value pairs>] \
[-map <column mapped name pairs>] \
[-separator <column separator>] \
[-create | -update] \
[-loops]
[-blanks]
[-comment_indicator <table comment indicator>]
```

This command is used to set attributes on objects in the coreTools data model. Attribute tables may have the columns listed in [Table 7-5](#).

Table 7-5 Attribute Table Columns

Column Name	Optional	Description
Type	No	Indicates the object type
Name	No	Indicates the object name
<attribute name>	Yes	Indicates the name of the attribute to be set. An arbitrary number of these columns is allowed.

This command is used to set the indicated attributes on the indicated objects. Objects are located using the item type and item name. When a table contains objects of multiple types, scoping is handled automatically when switching between object types that have a parent-child relationship. For example, ‘port’ lines appearing after a ‘design’ line are assumed to indicate ports within that design. Scoping is supported for the following object type transitions:

- design --> port
- memoryMap --> addressBlock
- addressBlock --> register
- addressBlock --> registerArray
- registerArray --> register
- register --> registerField

As each of these transitions occurs, the current context is changed by pushing the prior element onto the context stack. When the reverse transitions occur, elements are popped from the stack as appropriate. This scoping means that names specified for items can always be simple local names without context in the name itself.

If neither -create nor -update option is specified, each line is assumed to refer to an object that already exists in the data model. If -create is specified, each object is assumed to not exist, and will be created before setting any attributes on it. If -update is specified, a search will be done and the object will be created only if it does not already exist.

If the -loops option is specified, each name will be scanned for loop syntax. The loop syntax is {<loop var>:<start index>...<end index>}. When seen, the line will be replicated for each inclusive index value. Each attribute value will be scanned for \$<loop var> and that string will be replaced with the loop index value. When a line with loop syntax is used for scoping, all scoped objects will be created within each replicated scoping object. For example, [Table 7-6](#) would result in two registers being created, each with two fields, F1 and F2.

Table 7-6 Example Creating Two Registers with Two Fields F1 and F2

Type	Name	Size	Description
register	Interrupt{i:0...1}	32	Interrupt \$i
field	F{j:0...1}	16	Field \$j in interrupt \$i

7.8.2.2 write_attribute_table

This command provides basic attribute table write capability:

```
write_attribute_table <table filename> \  
[-name <table name>] \  
[-append] \  
[-separator <column separator>] \  
[-comment_indicator <table comment indicator>] \  
[-map <column name map pairs>] \  

```

```
[-columns <column name value pairs>] \  
-items <list of items> \  
-attributes <list of attributes> \  
-defaults
```

This command writes attribute data in table format to the indicated file. The resulting table will include a row for each specified item and a column for each specified attribute, plus columns for the object name and type (unless removed with `-columns`). If there are multiple types of objects in the list to be written, only those attributes that apply to an object type will be written for a given object. If the `-defaults` option is not specified, attributes whose value matches their default will not be written.

8

Customizing a Technology Library

This chapter describes the customization of a technology library.

Under certain circumstances, you may wish to customize your technology library for use with coreConsultant by setting attributes on the library or its individual cells. For example, you can disable the selection of certain library cells or make a certain cell representative for its function.

Although you apply these coreConsultant-specific attributes to the technology library .db file, which is then read into the Design Compiler, the Design Compiler simply ignores them. Only coreConsultant uses these special attributes to generate synthesis scripts.

To customize a technology library:

1. Invoke Design Compiler:

```
% dc_shell-t
```

2. Read the technology library .db file:

```
dc_shell-t> read lib_file_name.db
```

3. Apply to the library and its cells the attributes listed, respectively, in [“Library Customization Attributes”](#) on page 201 and [“Cell Customization Attributes”](#) on page 202.

Use the following syntax:

```
dc_shell-t> set_attribute lib_name attr_name attr_value
```

```
dc_shell-t> set_attribute cell_name attr_name attr_value
```

For example, to specify a library cell as the representative high drive strength combinational cell, use the following commands:

```
dc_shell-t> set_attribute cell_name representative_combinational true
```

```
dc_shell-t> set_attribute cell_name representative_drive_strength high
```

coreConsultant then selects the specified cell when evaluating the {=select_by_class “combinational” “high”} formula.

4. Write out the library to a new file:

```
% write_lib -format db new_lib_file_name.db
```

5. Perform (or repeat) the coreConsultant Specify Technology Library activity to (re)analyze the modified technology library. Make sure you specify the new library file name.
6. Re-execute any previously executed steps in the design flow.

After you set the library and cell attributes, whenever coreConsultant generates synthesis strategy scripts, it considers the attributes that you have set on the technology library. Once again, these new attributes have no direct affect on Design Compiler.

This chapter contains the following sections:

- [“Library Customization Attributes”](#) on page 201
- [“Cell Customization Attributes”](#) on page 202

8.1 Library Customization Attributes

Set the attributes listed in [Table 8-1](#) to customize your technology library.

Table 8-1 Library Customization Attributes

Attribute	Description
default_clock_uncertainty	Establishes a default clock skew uncertainty for the library: ■ aux_timing_script_during_setup ■ aux_area_script_during_setup ■ aux_timing_script_before_initial_compile ■ aux_area_script_before_initial_compile ■ aux_timing_script_before_incr_compile ■ aux_area_script_before_incr_compile These attributes allow you to execute a technology library customization script during or before a specific phase of an area- or timing-driven compile: ■ During technology library analysis. ■ Before the initial compile. ■ Before an incremental compile. The coreConsultant first looks for the script in the current working directory, then in the path or paths specified in your Design Compiler search_path variable. The application then copies the script to your user workspace and executes it at the specified time. For example, if you specify aux_timing_script_before_initial_compile on a library, coreConsultant executes the specified script before using that library to execute the initial compile of a timing-driven synthesis.
ignore_design_rules_during_initial_compile	Some libraries use overly restrictive design rules that reduce QoR, especially if those rules are applied during the early phases of optimization. Setting ignore_design_rules_during_initial_compile to true defers design rule fixing until the later compile phases. This improves QoR for some libraries.
use_fast_delay_calculation	This attribute enables you to specify exactly when to use fast delay calculation for the specified library. The legal values are: ■ Initial – The initial mapping phase only. ■ Pre-layout – All phases prior to layout. ■ Never – Does not use fast delay calculation. Using fast delay calculation for early phases of optimization can decrease synthesis runtime without adversely affecting QoR. By default, this attribute disables high timing effort and uses fast delay calculation for the early phases of synthesis.

8.2 Cell Customization Attributes

Set the attributes listed in [Table 8-2](#) to customize the use of individual cells within your technology library.

Table 8-2 Cell Customization Attributes

Attribute	Description
representative_combinational	Overrides coreConsultant's automatic selection of a representative combinational cell when evaluating the {=select_by_class "combinational"} formula. The application instead uses this cell as the representative combinational cell to generate electrical constraints.
representative_sequential	Overrides coreConsultant's automatic selection of a representative sequential cell when evaluating the {=select_by_class "sequential"} formula. The application instead uses this cell as the representative sequential cell to generate electrical constraints.
representative_drive_strength	Specifies a cell as representative for the specified drive strength. Valid values are low, median, or high.
representative_cell	Specifies a cell as representative for its function. For example, setting representative_cell on a nand2 cell, selects this cell during evaluation of {=select_by_function "nand2"}. Setting representative_drive_strength on the same cell (to low, for example), selects the cell during evaluation of {=select_by_function "nand2" "low"}.
dont_use_initial_timing	Prevents Design Compiler from using this cell during the initial compile of timing-driven synthesis.
dont_use_initial_area	Prevents Design Compiler from using this cell during the initial compile of area-driven synthesis.
dont_use_incremental_timing	Prevents Design Compiler from using this cell during incremental compiles of timing-driven synthesis.
dont_use_incremental_area	Prevents Design Compiler from using this cell during incremental compiles of area-driven synthesis.